

**DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO**

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#01

Roll No. _____ 20ES062 _____ Submission Date: _____

LAB PERFORMANCE INDICATOR	SUBJECT KNOWLEDGE	DATA ANALYSIS AND INTERPRETATION	ABILITY TO CONDUCT EXPERIMENT	PRESENTATION	CALCULATION AND CODING	OBSERVATION /RESULTS	SCORE

Introduction to AVR Assembly Programming and AVR Studio

PERFORMANCE OBJECTIVE:

After successful completion of this lab, students will be able to:

- Introduce the AVR Microcontrollers
- Become familiar with lab Tool – AVR Studio
- Understand the basics of Assembly Programming of AVR Microcontrollers
- Debug and observe the Assembly program

REQUIRED TOOLS AND SOFTWARES:

- ✦ WinAVR (GCC Compiler)
- ✦ AVR Studio 4 or any latest version
- ✦ AVR ATMEGA 16 Chip

AVR MICROCONTROLLER:

The AVR is a modified Harvard architecture 8-bit RISC single chip microcontroller which was developed by Atmel in 1996. The AVR was one of the first microcontroller families to use onchip flash memory for program storage, as opposed to one-time programmable ROM, EPROM, or EEPROM used by other microcontrollers at the time. AVR microcontrollers are obtainable in three categories.

Tiny AVR: This microcontroller has Less memory, small in the size, appropriate only for simpler applications.

Mega AVR: This microcontroller is the most popular ones having a good amount of memory up to 256KB, higher no. of inbuilt peripherals and fit for modest to difficult applications.

Xmega AVR: This microcontroller is used commercially for compound applications, which need large program memory and also high speed.



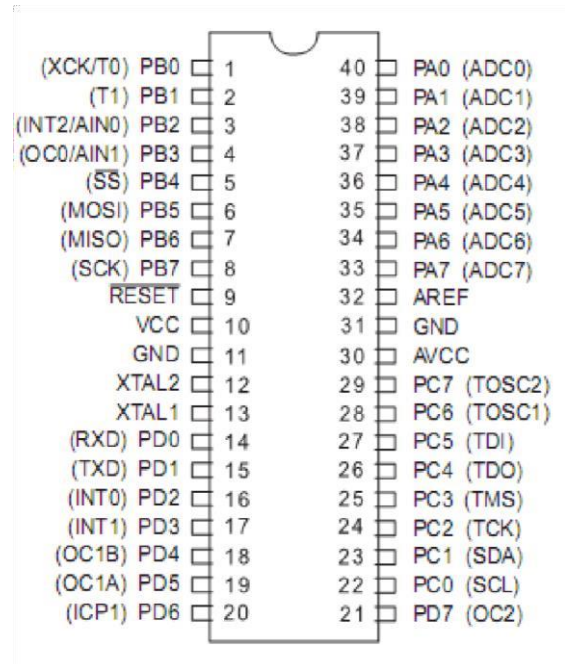
DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#01

ATMEGA16 Features:

- ✦ 8-bit Atmel with 16KB in-System Programmable Flash
- ✦ 32 × 8 General Purpose Working Registers
- ✦ Up to 16MIPS Throughput at 16MHz
- ✦ 512 B EEPROM
- ✦ 1KB Internal SRAM
- ✦ Two 8-bit Timer/Counters and One 16-bit Timer/Counter
- ✦ 8 channels, 10 bit ADC
- ✦ 4 PWM Channels
- ✦ Two-wire Serial Interface
- ✦ Programmable Serial USART
- ✦ Master/Slave SPI Serial Interface
- ✦ 32 Programmable I/O Lines
- ✦ 0 - 8MHz (ATmega16L)
- ✦ 0 - 16MHz (ATmega16)



ATMEGA16 Pin diagram

AVR Programming:

There are various compilers available for programming AVR Microcontrollers, including AVR Studio, ATMEL Studio, Arduino etc. To program it using C-Language, we will use AVR Studio 4(or higher) with AVR GCC &WinAVR and for using Assembly Language we will use Atmel Assembler AVR.

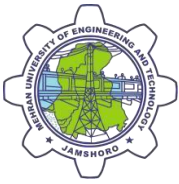
AVR Studio

AVR Studio is a free Integrated Development Environment (IDE) for developing different embedded applications based on 8-bit AVR microcontroller offered by Atmel AVR Studio works with the WinAVR avr-gcc compiler and contains built-in support for AVR ISP programming.

Step by Step Guide to AVR Studio:

Step 1

Open AVR Studio and click **New Project**. Select **AVR GCC or Atmel AVR Assembler** for the project type as shown in figure 1 and 2.



Lab Experiment#01

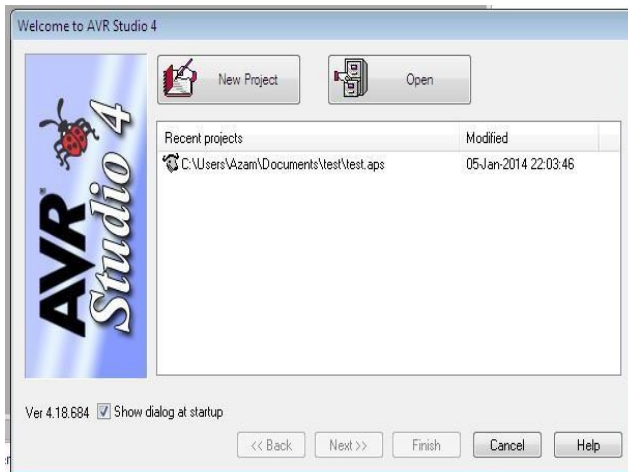


Figure 1: Select “New Project”

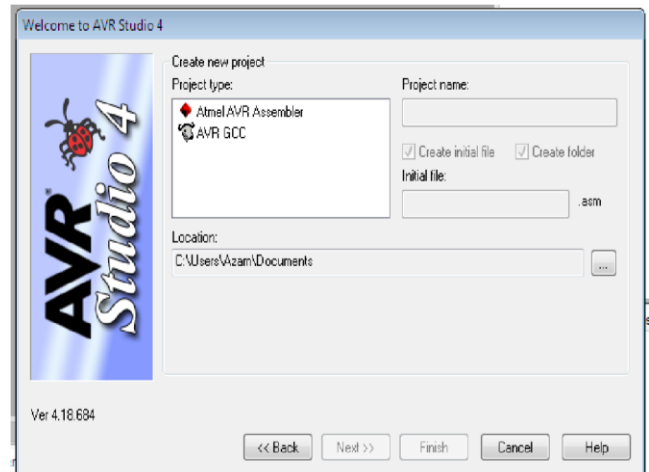


Figure 2: Select AVR Assembler or AVR GCC

Step 2

Enter the project name Click **Next >>**. In the Figure 3 below, we named our project “test2”. Do not click “Finish” yet. If you do accidentally click “Finish”, you will not be able to perform step 2 and will instead have to set the device by going to the “Project” menu and selecting “Configuration Options”.

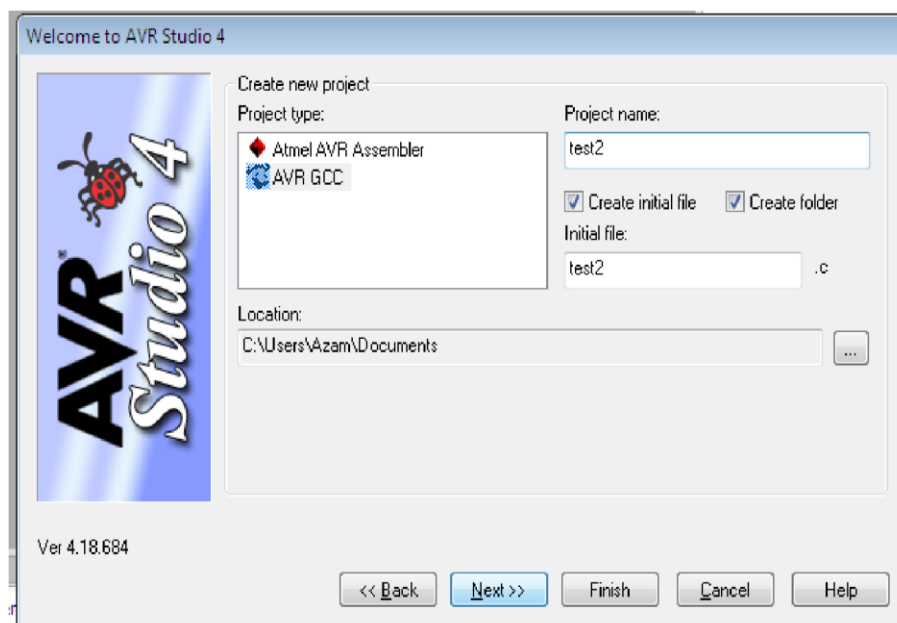
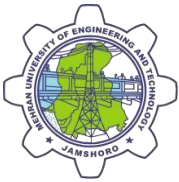


Figure 3: Insert a name for Project, Click NEXT

Step 3

Select **AVR Simulator** as the debug platform and then select the appropriate device for your target AVR. In our lab experiments, we will use ATmega16 or ATmega32. After click on finish button you will see AVR Studio Environment as shown in figure 5.



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#01

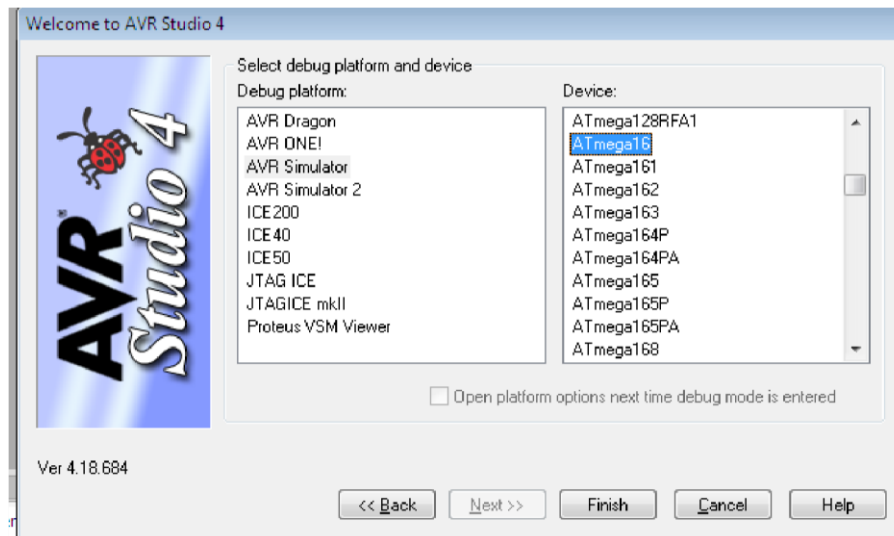


Figure 4: Select “AVR Simulator” and the “ATmega 16”, Click FINISH

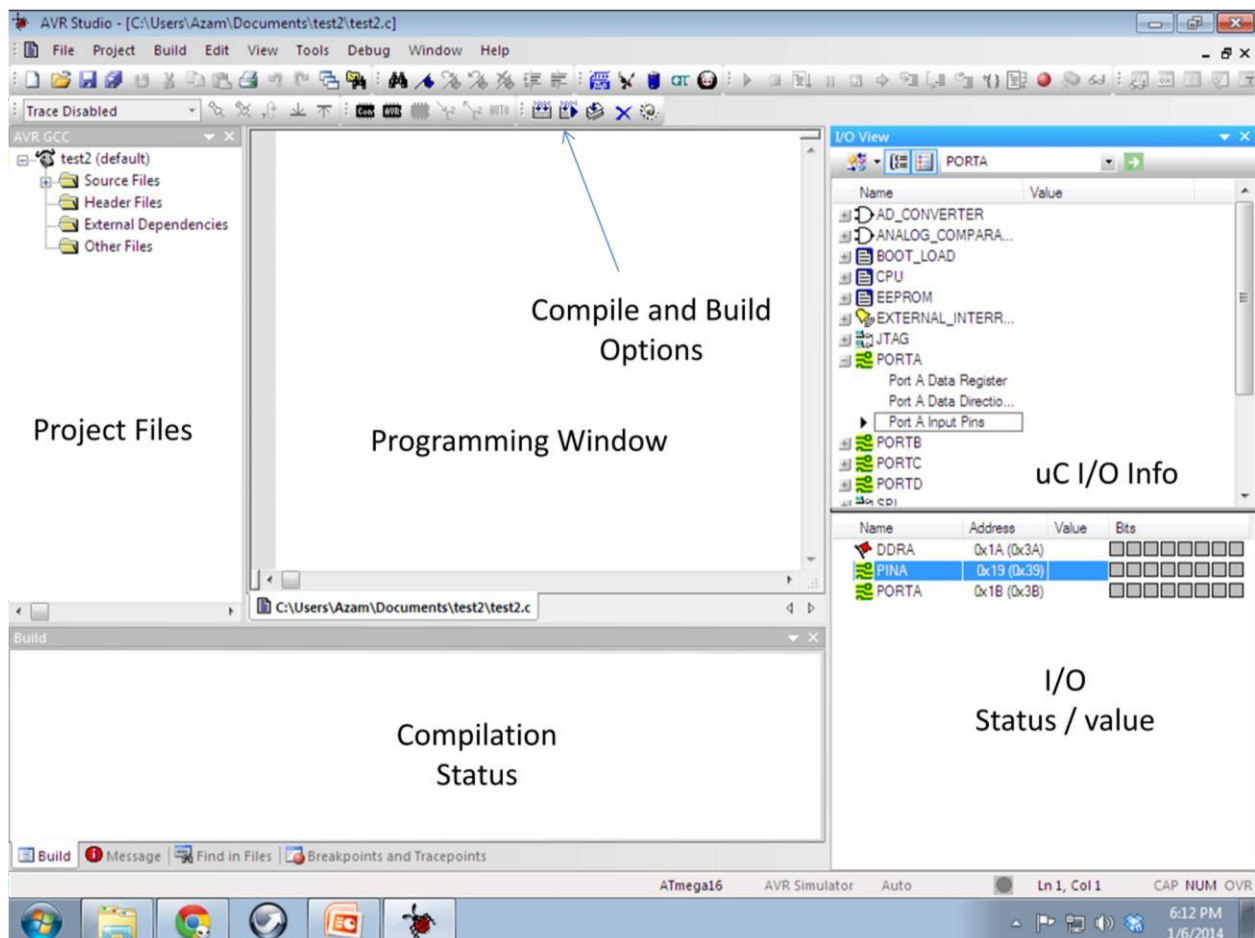
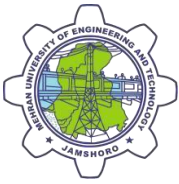


Figure 5: AVR Studio Environment

Step 4

AVR studio Environment is divided into different sections. In programming window, you can write your AVR C or Assembly language program. For a test, you can write the program given below into this window and don't forget to save the project.



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#01

Program: Assembly program to add numbers from 0 to 5.

```
.include "M16DEF.INC"
.org 0000
LDI R16,0x05
LDI R17,0x00
LOOP1:ADD R17,R16
DEC R16
BRNE LOOP1
AGAIN: JMP AGAIN
```

First save the program and then click on **Build button or Press F7** in order to check the errors and warnings in program as shown in below figure 6, here you can see 0 errors and 0 warnings. If there is any error in program then double click on it cursor will goes to that line and check what kind of error is. If you cannot rectify then call the instructor to rectify.

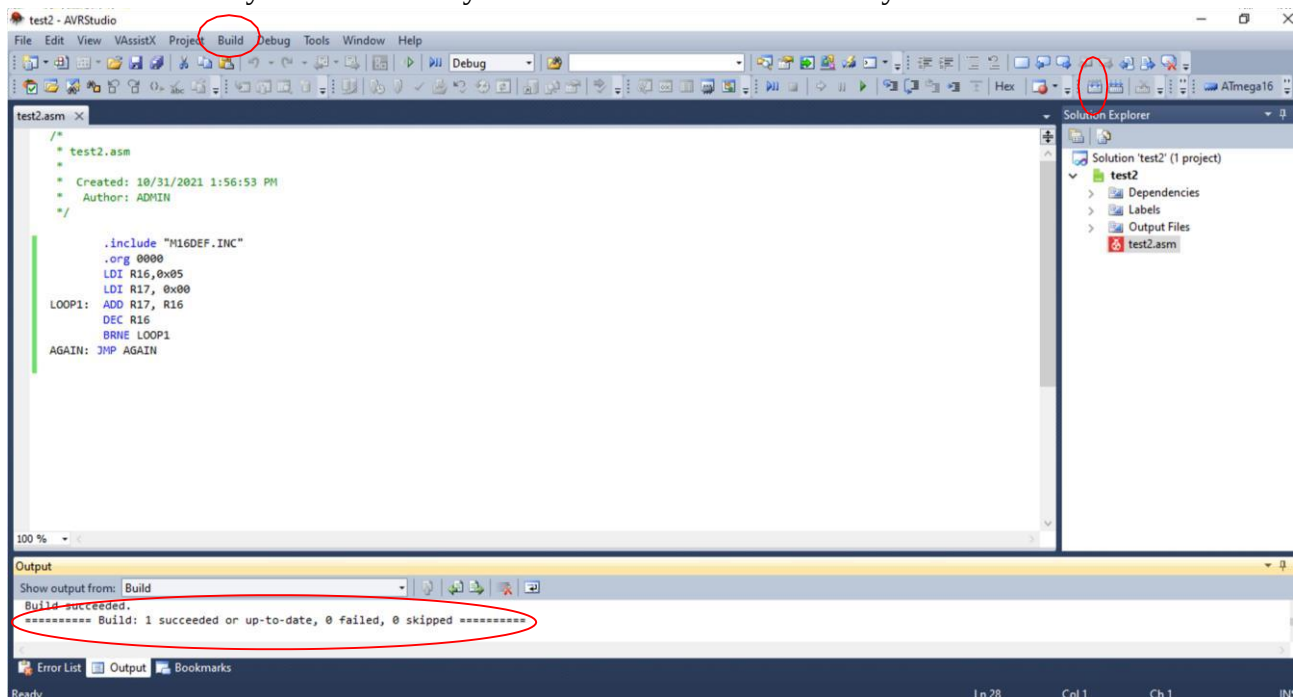
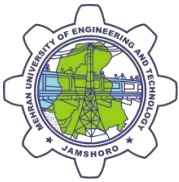


Figure 6: Build the Program

Step 5: Debug the program

Now you can debug the program by selecting "Start debugging and break" from the "Debug" menu or clicking on the "Start debugging and Break button" as shown in below figure 7.



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#01

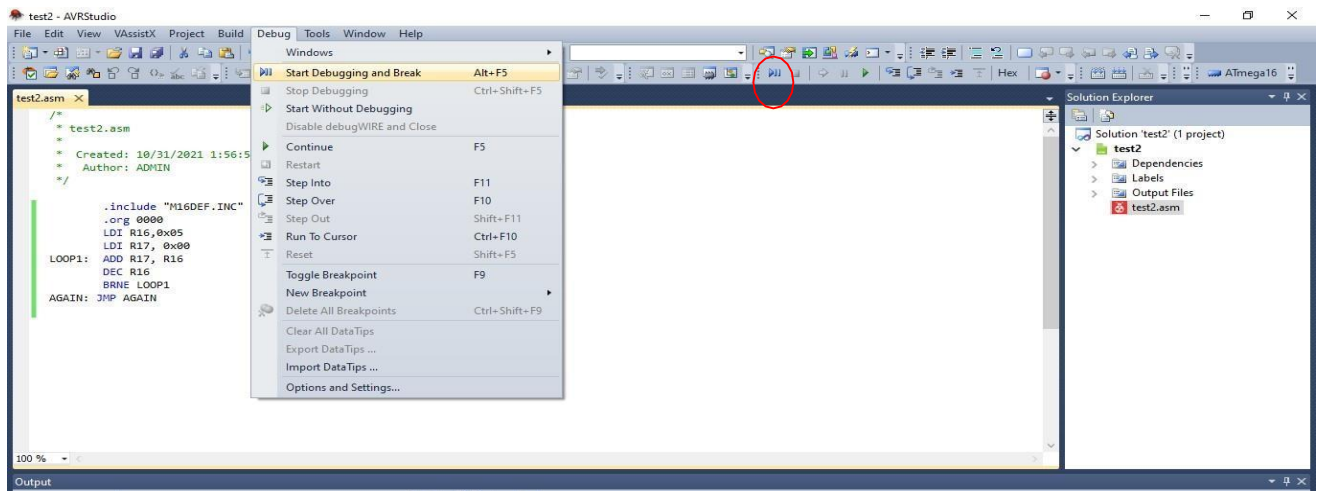


Figure 7: Debug the Program

Open your AVR Studio to full screen to have a better and complete view. The Right-hand side window shows you the Process window and the left side window shows the program code as shown in below figure 8. There are some other windows open. You may adjust the size of them to see well. Run the program step by step, you can observe the change of the values in the general purpose and Status registers.

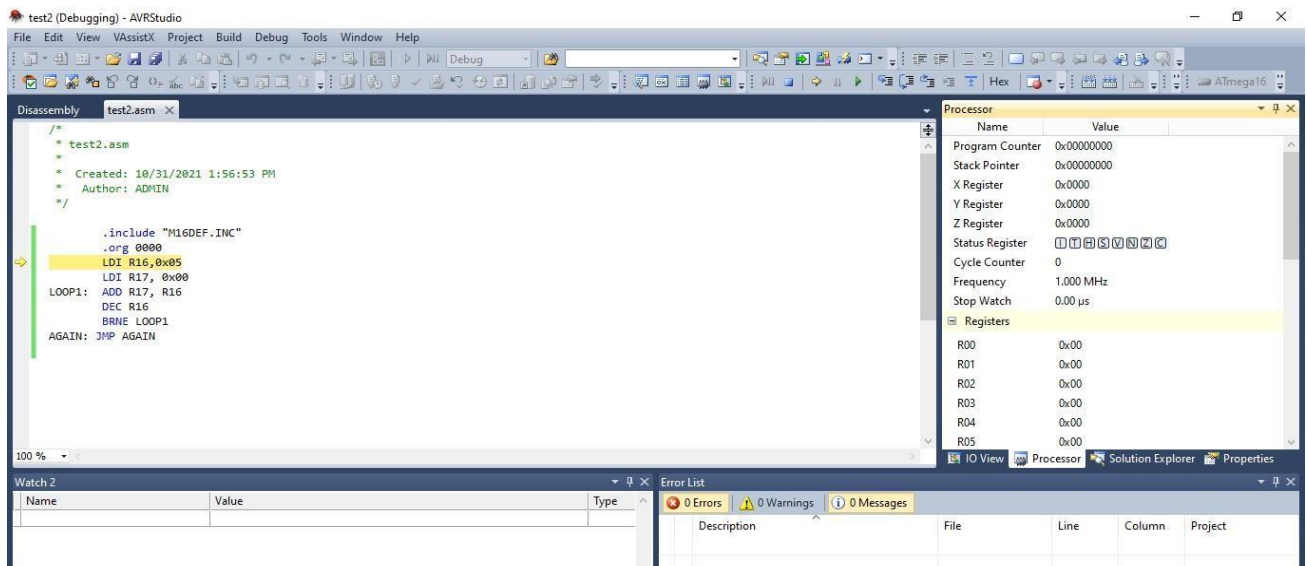
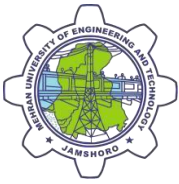


Figure 8: Debug window

Note: Analyze the program in debugging mode and see how it will work.

- Above figure 8, Section A lists the program code.
- Above Figure 8, Section B shows the registers as the program is debugged.
- Step through the code a single line at a time using the “step into” button





Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#01

- Open the Registers window (View->Registers Window). What are the values of the generalpurpose registers (R₀-R₃₁), for flags values check the status Register, you can also check the values of Stack pointer, the program counter and cycle counter? (See above figure 8)

Lab Tasks:

- Write an assembly code that calculates the sum as given below. Variable COUNT is saved in register R₂₀ and the sum is saved in register R₂₁. Also Attached the simulation window.

$$SUM = \sum_{i=0}^{COUNT=10} (i = 1 + 2 + 3 + 4 + \dots + COUNT)$$

The screenshot shows a simulation window with two main panes. The top pane displays the values of all 32 registers (R0-R31). The bottom pane shows the assembly code for a file named 'main.asm'.

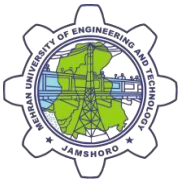
Register values (R0-R31):

R00 = 0x00	R01 = 0x00	R02 = 0x00	R03 = 0x00	R04 = 0x00	R05 = 0x00	R06 = 0x00	R07 = 0x00
R08 = 0x00	R09 = 0x00	R10 = 0x00	R11 = 0x00	R12 = 0x00	R13 = 0x00	R14 = 0x00	R15 = 0x00
R16 = 0x00	R17 = 0x00	R18 = 0x00	R19 = 0x00	R20 = 0x0B	R21 = 0x37	R22 = 0x00	R23 = 0x00
R24 = 0x00	R25 = 0x00	R26 = 0x00	R27 = 0x00	R28 = 0x00	R29 = 0x00	R30 = 0x00	R31 = 0x00

Assembly code (main.asm):

```
.include "M16DEF.INC"
.org 0000

    LDI R20,0X01
    LDI R21,0X00
SUM: ADD R21,R20
    INC R20
    CPI R20,0X0B
    BREQ STOP
    JMP SUM
$STOP: JMP STOP
```



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

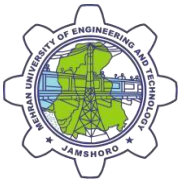
Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#01

02. Write and debug an assembly program to find the logical AND, & OR operations between the values 0xFF and 0xRR (where RR is your Roll Number in hexadecimal). Put the results into R5 and R6 register respectively. Attached the registers window showing the contents of registers after execution of program.

```
* _20ES062.asm
*
* Created: 2/25/2023 9:13:56 PM
* Author: DELL
*/ .include "M16DEF.INC"
.org 0000
LDI R16, 0xFF
LDI R17, 0xFF
LDI R18, 0x62
AND R16,R18
OR R17,R18
MOV R5, R16
MOV R6,R17
CLR R16
CLR R17
CLR R18
STOP: JMP STOP
```

Registers																																																																																															
R00	=	0x00	R01	=	0x00	R02	=	0x00	R03	=	0x00	R04	=	0x00	R05	=	0x62	R06	=	0xFF	R07	=	0x00	R08	=	0x00	R09	=	0x00	R10	=	0x00	R11	=	0x00	R12	=	0x00	R13	=	0x00	R14	=	0x00	R15	=	0x00	R16	=	0x00	R17	=	0x00	R18	=	0x00	R19	=	0x00	R20	=	0x00	R21	=	0x00	R22	=	0x00	R23	=	0x00	R24	=	0x00	R25	=	0x00	R26	=	0x00	R27	=	0x00	R28	=	0x00	R29	=	0x00	R30	=	0x00	R31	=	0x00



Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#01

03. Write and debug an assembly program to find the sum of the values 0x80, 0x30, 0x20 and 0x15. Put the sum result into R3. Also subtract the sum result from 0xFF, put the results into R4 register. Attached the registers window showing the contents of registers after execution of program.

Registers

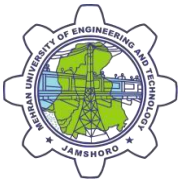
R00 = 0x00 R01 = 0x00 R02 = 0x00 R03 = 0xE5 R04 = 0x1A R05 = 0x00 R06 = 0x00 R07 = 0x00
R08 = 0x00 R09 = 0x00 R10 = 0x00 R11 = 0x00 R12 = 0x00 R13 = 0x00 R14 = 0x00 R15 = 0x00
R16 = 0xE5 R17 = 0x30 R18 = 0x20 R19 = 0x15 R20 = 0x1A R21 = 0x00 R22 = 0x00 R23 = 0x00
R24 = 0x00 R25 = 0x00 R26 = 0x00 R27 = 0x00 R28 = 0x00 R29 = 0x00 R30 = 0x00 R31 = 0x00

3RDPROGRAM main.asm

```
.include "M16DEF.INC"
.org 0000

    LDI R16,0X80
    LDI R17,0X30
    LDI R18,0X20
    LDI R19,0X15
    LDI R20,0XFF
    ADD R16,R17
    ADD R16,R18
    ADD R16,R19
    SUB R20,R16
    MOV R3,R16
    MOV R4,R20
    $TOP: JMP STOP
```

119 %



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#01

04. Write and debug an assembly program to find the sum of first 10+R even integers. (Where R is LSB of your Roll Number). Put the results into R7 register. Attached the registers window showing the contents of registers after execution of program.

```
.include "M16DEF.INC"
.ORG 0000
    LDI R16, 0x12; // roll number= 20ES062 10+2=12
    LDI R17, 0x20;
    LDI R18, 0x00;
SUM:  ADD R18,R17;
    DEC R17;
    DEC R17;
    DEC R16;
    BRNE SUM
STOP: JMP STOP;
```

Registers

R00 = 0x00 R01 = 0x00 R02 = 0x00 R03 = 0x00 R04 = 0x00 R05 = 0x00 R06 = 0x00 R07 = 0x00
R08 = 0x00 R09 = 0x00 R10 = 0x00 R11 = 0x00 R12 = 0x00 R13 = 0x00 R14 = 0x00
R15 = 0x00 R16 = 0x12 R17 = 0x00 R18 = 0x00 R19 = 0x00 R20 = 0x00 R21 = 0x00 R22 = 0x00
R23 = 0x00 R24 = 0x00 R25 = 0x00 R26 = 0x00 R27 = 0x00 R28 = 0x00 R29 = 0x00
R30 = 0x00 R31 = 0x00



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES
Lab Experiment#02

Roll No. 20ES062 _____ Submission Date: _____

LAB PERFORMANCE INDICATOR	SUBJECT KNOWLEDGE	DATA ANALYSIS AND INTERPRETATION	ABILITY TO CONDUCT EXPERIMENT	PRESENTATION	CALCULATION AND CODING	OBSERVATION /RESULTS	SCORE

AVR I/O PORT Assembly Programming and Simulation of the Program in Proteus

PERFORMANCE OBJECTIVE:

After successful completion of this lab, students will be able to:

- Understand the basics of Assembly Programming of AVR I/O Port Programming
- Become familiar with lab Tool – AVR Studio and **Proteus**

REQUIRED TOOLS AND SOFTWARES:

- WinAVR (GCC Compiler)
- AVR Studio 4 or any latest version - (for Programming)
- Proteus 8 or any latest version - (for Simulation)

AVR I/O Port Assembly Programming Basics:

Digital input output (I/O) is the basic feature supported by AVR micro controller. To facilitate digital input output, three registers are associated with each port of the micro controller.

- **Data Direction Register**– This register determines which of the pins will act as an output port, and which of them as input.
- **Data Output Register**-This register holds the Data output to the port.
- **Data Input Register**– Reads data from the port

Initialize ports as Input / Output:

- Following registers should be loaded with the described data to initialize ports as Input / Output:
 - `DDRx = 0xFF;` *// Initialize PORT as an Output*
and in binary we can write as: `DDRx= 0b11111111`
 - `DDRx = 0x00;` *// Initialize PORT as an Input*
And in binary we can write as : `DDRx= 0b00000000`

Getting / Writing Data on Ports:

`PINx` *// to get data from PORT (Input)*
`PORTx` *// to write on PORT (Output)*

x is the Port name i.e. A, B, C etc.

- For example, we want to set the output direction on PIN#03 of PORTB i.e PB3, then we can set it as



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES
Lab Experiment#02

DDRB= 0b00001000 OR DDRB= (1<<3) OR DDRB= 0x04

- For example, we want to write High on PB3, then we can write as
PORTB= 0b00001000 OR PORTB= (1<<3) OR PORTB= 0x04
- As the data cannot directly be loaded into SFRs so it must be loaded through allowed GPR's (i.e. $R_{16} \leq R \leq R_{31}$). This is further explained in Program Example # 01.

Example Program # 01:

An Assembly language program that turns on LED connected to PB2 of Port B.

Program:

```
.include "M16DEF.INC"
.org 0000
LDI R16, (1<<2)
OUT DDRB,R16                ; making pin#02 (PB2) of PORT B as output
LDI R17, (1<<2)              ; sending data High to PB2
OUT PORTB,R17
AGAIN: JMP AGAIN
```

Simulate the Program in Proteus

Now, for the execution of the program, we are going to use simulator- Proteus 8. A simulator is a software that is used to test the program written for another machine before loading it. It is good to simulate the program first because a bad program can damage the whole system. The point is that you are allowed to do mistakes without the risk of damaging your system. Proteus 8 is one such software. For simulation of the program in Proteus, follow these steps given below:

Step 1: Write the above program#01 in AVR Studio Assembler and build the Program. Note the Hex file creation and its path.

Step 2: Open Proteus. At the home page, click on New Project as shown in figure 1.

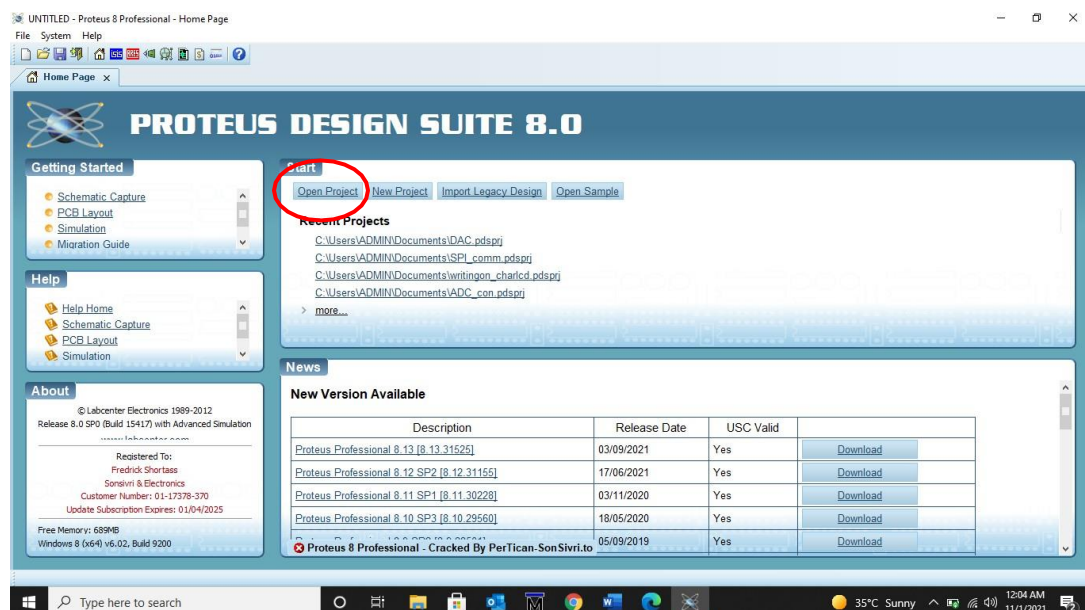


Figure 1: Home page of Proteus Design Suite 8.0



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES
Lab Experiment#02

Step 3: In the name section type the name you want to give to your project and then click on Next (figure 2).

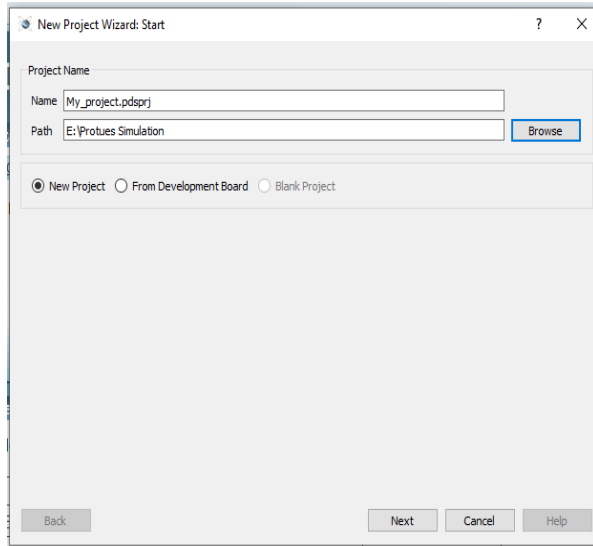


Figure 2: New Project Wizard: Start

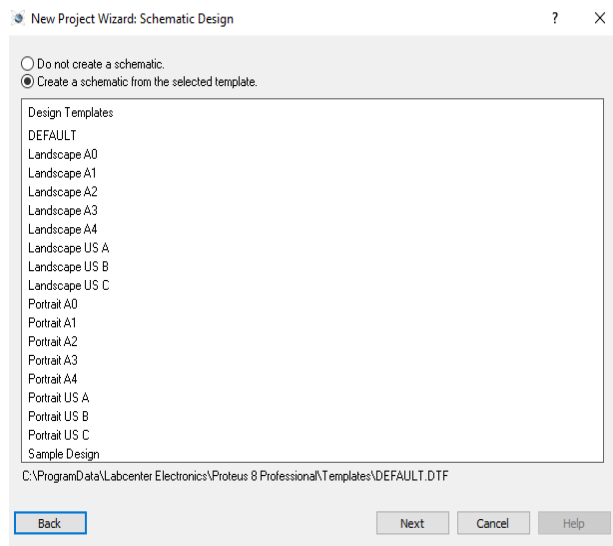


Figure 3: New Project Wizard: Schematic Design

Step 4: Click on a circle that says create a schematic from a selected template, and then select DEFAULT and click on Next (figure 3).

Step 5: Click on a circle that says do not create a PCB layout and click on Next (figure 4).

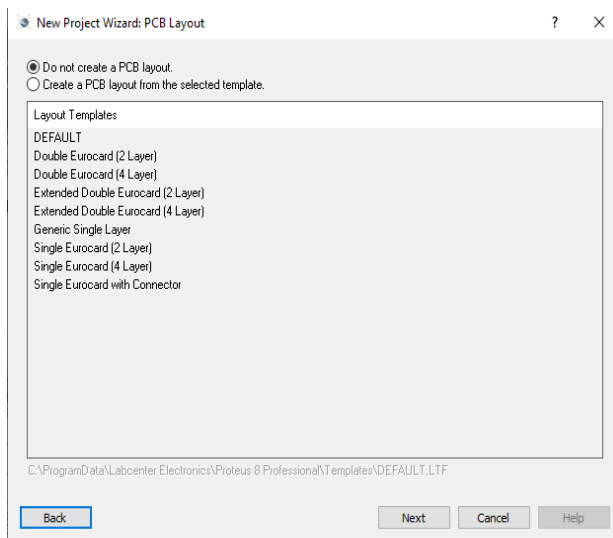


Figure 4: New Project Wizard: PCB layout

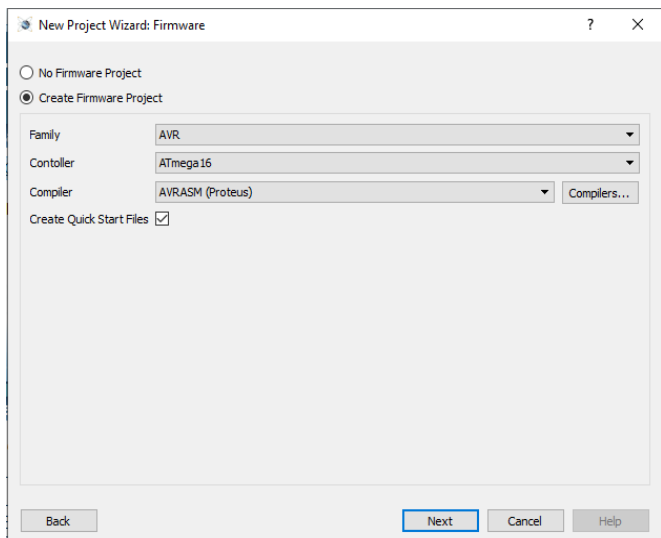


Figure 5: New Project Wizard: Firmware

Step 6: Click on firmware project, select the family-AVR, Controller- ATmega16/ ATmega32 and Compiler-AVRASM(Proteus) as shown in figure 5 and click on Next. It will show you summary of your project as shown in figure 6.

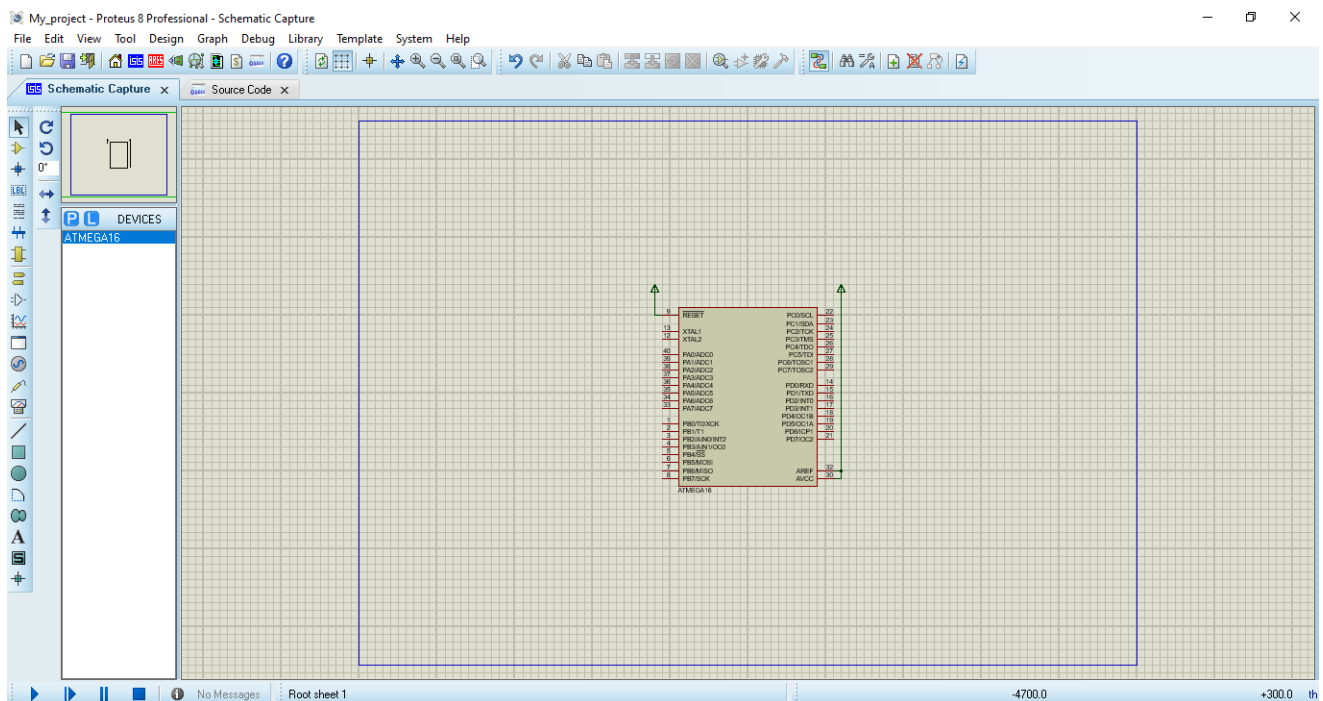
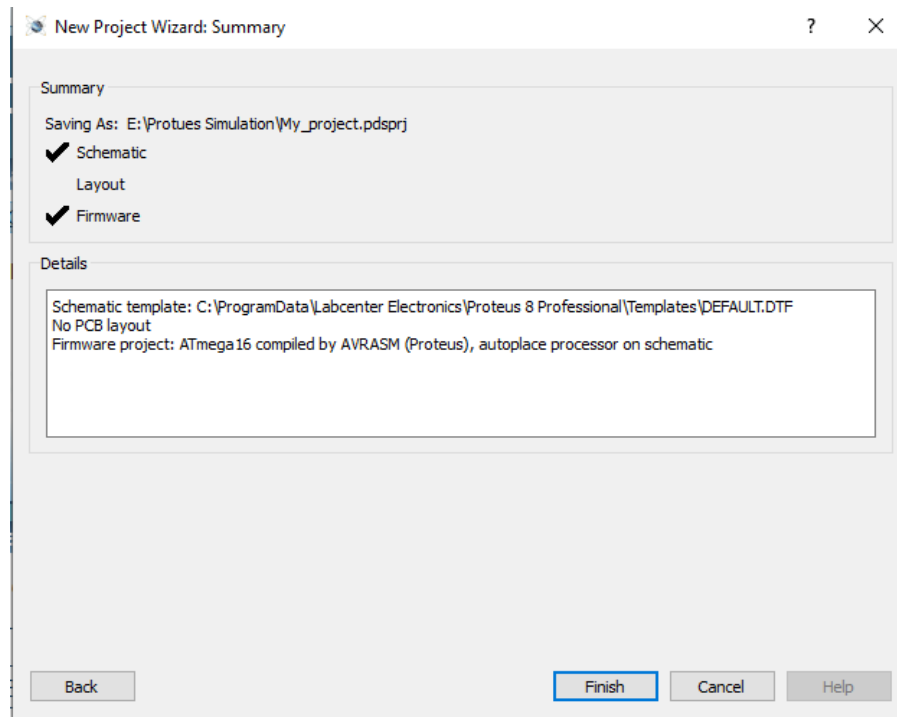


Figure 7: Proteus Workplace



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES
Lab Experiment#02

Designing of the Required Circuit in Proteus

Now, this is your workplace in Proteus. We are going to design a circuit in this page only. We can simulate the circuit as well. For designing the circuit, we have to add components which can be done by:

- In the left corner, there should be a label named device. Click on P button and under the Keywords block, type LED, click any color LED you want to add Double click on it as shown in figure 8. It should be added in your device list as you can see in figure 9.

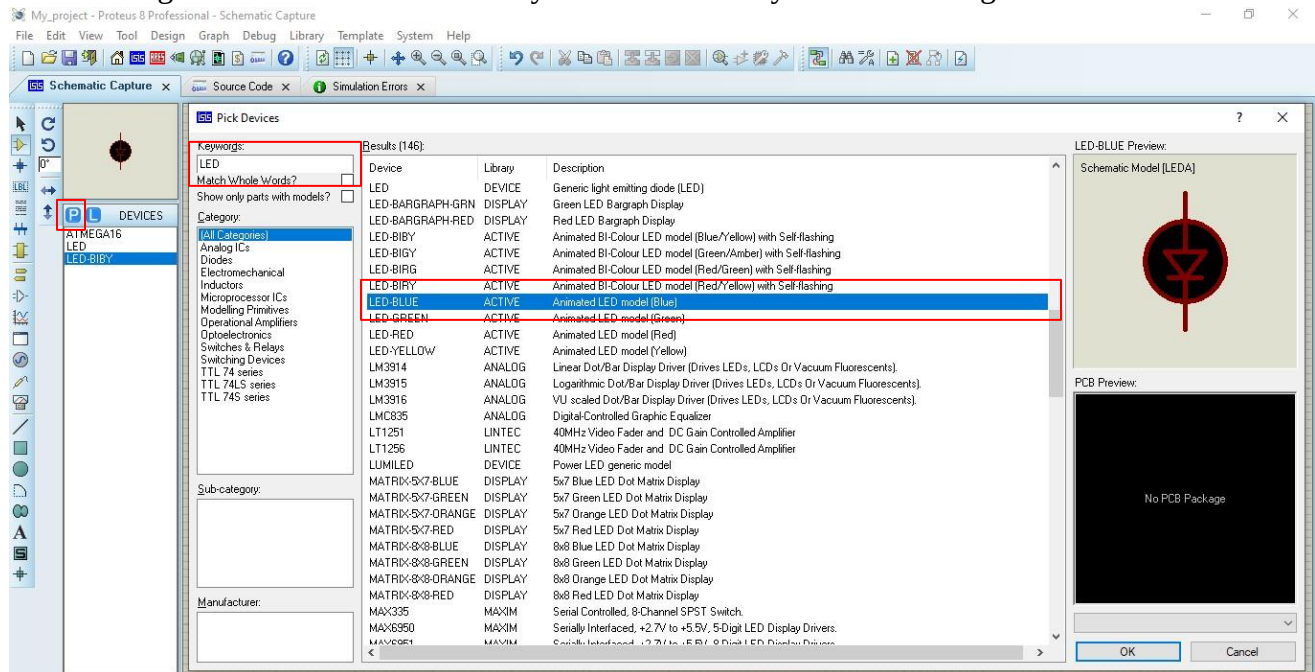


Figure 8: Pick Device and Components

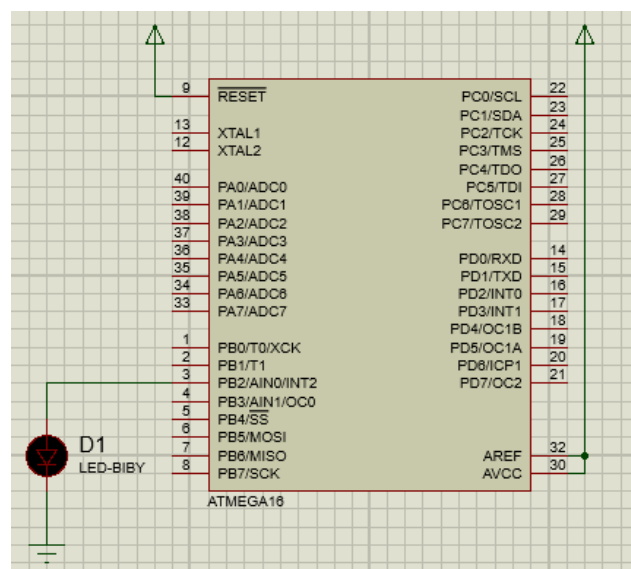


Figure 9: Required Schematic Diagram



Introduction to Embedded Systems (5th Semester) 20ES
Lab Experiment#02

- Click on LED and place anywhere you want to. For ground, there is an option in left side called terminal mode. Select ground and place anywhere you want to. Draw the schematic as shown above figure 9.

Simulation of the program in Proteus

Now our work is almost done here. Our circuit is for simulation is complete but for simulation of the program in Proteus, we have to add the program in this microcontroller. For that follow the steps given below-

- Double click on Atmega 16/32. There should be a window appear as shown in figure 10.
- There in the option-program files, click on folder icon button and go to the folder of your project. In the folder, there should be another folder named Debug. In Debug, double-click on your “.hex” file. Drop down the CKSEL Fuses option and click on the frequency 1 MHz and click on OK.

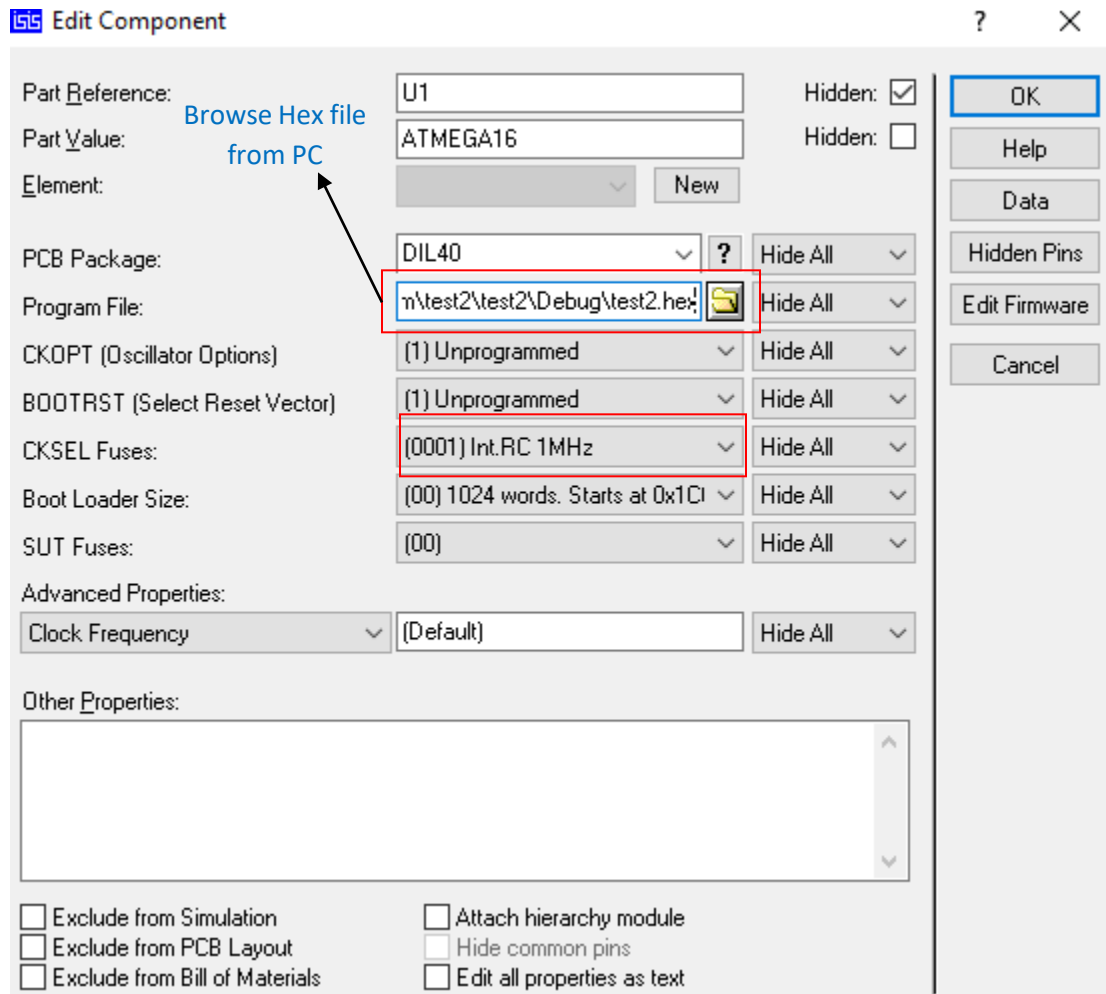


Figure 10: Selection of Program file and clock frequency



Introduction to Embedded Systems (5th Semester) 20ES
Lab Experiment#02

- Now you have loaded this program into the microcontroller Atmega16/Atmega32. You just have to simulate the program. Click on the play button in the bottom shown in figure 11, left corner of the page and if everything works fine, your LED should glow.

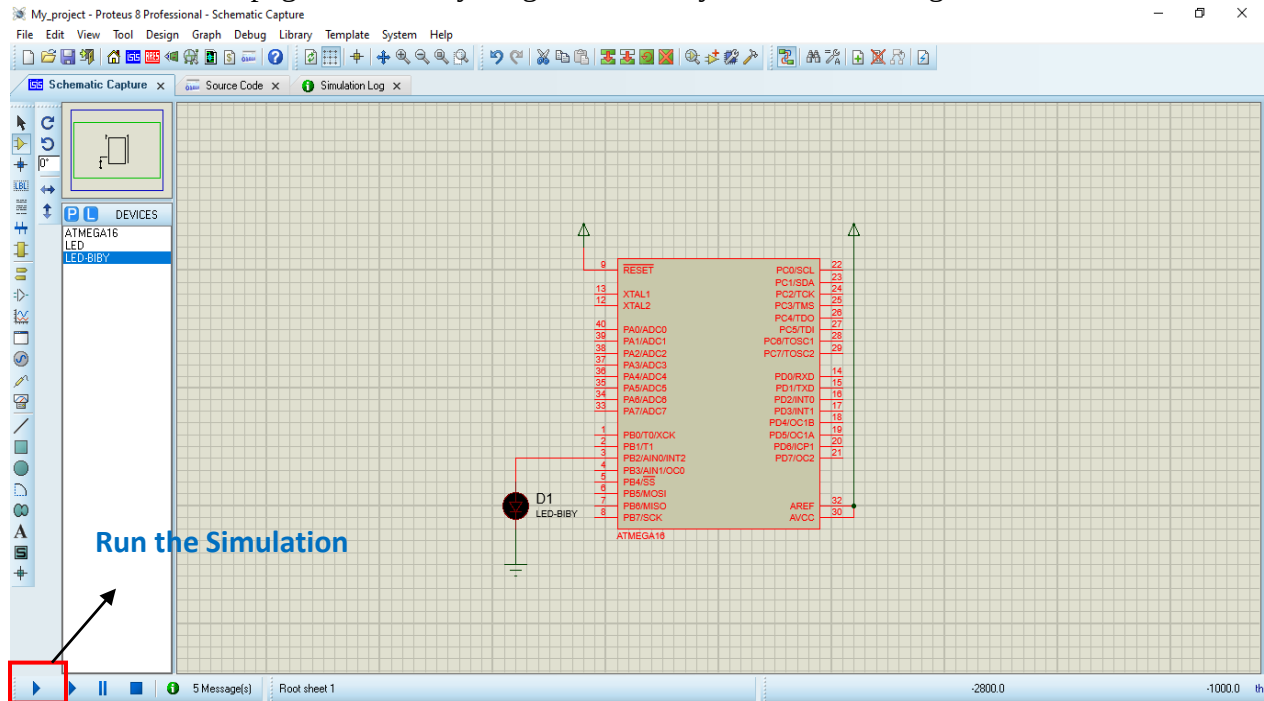


Figure 11: Run the Simulation

- Observe the output, as shown in the figure below 12.

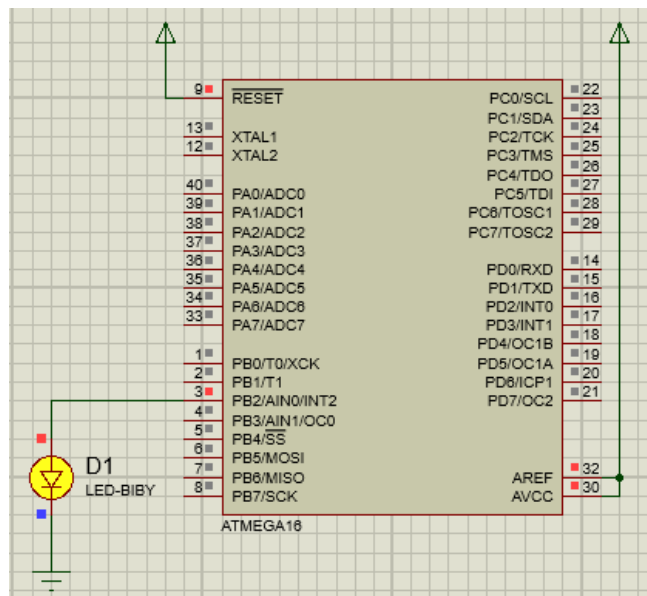


Figure 12: Output Simulation Result

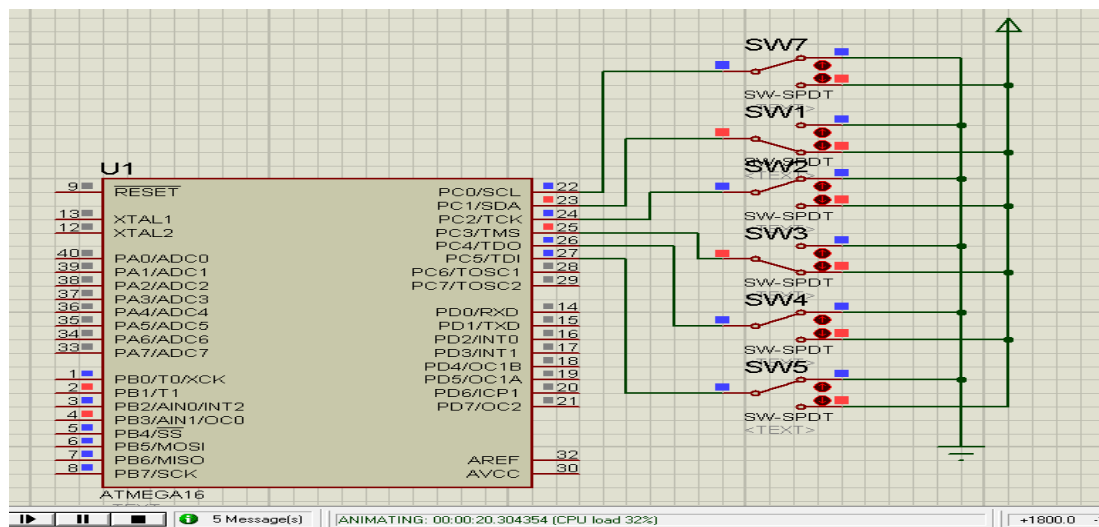


Introduction to Embedded Systems (5th Semester) 20ES
Lab Experiment#02

Example Program # 02: Get Data from Port C (Switches) and Put it on Port B (LEDs)

```
.INCLUDE "M16DEF.INC"
.ORG 0000
LDI R16,0x00
OUT DDRC, R16           ; making PORT C as input
LDI R17,0xFF
OUT DDRB,R17           ; making PORT B as output
AGAIN: IN R0,PINC       ; Read continuously from PORT C
OUT PORTB,R0           ; & send it to PORT B
JMP AGAIN
```

Proteus Output Simulation result



Lab Tasks:

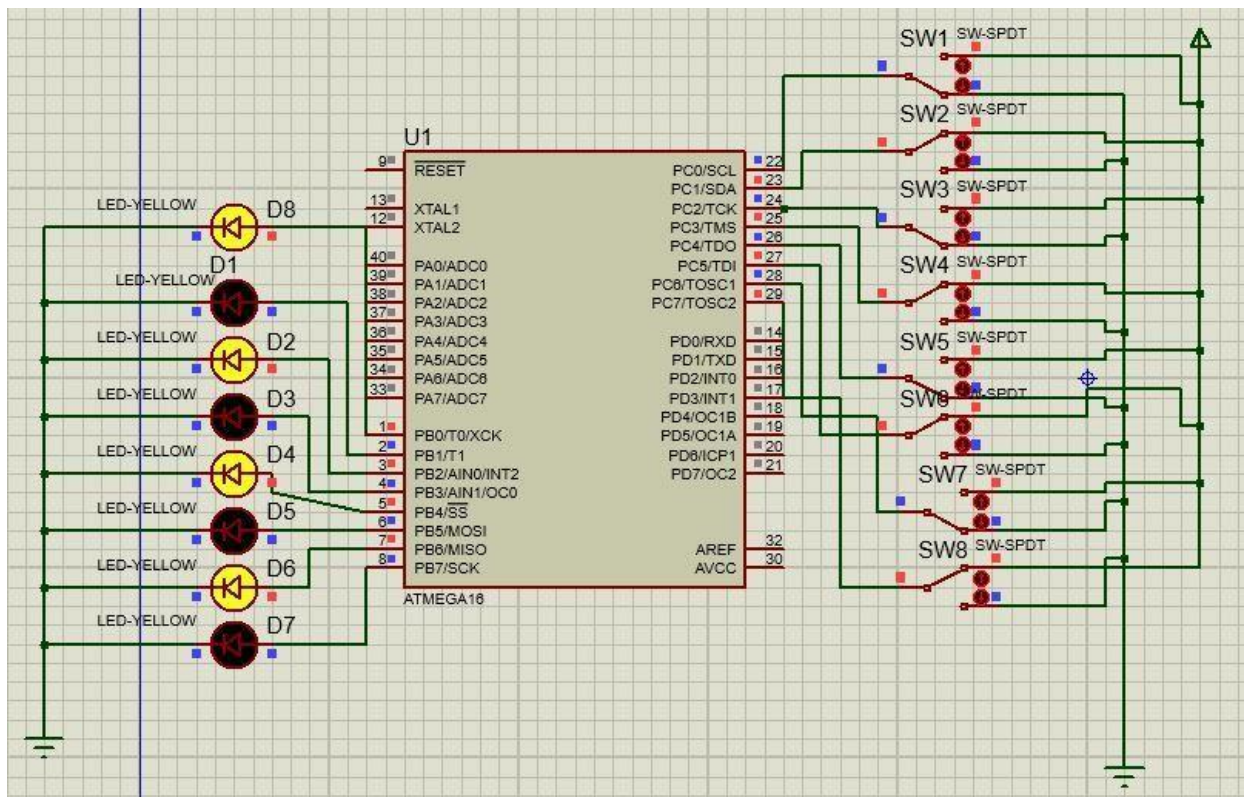
Simulate on Proteus and execute the following programs on AVR chip. Attach the Assembly program files and simulation results snapshots along with the handouts.

1. Write an Assembly language program that get data from switches connected at Port C and display the complimented output data to Port B.
2. Write an Assembly program that get data from lower nibble of Port C and put it to upper nibble of port B.
3. Write an Assembly program that that count numbers from 0 to 255 on Port B.
4. Write an Assembly program that toggle an LED connected PA2 with a small delay.

1. Write an Assembly language program that get data from switches connected at Port C and display the complimented output data to Port B.

```
main.asm  X
.include "M16DEF.INC"
.org 0000

        LDI R16,0xFF
        LDI R17,0x00
        OUT DDRC,R17
        OUT DDRB,R16
L:       IN  R2,PINC
        COM R2
        OUT PORTB,R2
        JMP L
```

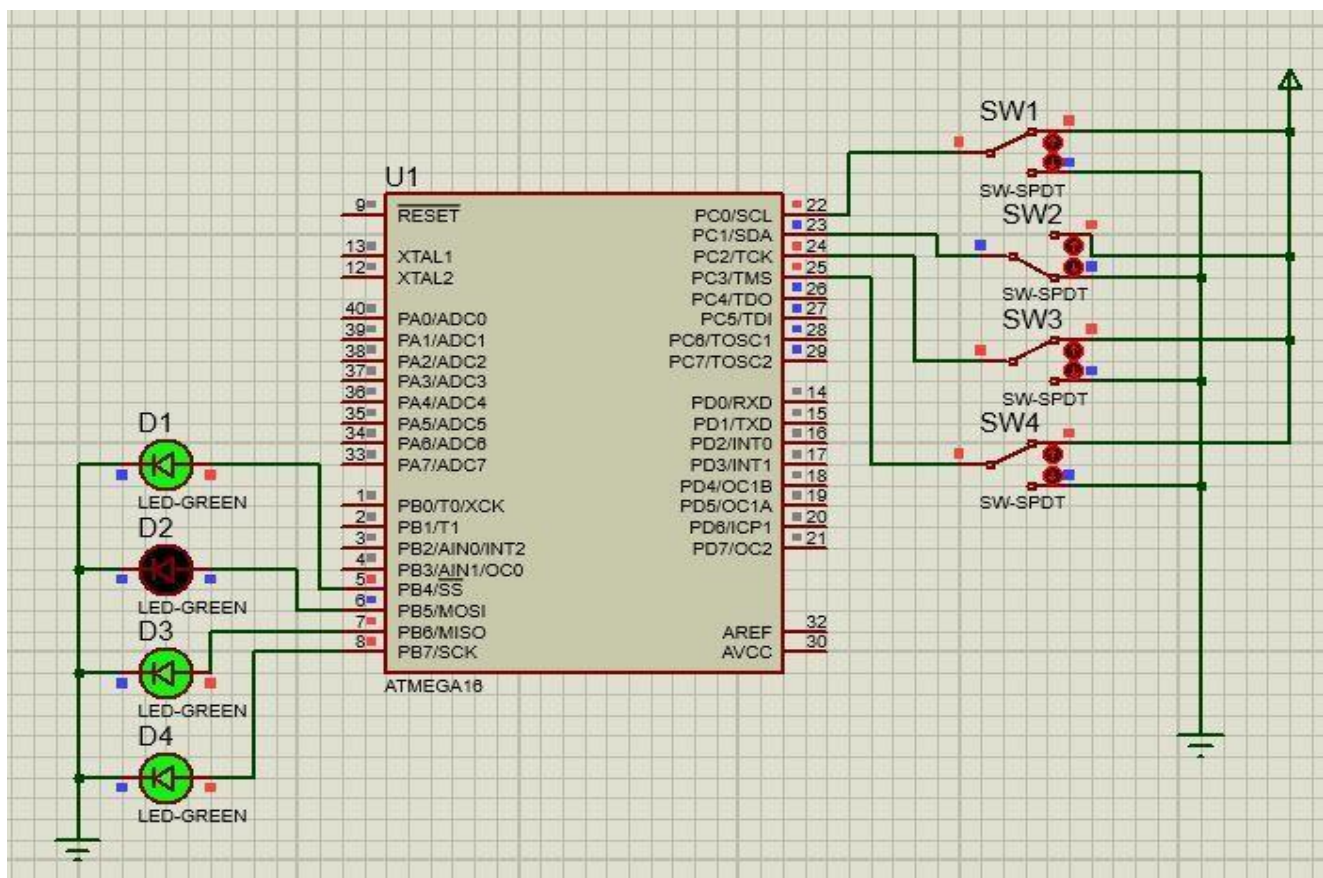


2. Write an Assembly program that get data from lower nibble of Port C and put it to upper nibble of port B.

```
2NDPROGRAM1  main.asm  X
#include "M16DEF.INC"
.org 0000

        LDI R16,0XF0
        OUT DDRC,R16
        OUT DDRB,R16

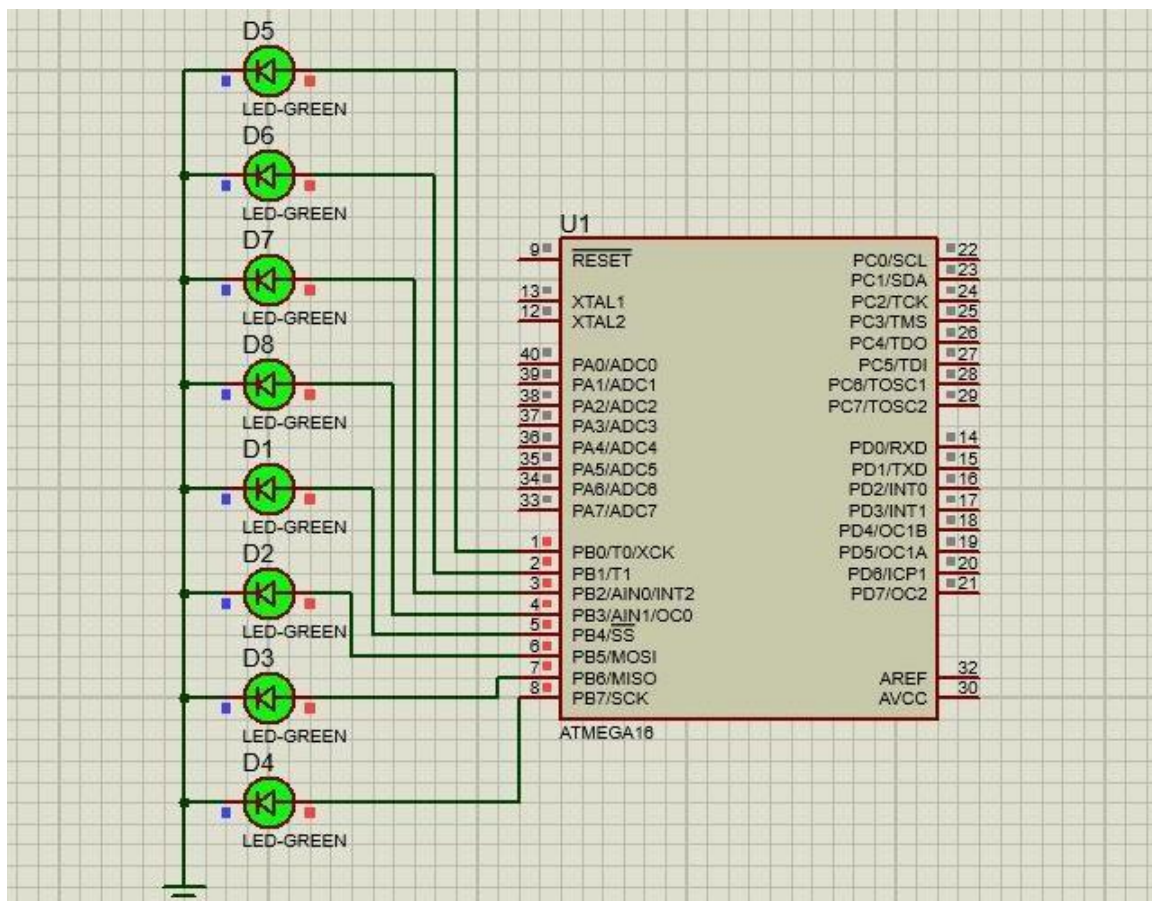
INPUT:   IN R1,PINC
        SWAP R1
        OUT PORTB,R1
        JMP INPUT
```



3. Write an Assembly program that that count numbers from 0 to 255 on Port B.

```
PROGRAM3  main.asm  X
.include "M16DEF.INC"
.org 0000

        LDI R16,0xFF
        OUT DDRB,R16
        LDI R17,0x00
        LDI R18,0xFF
OUTPUT:  OUT PORTB,R17
        INC R17
        DEC R18
        BRNE OUTPUT
        OUT PORTB,R16
STOP:    JMP STOP
```

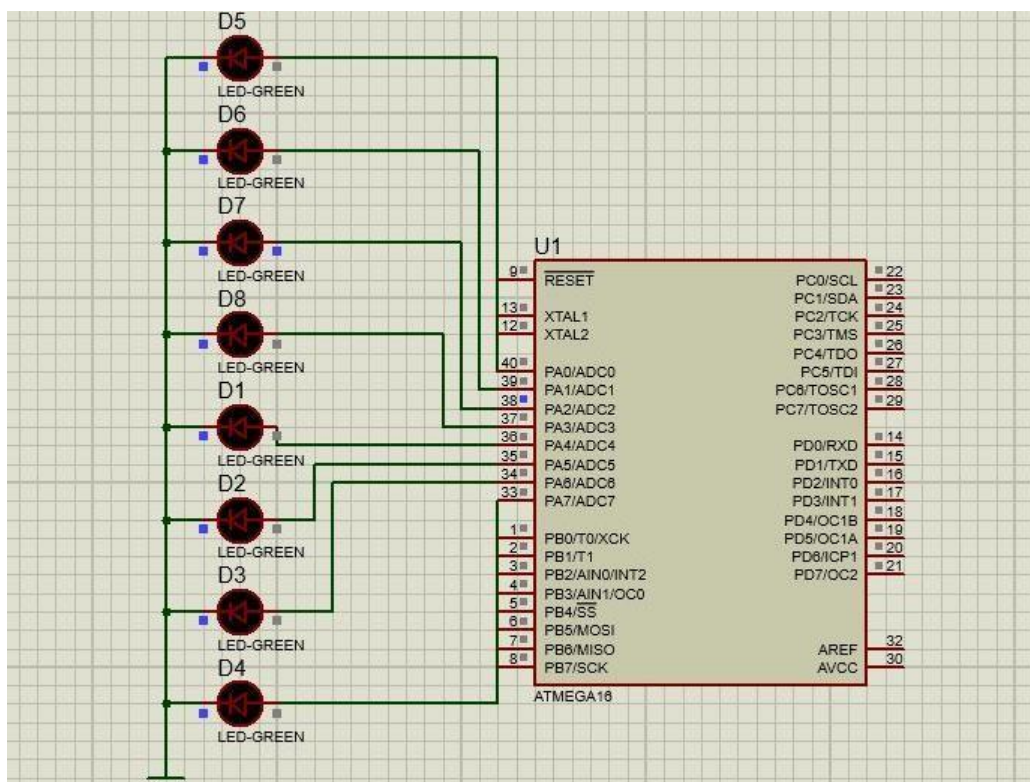
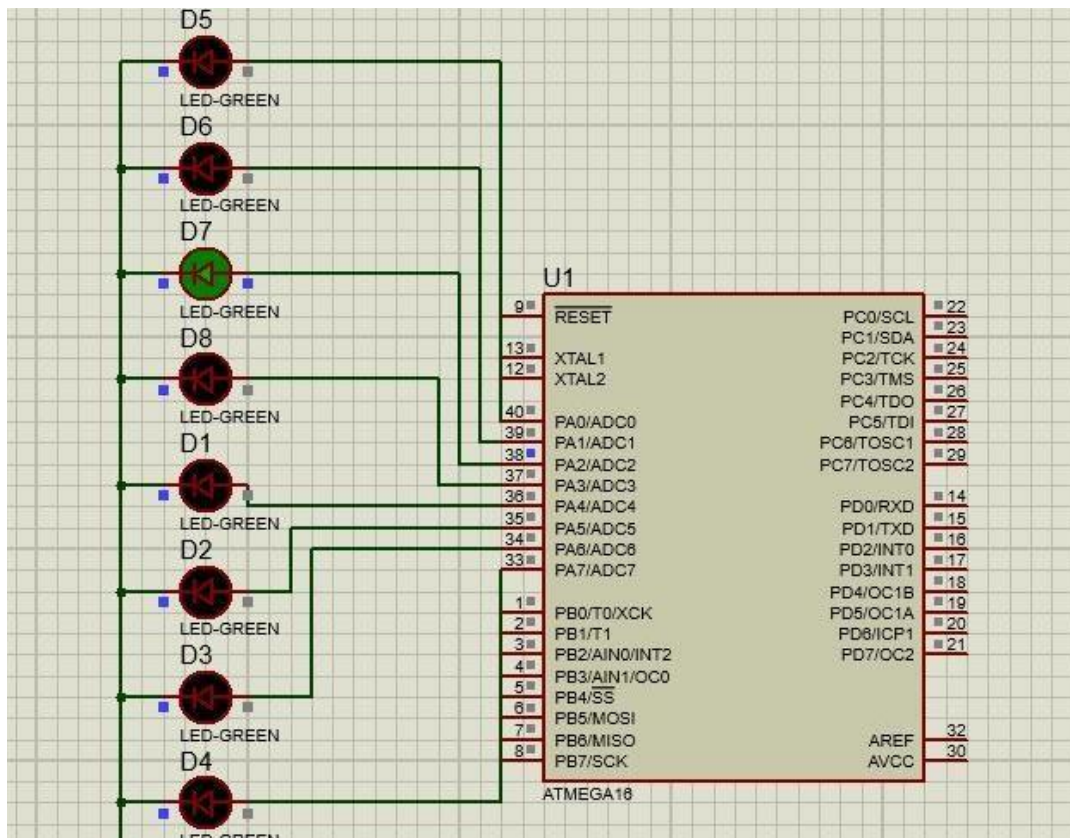


4. Write an Assembly program that toggle an LED connected PA2 with a small delay.

```
main.asm  + X
.include "M16DEF.INC"
.org 0000

        LDI R16,0X04
        LDI R18,0XFF
        LDI R19,0XFF
        OUT DDRA,R16
        LDI R17,(1<<2)
OUTPUT:  OUT PORTA,R17
NOOP:    NOP
        NOP
        NOP
        NOP
        NOP
        DEC R18
        BRNE NOOP ; NESTED LOOP USED TO GIVE DELAY TO LED.
        DEC R19
        BRNE NOOP
        COM R17
        JMP OUTPUT
```


ONLY PA2 IS AFFECTED





**DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO**

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#03 & 04

Roll No. 20ES062 _____ Date of Conduct: _____

LAB PERFORMANCE INDICATOR	SUBJECT KNOWLEDGE	DATA ANALYSIS AND INTERPRETATION	ABILITY TO CONDUCT EXPERIMENT	PRESENTATION	CALCULATION AND CODING	OBSERVATION/RESULTS	SCORE

AVR I/O PORT C Programming, Simulation of the Program in Proteus and Programming the code on Microcontroller chip using Extreme Burner

PERFORMANCE OBJECTIVE:

After successful completion of this lab, students will be able to:

- Introduce the AVR Microcontrollers Programmers.
- Become familiar with lab Tool – AVR Studio and Proteus
- Understand the basics of C Programming of AVR I/O Port Programming

REQUIRED TOOLS AND SOFTWARES:

- WinAVR (GCC Compiler)
- AVR Studio 4 or any latest version - (for Programming)
- Proteus 8 or any latest version - (for Simulation)
- Extreme Burner – AVR (for Programming through ICSP)
- AVR Trainer or AVR ATMEGA 16 Chip
- USBasp Programmer

AVR C-language Programming Basics:

The basic Structure of AVR Programming in C-language is:

```
#include "avr/io.h"
int main(void)
{
    // Your Program Here

    return 0;
}
```

Declaring a variable:

Variable declaration is like putting your data in registers of Microcontroller. Holding your data in variables / registers is necessary to process the data or holding it for later use in program.

Data can be of 1 byte or more, C-language offers multiple data types, each data type has different data holding capacities:



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#03 & 04

Table 1: Some Data types widely used by C Compilers

Data Type	Size in Bits	Data Range/Usage
unsigned char	8-bit	0 to 255
char	8-bit	-128 to +127
unsigned int	16-bit	0 to 65,535
int	16-bit	-32,768 to +32,767
unsigned long	32-bit	0 to 4,294,967,295
long	32-bit	-2,147,483,648 to +2,147,483,648
float	32-bit	$\pm 1.175e-38$ to $\pm 3.402e38$
double	32-bit	$\pm 1.175e-38$ to $\pm 3.402e38$

If you want to store data from a port (i.e. 1 Byte) and only positive numbers then use:

unsigned char variable;

if data is also negative numbers and value is between -128 to +127, then simply use:

char variable;

Configuring Port as Input / Output:

To configure / initialize a port to work as Input or output, there is a register named DDR (Data Direction Register), which is used to set the direction of Port as Input or Output. Writing one (i.e. LOGIC HIGH) on any bit, makes that particular bit as OUTPUT, for example:

DDRA = 0b00000001;

This makes bit 0 of PORT A as output, while remaining all bits as Input, because writing ZERO (i.e. LOGIC LOW) makes that bit as Input.

To declare all pins of PORT A as output:

DDRA = 0b11111111; or
DDRA = 0xFF;

And as input:

DDRA = 0b00000000; or
DDRA = 0x00;



**DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO**

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#03 & 04

Reading or Writing Data to Ports:

To Read data from any Port, we will use PIN function, if we want to read data from Port A, then syntax will be:

```
Unsigned char z = PINA;
```

This will read data from PORT A and store data in variable named “z”, whose data type is unsigned char and can store values from 0 to 255 (0x00 to 0xFF).

And to write data to Port, the keyword PORT will be used, like:

```
PORTA=0xA2;
```

It will write data “A2” in hex to Port A, however you can also assign variable instead of constant data 0xA2, like:

```
unsigned char z=0xA2;  
PORTA=z;
```

PORTA = z; will write data of z to PORT A, and the data in z variable is 0xA2, hence 0xA2 will be written to PORT A.

Looping in Programs:

Sometimes we need to do counting in program, or repeat some portion of code in program, this can be done through loops. AVR supports same C-language loops, like: FOR-LOOP, WHILE LOOP and DO-WHILE LOOP.

For Loop:

```
char i=0;  
  
for(i=0;i<10;i++)  
{  
    // code here  
}
```

This will repeat a portion of code, 10 times.

While Loop:

```
while (a < 10)  
{  
    // code here  
}
```

This will repeat the portion of code, until value of “a” is less than 10. For an infinite loop, use:



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#03 & 04

```
while(1)
{
    // code here
}
```

This will repeat code unlimited times.

Example Program # 01: A program that will write a constant Data 0xF0 to Port B (upper nibble of Port B as ON and lower nibble as OFF).

Program:

```
#include "avr/io.h"
int main(void)
{
    DDRB=0xFF;           // Port B as output
    PORTB=0xF0;          // Writing F0 to Port B

    while(1);            // Infinite Loop Here

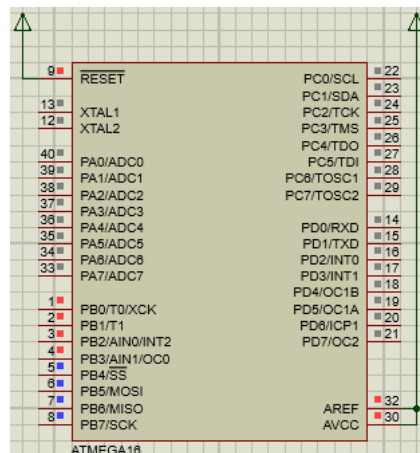
    return 0;
}
```

Explanation:

First, we need to initialize ports that we will use in this program, Ports as input or Output, then do the task as said, write the data to PortB. For writing we will use `PORTB = data;`

Simulate the Program in Proteus

- Write the above program#01 in AVR Studio and build the Program. Note the Hex file creation and its path.
- Open Proteus, draw the schematic in Proteus and then link the Hex file created by AVR Studio with the Microcontroller in the Proteus as done in previous lab.
- Observe the output, as shown in the figure below.





DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#03 & 04

Example Program # 02:

Get Data from Port C (Switches) and put it on Port B (LEDs)

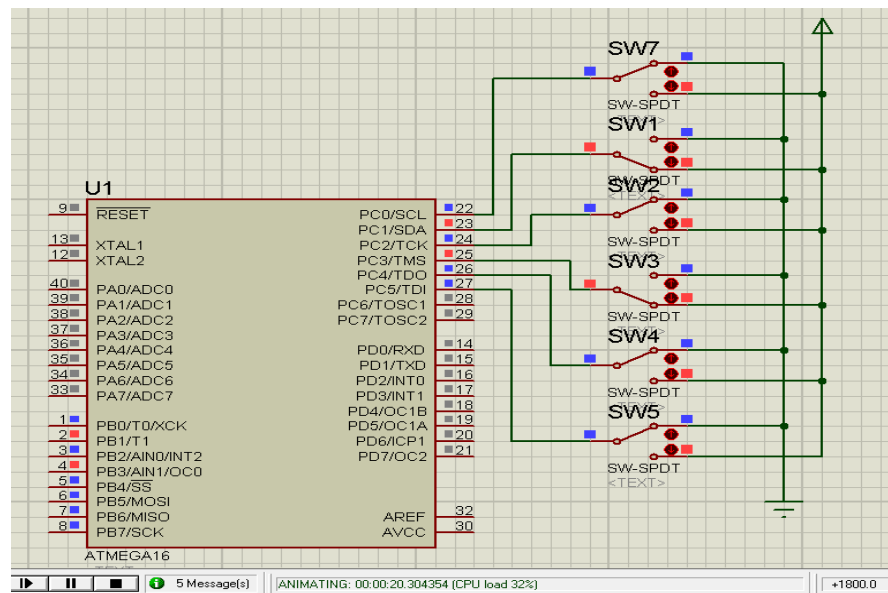
Program:

```
#include "avr/io.h"
int main(void)
{
    DDRB=0xFF;           // Output
    DDRC=0x00;           // Input
    while(1)
    {
        char z=PINC;
        PORTB=z;
    }
    return 0;
}
```

Explanation:

First initialize the ports, PORTB as output and PORTC as input, then copy the data from switches, to copy / read we will use PINC (Reading from PortC). The data which is read from switches must be stored in some variable; we have used “z” here. Now write the data which is read from switches, to PortC, to write, we will use PORTC (Writing to PortC).

Proteus Output Simulation result of program#02





Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#03 & 04

Example Program # 03: A C-program that toggle an LED connected to PB5 of Port B with a small delay. Use a predefined delay function in Win AVR.

Program:

```
#include "avr/io.h"
#include "util/delay.h"
int main(void)
{
    DDRB=0xFF;           // Output mode
    while(1)
    {
        PORTB ^= (1<<5); //toggle PB5
        _delay_ms(1000);

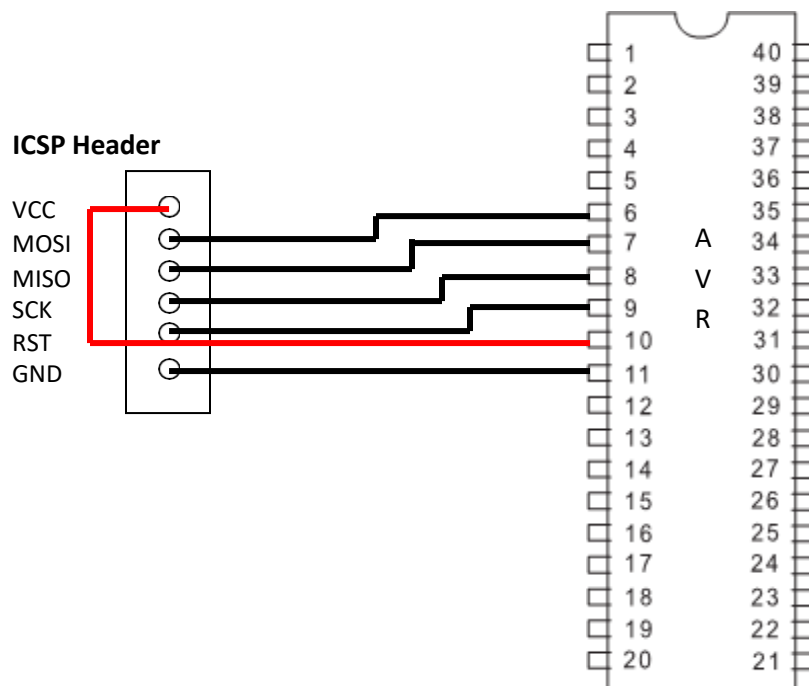
    }
    return 0;
}
```

Programming the AVR Microcontrollers

The Programming Software: The programming Software is required to transfer hex file to the target MCU. There are various options available depending on the programmer you're using. In this tutorial series we will be using Extreme Burner AVR. There is other software that you could also try is Khazama.

USBasp Programmer Connection Diagram:

Whenever you design any circuit, just place a header connector as shown in Fig below, by connecting USBasp Programmer with this, you can easily program the chip on the board without taking IC out of socket.





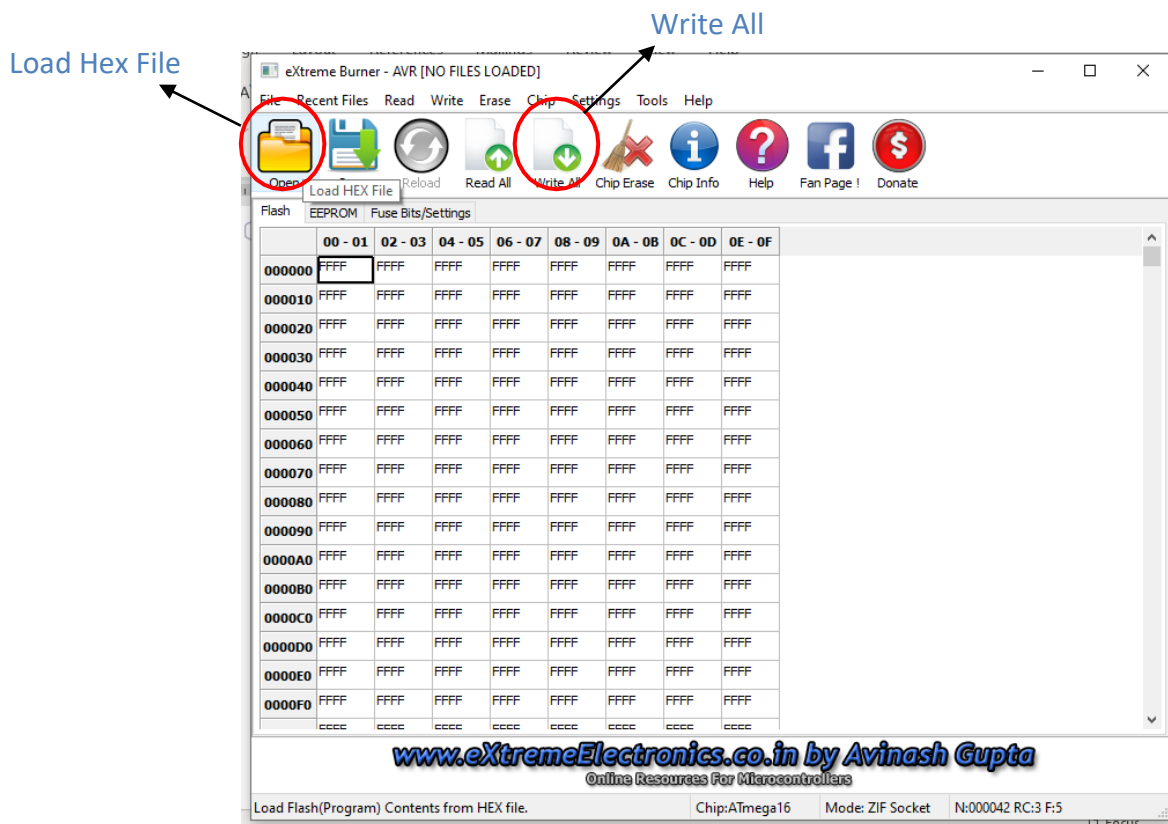
Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#03 & 04

Programming the code on Microcontroller chip

In order to burn the code on ATmega16 AVR chip and observe the effects directly on the hardware we need to connect the AVR chip with PC/Laptop using USBASP programmer as described in above figure. To burn the code in AVR chip follow the following steps.

- Write the above program#03 in AVR Studio and build the Program. Note the Hex file creation and its path.
- Open the Extreme Burner AVR-Programmer Application. Click on OPEN icon as shown in below figure.



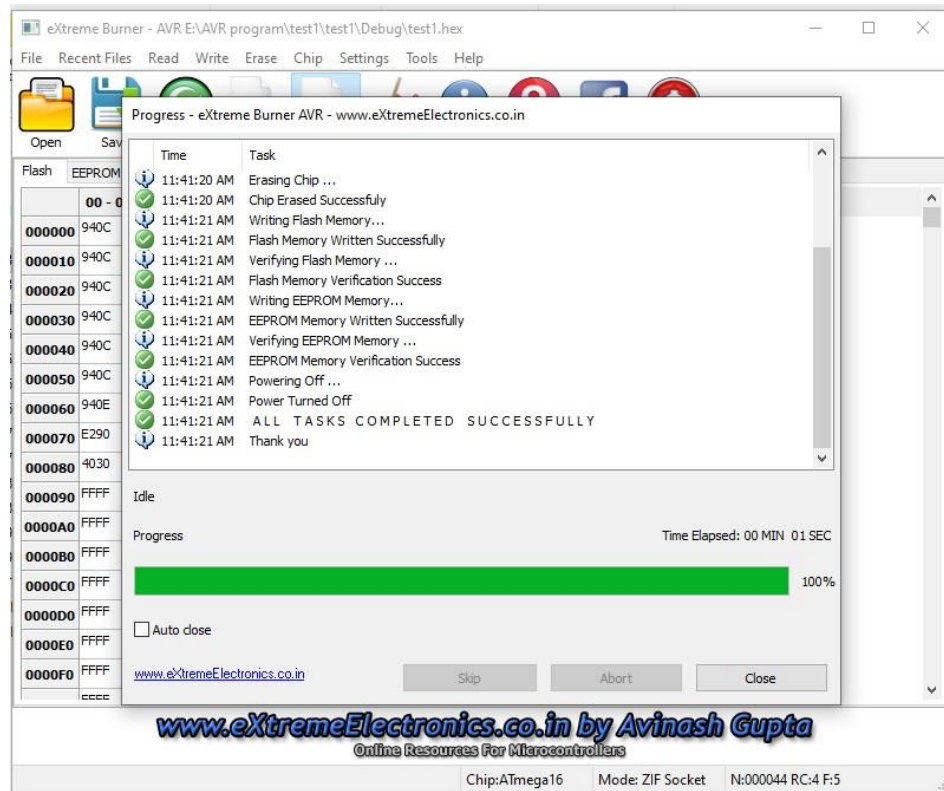
- Browse for your required Hex file and then click on Write All icon as shown in above figure.
- Following window will appear when you click on Write All icon, it will show you the Burn program progress.



DEPARTMENT OF ELECTRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

Introduction to Embedded Systems (5th Semester) 20ES

Lab Experiment#03 & 04



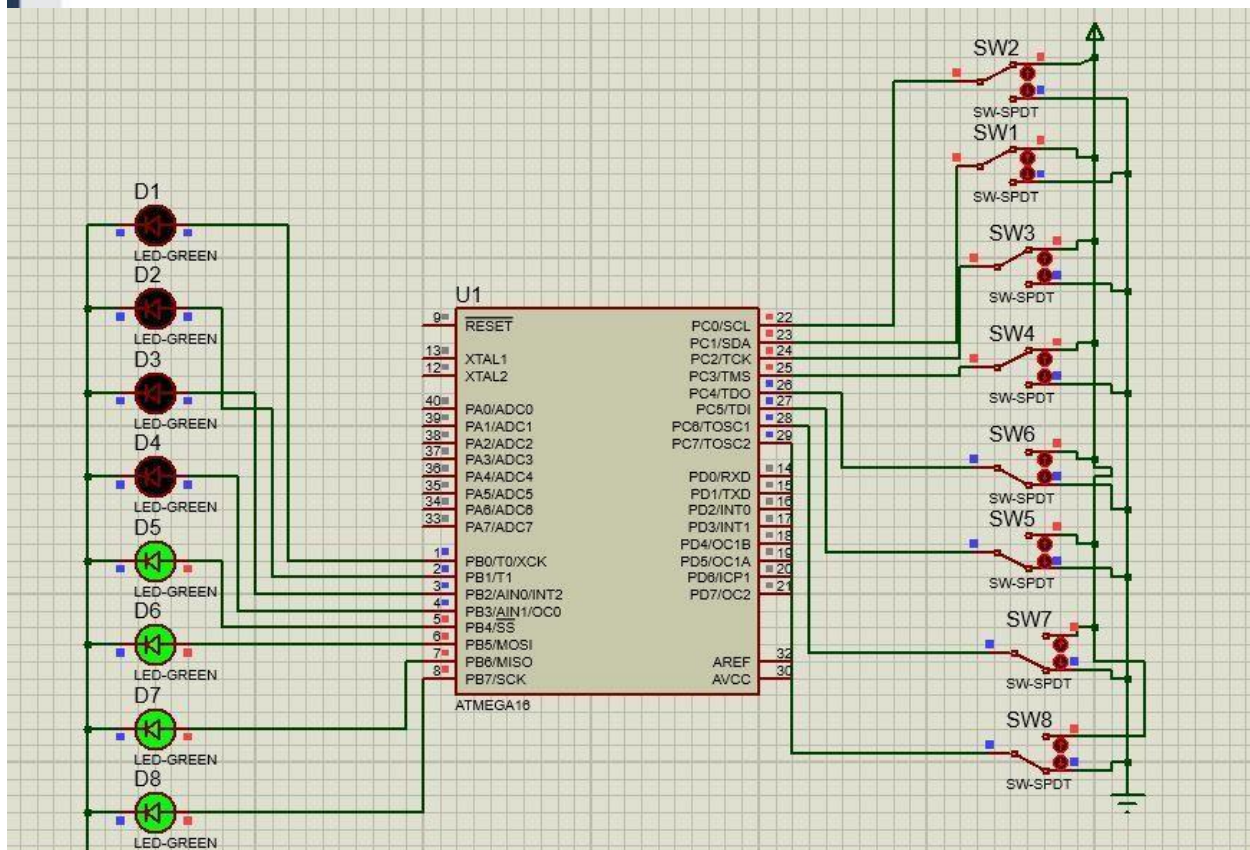
- After successfully uploaded your program in AVR microcontroller, Now observe the results on hardware.

Lab Tasks:

Simulate on Proteus and execute the following programs on AVR Chip. Attach the C-program files and simulation results snapshots along with the handouts.

1. Write a C program that get data from switches connected at Port C and display the complimented output data to Port B.
2. Write a C program that get data from lower nibble of Port C and put it to upper nibble of port B.
3. Write a C program that that count numbers from 0 to 255 on Port B.
4. Write a program to generate these random numbers on Port C: 100, 255, 123, 80, 66, 36, 15, 00.
5. Write a C program that toggle an LED connected PA2 with a small delay.

- [illegible]

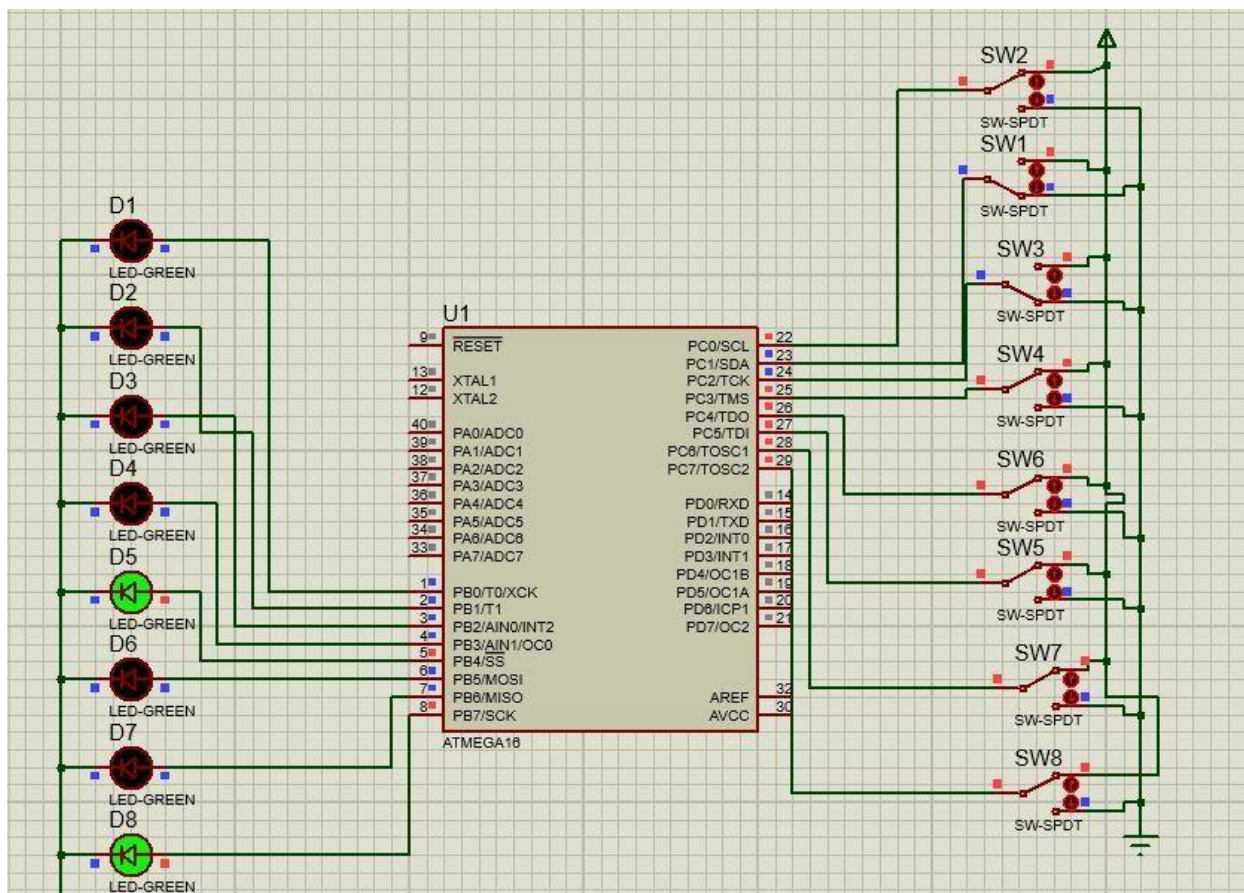


2. Write a C program that get data from lower nibble of Port C and put it to upper nibble of port B.

```
main.c X
main int main(void)
#include <avr/io.h>

int main(void)
{
    DDRC=0XF0;
    DDRB=0XF0;
    while (1)
    {
        PORTB=(PINC<<4);
    }
}
```

only lower nibble data of PORTC is displayed on upper nibble of PORTB other pins are not affecting

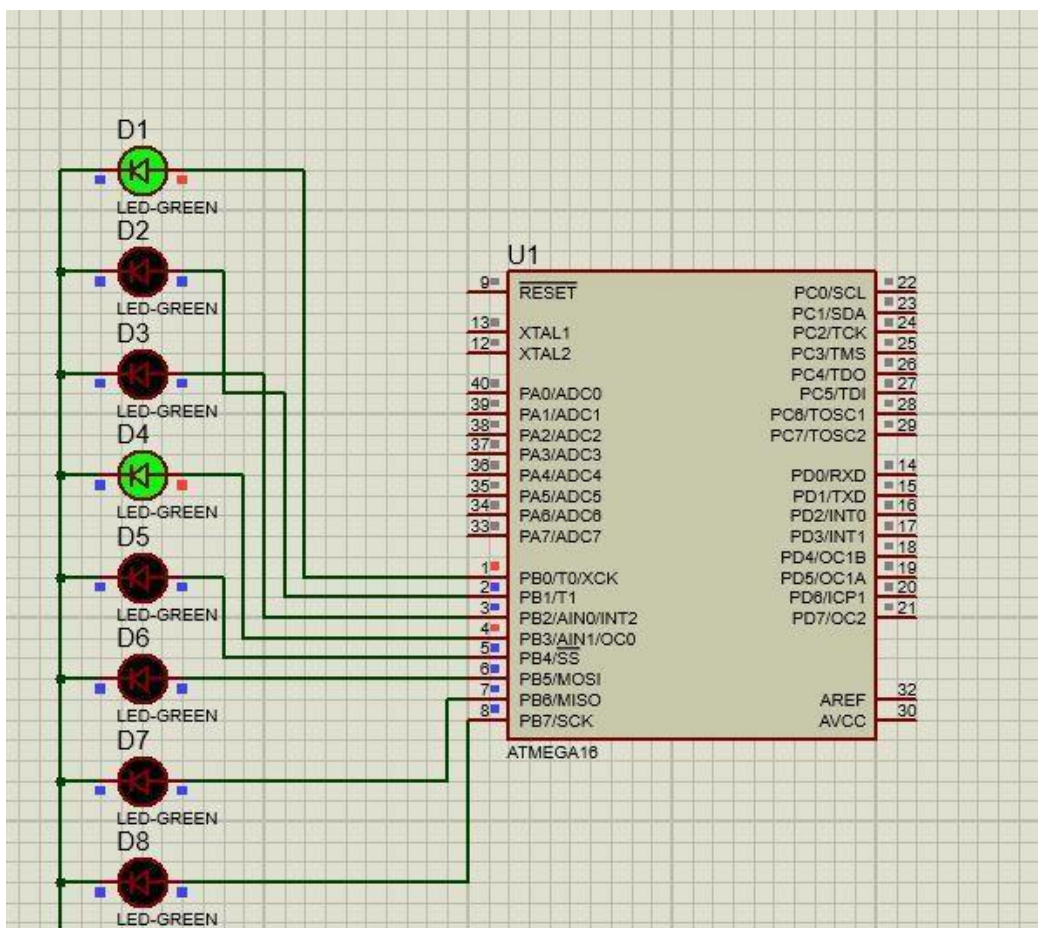


3. Write a C program that that count numbers from 0 to 255 on Port B.

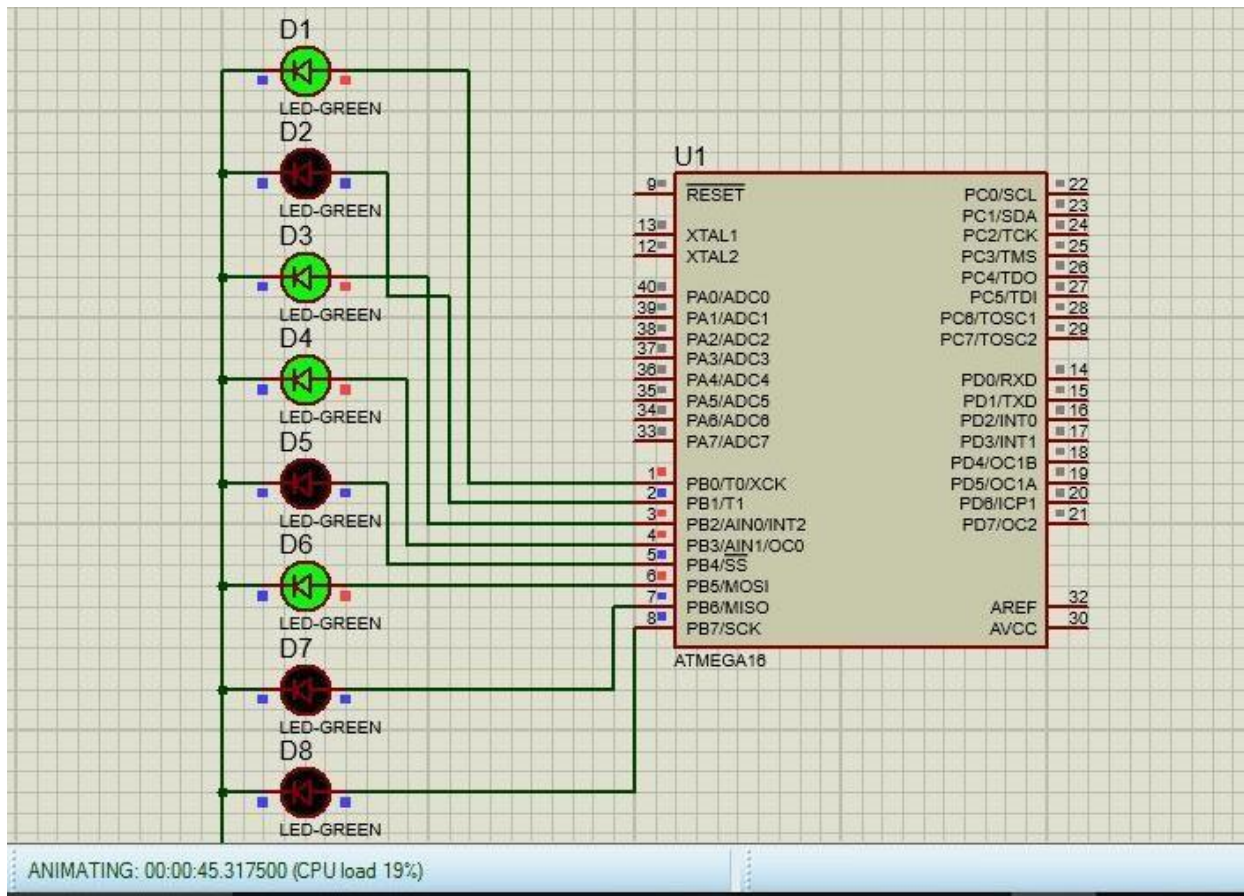
```
main.c* X
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRB=0xFF;
    unsigned int z;

    while (1) //first count 0-255 than again start from 0
    {
        for(z=0; z<256; z++)
        {
            PORTB=z;
            _delay_ms(1000); //give delay of 1 second
        }
    }
}
```



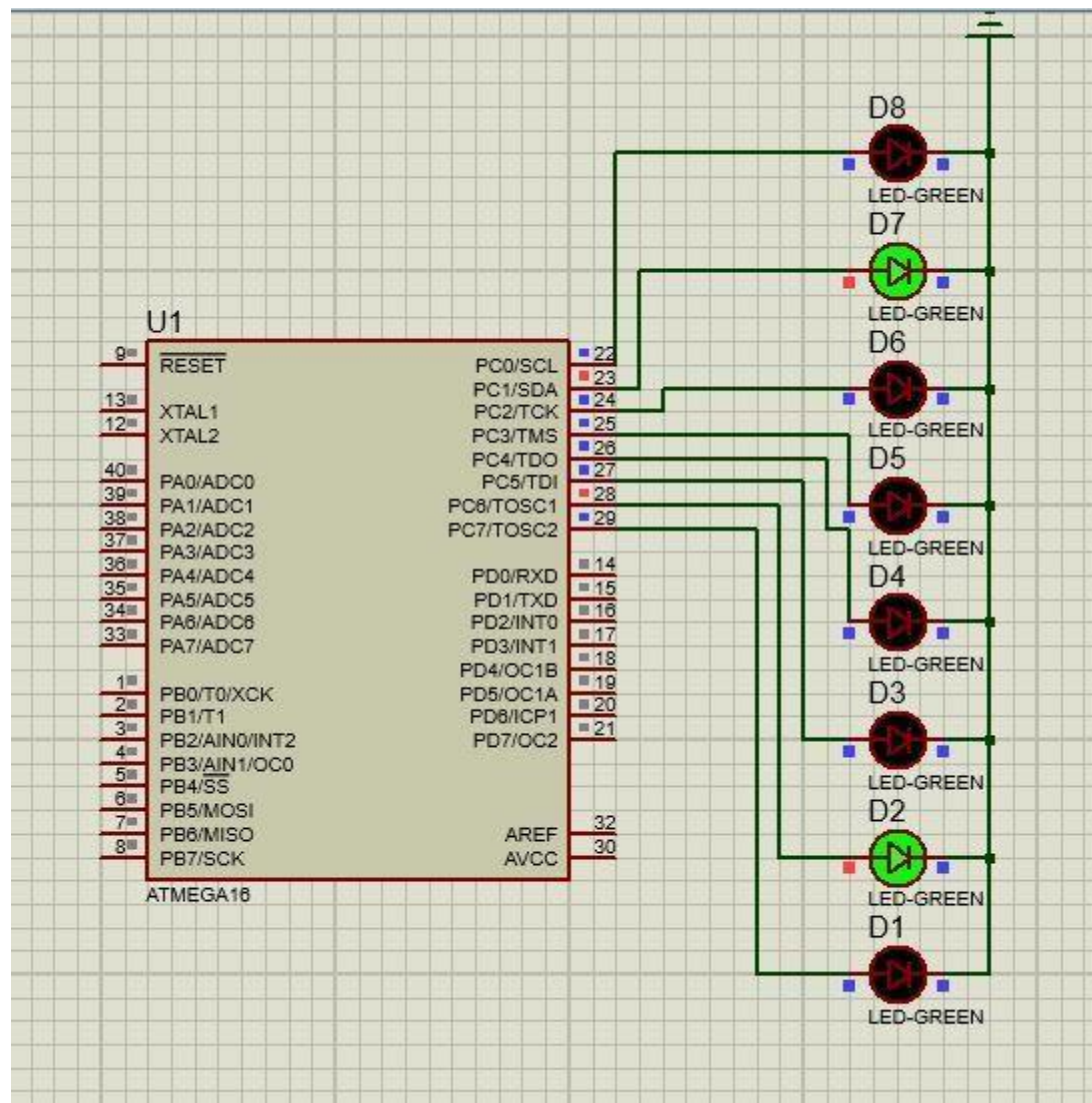
at 45 second no is 45.



4. Write a program to generate these random numbers on Port C: 100, 255, 123, 80, 66, 36, 15, 00.

```
main.c* X
main.c C:\Users\Ahmed Bux\Desktop\EMBEDDED SYSTEM ALL LAB
#include <avr/io.h>
#include <util/delay.h>
int main(void)
{
    DDRC=0xFF;
    while (1)
    {
        PORTC=0x64; //SEND 100
        _delay_ms(1000); //give delay of 1 second
        PORTC=0xFF; //SEND 255
        _delay_ms(1000);
        PORTC=0x7B; //SEND 123
        _delay_ms(1000);
        PORTC=0x50; //SEND 80
        _delay_ms(1000);
        PORTC=0x42; //SEND 66
        _delay_ms(1000);
        PORTC=0x24; //SEND 36
        _delay_ms(1000);
        PORTC=0x0F; //SEND 15
        _delay_ms(1000);
        PORTC=0x00; //SEND 00
        _delay_ms(1000);
    }
}
```

119 %

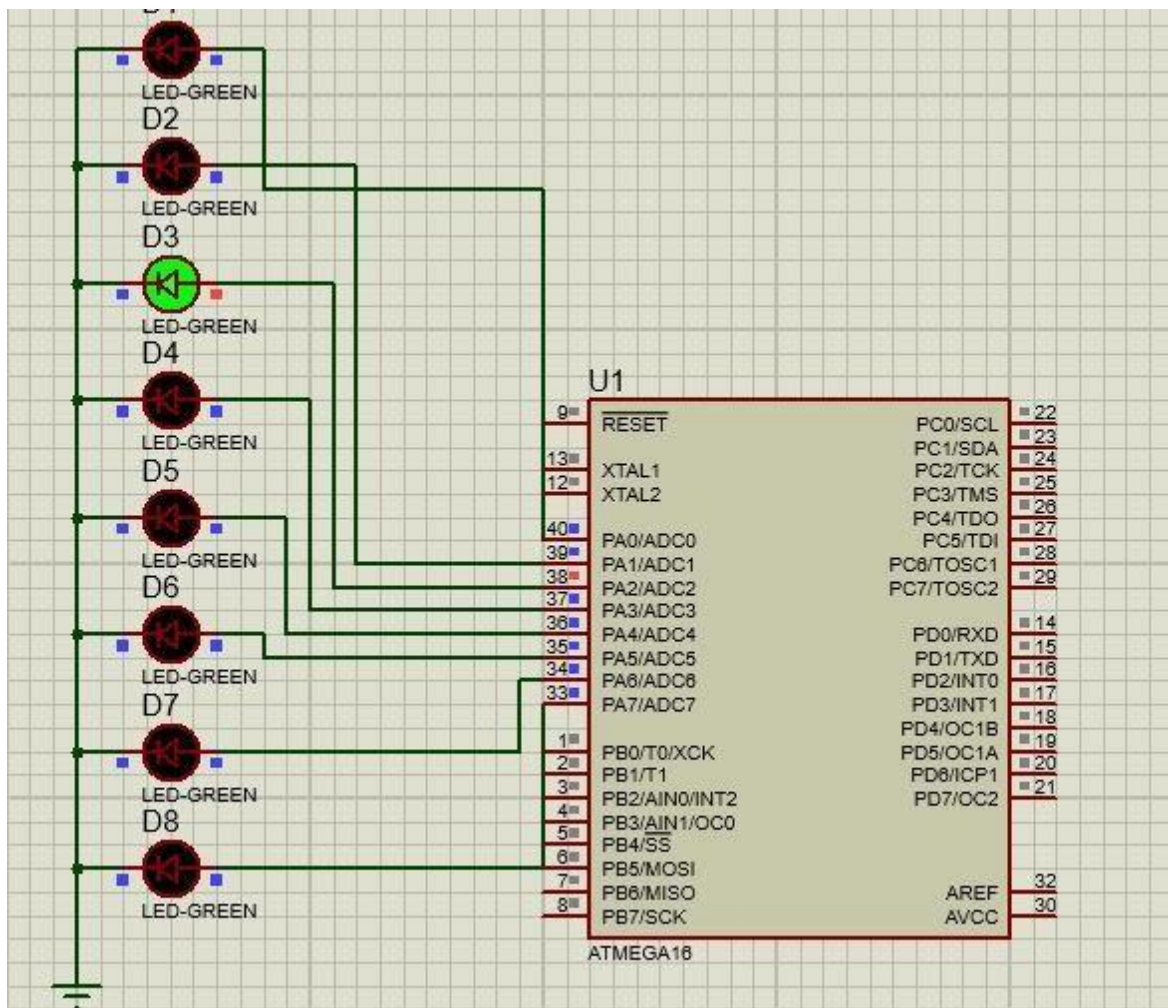


5. Write a C program that toggle an LED connected PA2 with a small delay

```
main.c  X
main_while  while (1)

#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRA=0xFF;
    while (1)
    {
        PORTA=(1<<2);
        _delay_ms(1000);    // delay of 1 sec
        PORTA &=~(1<<2);
        _delay_ms(1000);
    }
}
```





MEHRAN UNIVERSITY OF ENGINEERING AND TECHNOLOGY, JAMSHORO
DEPARTMENT OF ELECTRONIC ENGINEERING



INTRODUCTION TO EMBEDDED SYSTEMS (ES-314)

Batch: 20ES (5th Semester)

Lab Experiment #5

Interfacing Character LCD with AVR ATMEGA 16

Name	<u>KARAN</u>	Roll #	<u>20ES062</u>
Signature of Lab Tutor		Date	

RUBRICS:

Performance Metric	CONDUCTING EXPERIMENT	PARTICIPATION	TEAMWORK	USE OF MODERN ENGINEERING TOOLS	DATA ANALYSIS	CALCULATION AND CODING	OBSERVATION/ PROGRAM RESULTS	Total Score
	1	0.5	0.5	1		1	1	05
Obtained								

OBJECTIVE(S)

#	Topic	# Of Lectures	CLO	Taxonomy level
1	The purpose of this lab is to Become familiar with Character LCD and its working.	3	4,5	P3, A2
2	Display the characters on LCD through C programming of ATMEGA16			

OUTCOME(S)

a. An ability to use the techniques, skills, and modern engineering tools necessary for engineering practice.	PLO5: Modern Tool Usage
b. An ability to function on multi-disciplinary teams	PLO9: Individual and Teamwork

REQUIRED TOOLS AND SOFTWARES:

- ✦ WinAVR (GCC Compiler)
- ✦ AVR Studio 4 or any latest version - (for Programming)
- ✦ Proteus 8 or any latest version - (for Simulation)
- ✦ Extreme Burner – AVR (for Programming through ICSP)
- ✦ AVR ATMEGA16/ ATMEGA32 Chip
- ✦ USBAsp Programmer
- ✦ 16x2 Character LCD

INTRODUCTION:

Various display devices are usually used to show information to users, like LED is used as indicator to show the indication like the Power is On, Device is working, Internet is working fine on DSL Modems etc. LCDs are used to show messages and information to users, like in cell phones, it displays messages and help us to connect to a particular contact from various contacts stored etc. In this lab, we will learn to interface with any microcontroller, we will also work with various options of LCD. LCDs are of two types:

1. Graphics LCD
2. Character / Matrix LCD

Here, we will work with 16x2 Character LCD.

Character LCDs:

With various LCD Manufacturers, the pin positions also vary. In most of character LCDs the pin 1 starts from Left side of LCD and ends at either pin number 14 or 16. Pin description is same for all character LCDs, the pin number 15 and 16 are for Backlight, positive supply and GND. Pin Description of Character LCD is given in table 1.



Table 1: Pin Description for LCD

Pin	Symbol	I/O	Description
1	V _{SS}	--	Ground
2	V _{CC}	--	+5 V power supply
3	V _{EE}	--	Power supply to control contrast
4	RS	I	RS = 0 to select command register, RS = 1 to select data register
5	R/W	I	R/W = 0 for write, R/W = 1 for read
6	E	I/O	Enable
7	DB0	I/O	The 8-bit data bus
8	DB1	I/O	The 8-bit data bus
9	DB2	I/O	The 8-bit data bus
10	DB3	I/O	The 8-bit data bus
11	DB4	I/O	The 8-bit data bus
12	DB5	I/O	The 8-bit data bus
13	DB6	I/O	The 8-bit data bus
14	DB7	I/O	The 8-bit data bus

Table 2: LCD Command Codes

Code (Hex)	Command to LCD Instruction Register
1	Clear display screen
2	Return home
4	Decrement cursor (shift cursor to left)
6	Increment cursor (shift cursor to right)
5	Shift display right
7	Shift display left
8	Display off, cursor off
A	Display off, cursor on
C	Display on, cursor off
E	Display on, cursor blinking
F	Display on, cursor blinking
10	Shift cursor position to left
14	Shift cursor position to right
18	Shift the entire display to the left
1C	Shift the entire display to the right
80	Force cursor to beginning of 1st line
C0	Force cursor to beginning of 2nd line
28	2 lines and 5 × 7 matrix (D4–D7, 4-bit)
38	2 lines and 5 × 7 matrix (D0–D7, 8-bit)

Working of LCD:

The LCD works in two modes, command mode and data mode. RS pin of LCD is responsible for selecting Mode. When RS =0 then command mode is selected and when RS=1 data mode is selected.

When we instruct LCD to do some operations like clear LCD, set cursor position, increment cursor etc, we will use command mode (RS=0).

When we want to display data on screen of LCD then we will use Data Mode (RS=1).

Procedure to work with Character LCD:

1. Initialize LCD (RS=0 – Command Mode)
2. Display Data (RS=1 Data Mode)

Initializing LCD:

There are various commands by which LCD is initialized; the list of commands is given in Table 2:

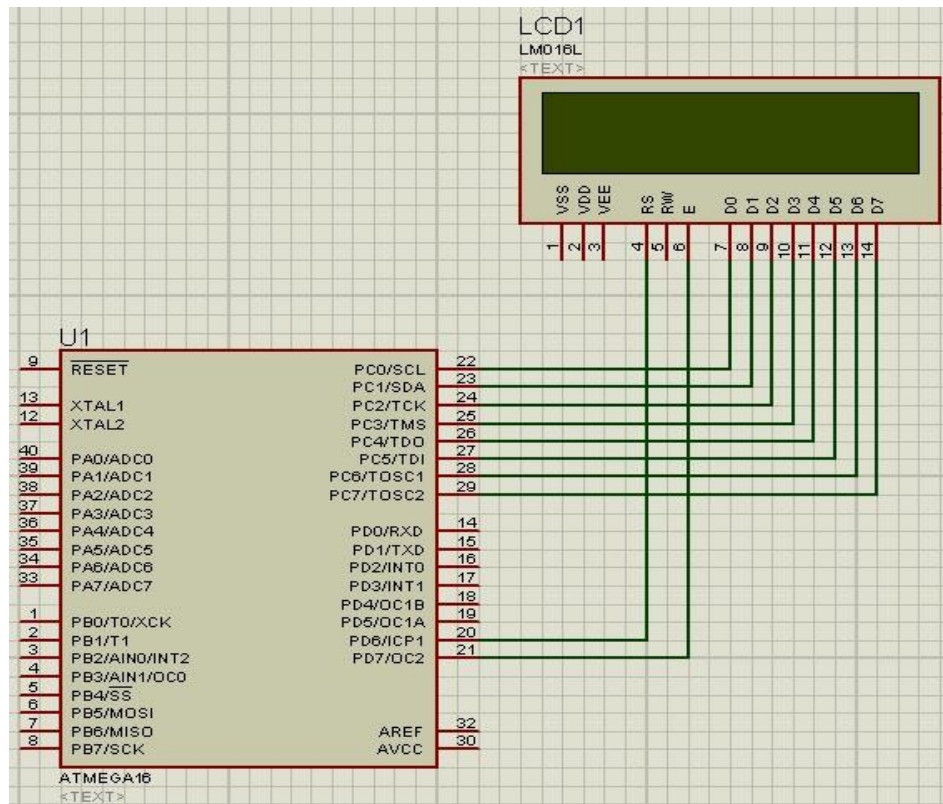
The list of necessary commands for initializing LCD is given below:

1. 8-Bit – 2 lines 5x7 matrix
2. Display ON
3. Increment Cursor
4. Clear Display Screen
5. Set Starting Cursor Position

Schematic Diagram:

We will connect LCD with Port C and D. Simplified connections are shown below:

Data lines of LCD are connected to Port C and Control Pins RS and EN of LCD are connected to Port D Pin 7 and 6 respectively.



Programming:

Delay subroutine:

There is a built-in delay subroutine, which can be added to program by including header file <delay.h>, and delay can be generated by calling.

```
_delay_ms(time in milliseconds);
```

Anywhere in program, where <time in milliseconds> will be replaced by a number.

For example, if we want to generate delay of 1 second (1000 milliseconds):

```
_delay_ms(1000);
```

Programming LCD:

As Port C is connected to Data lines of LCD and RS pin is connected to Port D-pin6, EN of LCD is connected to Port D-pin7.

To write any command to LCD:

First make Port C and Port D as output ports:

```
DDRC=0xFF;
```

```
DDRD=0xFF;
```

To send any command to LCD, make RS=0, by writing logic LOW to Port D-pin 6:

```
PORTD &=~(1 << 6);
```

Now, send command, for example we are sending 8-bit mode command, which is 0x38 from Table 2:

```
PORTC = 0x38;
```

After writing any command or sending any data, we must make EN signal of LCD HIGH for a little time and then make it off, such that data available on data lines can be stored in LCD's buffer.

Hence, the code will be:

```

PORTD &= ~(1<<6);           // RS=0
PORTC=0x38;                  // Command
PORTD |=(1<<7);              // EN=1
_delay_ms(1);                 // delay of 1 millisecond
PORTD &= ~(1 << 7);         // EN=0

```

To initialize LCD, we will send multiple commands and between each command we will create some delay say 10 milliseconds. To send multiple commands to LCD, we will repeat above code, by just changing command code, like for LCD on, Cursor Blinking, we have to write: 0x0E, the code will be:

```

PORTD &= ~(1<<6);           // RS=0
PORTC=0x0E;                // Command for Display ON, Cursor Blinking
PORTD |=(1<<7);              // EN=1
_delay_ms(1);                 // delay of 1 millisecond
PORTD &= ~(1 << 7);         // EN=0

```

Now this code will be repeated for every command. Why not to make a subroutine for this code.

Making a subroutine:

To make a function or subroutine, the basic structure is:

```

void name (datatype& variable) {
    // Code here
}

```

Whenever you call name, it will execute program above.

To make a subroutine for LCD commands:

```

void lcd_cmd(unsigned char data)
{
    PORTD &= ~(1<<6);           // RS=0
    PORTC=data;                // Command
    PORTD |=(1<<7);              // EN=1
    _delay_ms(1);                 // delay of 1 millisecond
    PORTD &= ~(1 << 7);         // EN=0
}

```

Now whenever we call lcd_cmd (any data); it will execute subroutine define above, but it will send data that is placed in brackets (here it is <any data>).

For example, we are calling:

```

lcd_cmd(0x38);

```

it will execute full subroutine of lcd_cmd but now data variable has value of 0x38. And 0x38 will write to Port C.

The main program to write multiple commands to LCD will look like:

```
#include "avr/io.h"
#include "util/delay.h"
void lcd_cmd(unsigned char cmnd)
{
    PORTD &= ~(1 << rs); data
    = cmnd;
    PORTD |= (1 << en);
    _delay_ms(1);
    PORTD &= ~(1 << en);
}
int main(void)
{
    DDRD=0xFF;
    DDRC=0xFF;
    lcd_cmd(0x38);           // 8-bit Mode
    _delay_ms(1);
    lcd_cmd(0x0E);          // Display on, Cursor Blinking
    ....
    return 0;
}
```

Similarly, if we want to make data subroutine (RS=1):

```
void lcd_data (unsigned char data2)
{
    PORTD |= (1 << 6);      // RS=1
    PORTC=data2;           // character data
    PORTD |= (1 << 7);      // EN=1
    _delay_ms(1);          // delay of 1 millisecond
    PORTD &= ~(1 << 7);     // EN=0
}
```

But the above subroutine will only print one character to LCD per call.

Now, we have to execute multiple commands on LCD and have to send more than one character to display on LCD. So the main program will be:

```
#include "avr/io.h"
```

```

#include "util/delay.h"

void lcd_cmd (unsigned char cmd)
{
    PORTD &= ~(1 <<6);      // RS=0
    PORTC= cmd;
    PORTD |= (1 <<7);       // EN=1
    _delay_ms(1);
    PORTD &= ~(1<< 7);      // EN=0
}

void lcd_data (unsigned char data) {

    PORTD |= (1<<6);        // RS=1
    PORTC = data;
    PORTD |= (1 << 7);      // EN=1
    delay(1);
    PORTD &= ~(1<< 7);      // EN=0
}

int main(void) {

    DDRD=0xFF;              // Port D as output
    DDRC=0xFF;              // Port C as output (LCD Data)
    lcd_cmd(0x38);          // 8-bit Mode
    _delay_ms(1);

    lcd_cmd (0x0E);         // Display On, Cursor Blinking
    _delay_ms(1);

    lcd_cmd (0x06);         // Increment Cursor
    _delay_ms(1);

    lcd_cmd (0x01);         // Clear Display
    _delay_ms(1);

    lcd_cmd(0x80);          // Cursor Position at 1st row, 1st col
    _delay_ms(1);

    lcd_data ('A');         // Data Display, char A
    _delay_ms(1);

    While      (1);
    return 0;
}

```


This program will write just one character 'A' to LCD screen, if we want to display multiple characters to LCD, we have to repeat `lcd_data()`; for each letter. Hence, to avoid that we can use arrays to write multiple bytes to LCD screen.

Using Arrays:

Using Arrays, we can store data which is to be displayed on LCD screen:

```
unsigned char <array-name>[<no. of characters>]="<text-here>;
```

for example: if we want to store word "Welcome" to an Array:

```
unsigned char data[7]="welcome";
```

in square-brackets we will write size of word that is stored in array. Here word "Welcome" occupies 7 bytes, hence we have written 7 in square brackets.

Now if want to write welcome on LCD, we will use loop and repeat `lcd_data()`; command, like:

```
unsigned char data[7]= "Welcome";
```

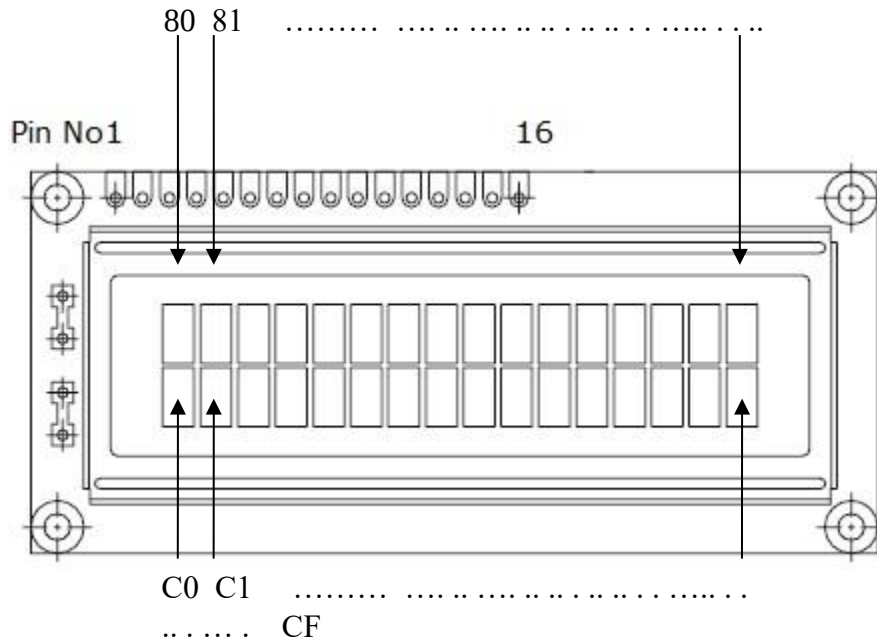
```
for(int i=0;i<7;i++)  
{  
    lcd_data(data[i]);  
    _delay_ms(1);  
}
```

This will repeat `lcd_data()`; command and each time it will take a byte from array "data" and display it on LCD.

Setting Position on LCD to display Data:

The LCD command list includes two commands for two rows of 16x2 LCD.

0x80 command is used to force cursor to be beginning of first line, and 0xC0 is used to force cursor to beginning of 2nd line.



To display data starting from row 1, column 2, 0x81 command will be sent to LCD.

1. Display your name and Roll No. at Center of LCD.

```
#include <avr/io.h>
#include <util/delay.h>

void command(unsigned char comnd)
{
    PORTB &=~(1<<6);
    PORTA=comnd;
    PORTB |=(1<<1);
    _delay_ms(1);
    PORTB &=~(1<<1);
}

void DATA(unsigned char data)
{
    PORTB |=(1<<6);
    PORTA=data;
    PORTB |=(1<<1);
    _delay_ms(1);
    PORTB &=~(1<<1);
}

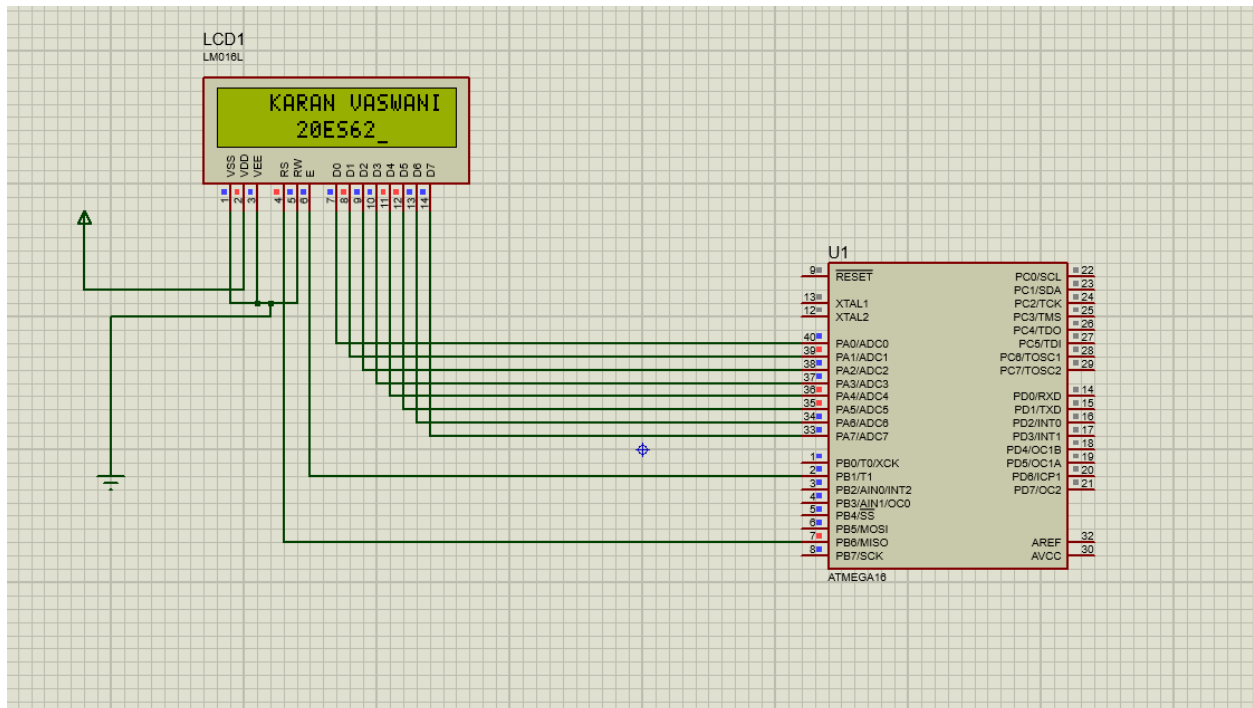
int main(void)
{
    DDRA=0XFF;
    DDRB=0XFF;

    while (1)
    {
        command(0x38);
        _delay_ms(1);
        command(0x0E);
        _delay_ms(1);
        command(0x06);
```

```

    _delay_ms(1);
    command(0x01);
    _delay_ms(1);
    command(0x83);
    _delay_ms(1);
    unsigned char array[]="KARAN VASWANI";
    for (int z=0; z<13; z++)
    {
        DATA(array[z]);
        _delay_ms(1);
    }
    command(0xC5);
    _delay_ms(1);
    unsigned char array1[]="20ES62";
    for (int z=0; z<6; z++)
    {
        DATA(array1[z]);
        _delay_ms(1);
    }
}
}

```



2. Make that text blink with delay of 1 seconds.

```

#include <avr/io.h>
#include <util/delay.h>

void command(unsigned char comnd)
{
    PORTB &=~(1<<6);
    PORTA=comnd;
    PORTB |= (1<<1);
    _delay_ms(1);
    PORTB &=~(1<<1);
}

```

```

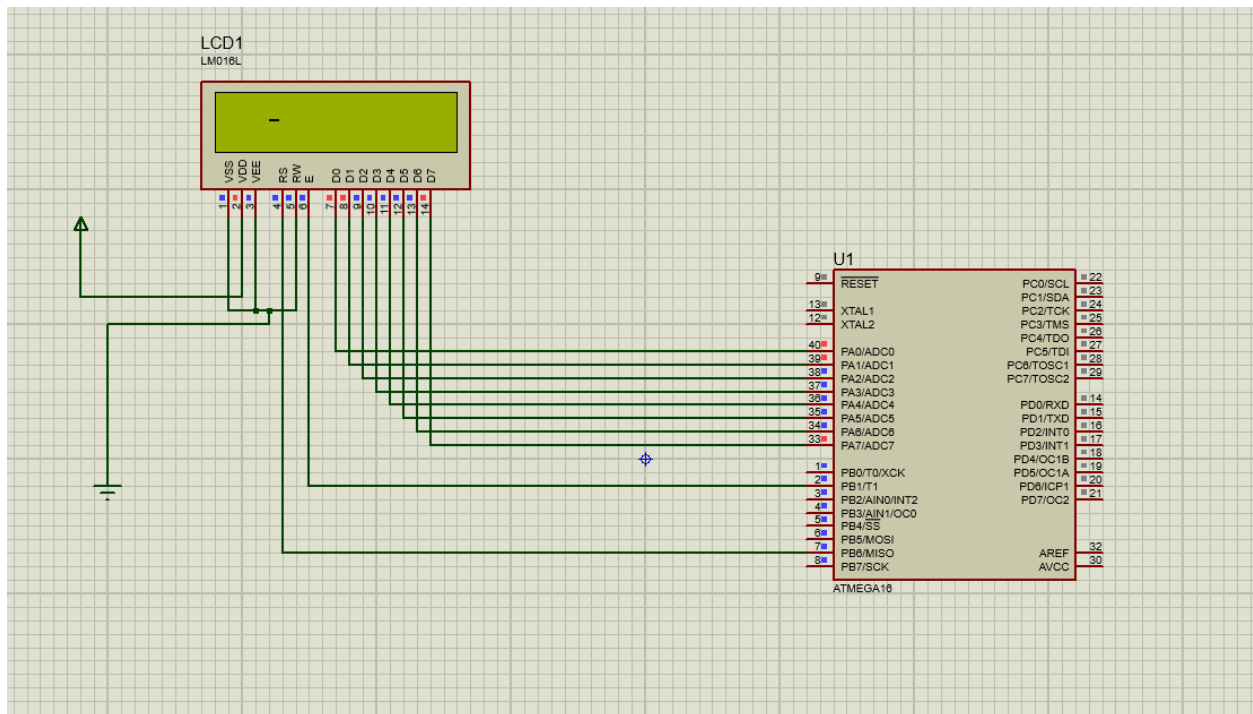
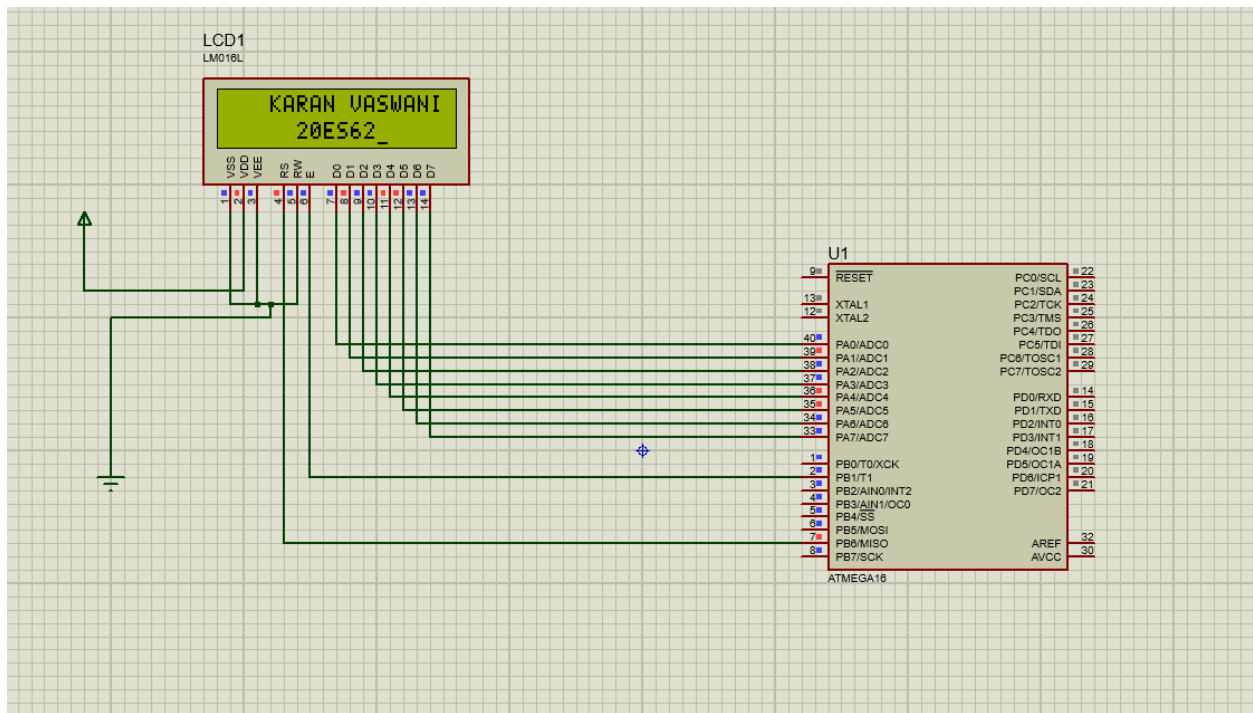
}
void DATA(unsigned char data)
{
    PORTB |=(1<<6);
    PORTA=data;
    PORTB |=(1<<1);
    _delay_ms(1);
    PORTB &=~(1<<1);
}
int main(void)
{
    DDRA=0XFF;
    DDRB=0XFF;

    while (1)
    {
        command(0x38);
        _delay_ms(1);
        command(0x0E);
        _delay_ms(1);
        command(0x06);
        _delay_ms(1);
        command(0x01);
        _delay_ms(1);
        command(0x83);
        _delay_ms(1000);
        unsigned char array[]="KARAN KUMAR";
        for (int z=0; z<13; z++)
        {
            DATA(array[z]);

        }
        _delay_ms(1);
        command(0xC5);
        _delay_ms(1);
        unsigned char array1[]="20ES62";
        for (int z=0; z<6; z++)
        {
            DATA(array1[z]);

        }
        _delay_ms(1000);
    }
}

```



3. Consider text written in task 1, make that text rotate left and right, the text should completely disappear towards right and then returns and disappear to left and this process continues infinite times.

```
#include <avr/io.h>
```

```

#include <util/delay.h>

void command(unsigned char comnd)
{
    PORTB &=~(1<<6);
    PORTA=comnd;
    PORTB |=(1<<1);
    _delay_ms(1);
    PORTB &=~(1<<1);
}

void DATA(unsigned char data)
{
    PORTB |=(1<<6);
    PORTA=data;
    PORTB |=(1<<1);
    _delay_ms(1);
    PORTB &=~(1<<1);
}

int main(void)
{
    DDRA=0XFF;
    DDRB=0XFF;

    while (1)
    {
        command(0x38);
        _delay_ms(1);
        command(0x0E);
        _delay_ms(1);
        command(0x06);
        _delay_ms(1);
        command(0x01);
        _delay_ms(1);
        command(0x83);
        _delay_ms(1);
        unsigned char array[]="KARAN VASWANI";
        for (int z=0; z<13; z++)
        {
            DATA(array[z]);
        }
        command(0xC5);
        _delay_ms(1);
        unsigned char array1[]="20ES62";
        for (int z=0; z<6; z++)
        {
            DATA(array1[z]);
        }
        for(int i=0; i<13; i++)
        {
            command(0x1C);
            _delay_ms(100);
        }
        for(int i=0; i<26; i++)
        {
            command(0x18);

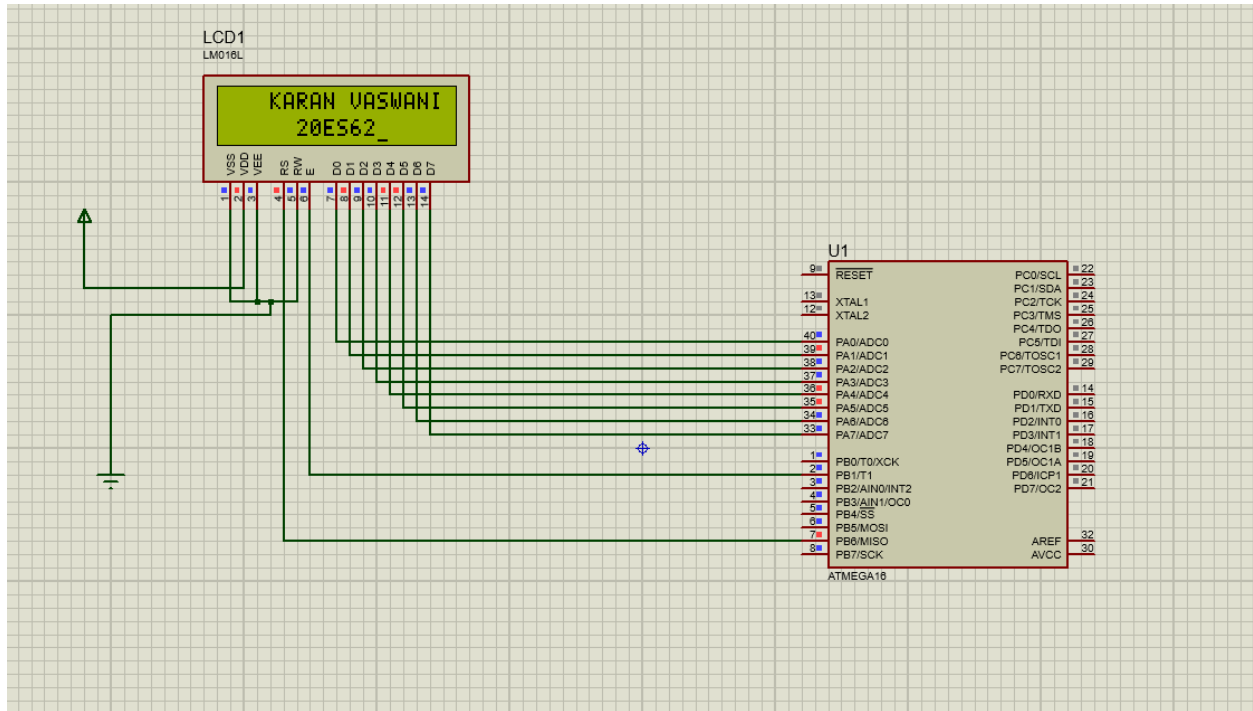
```

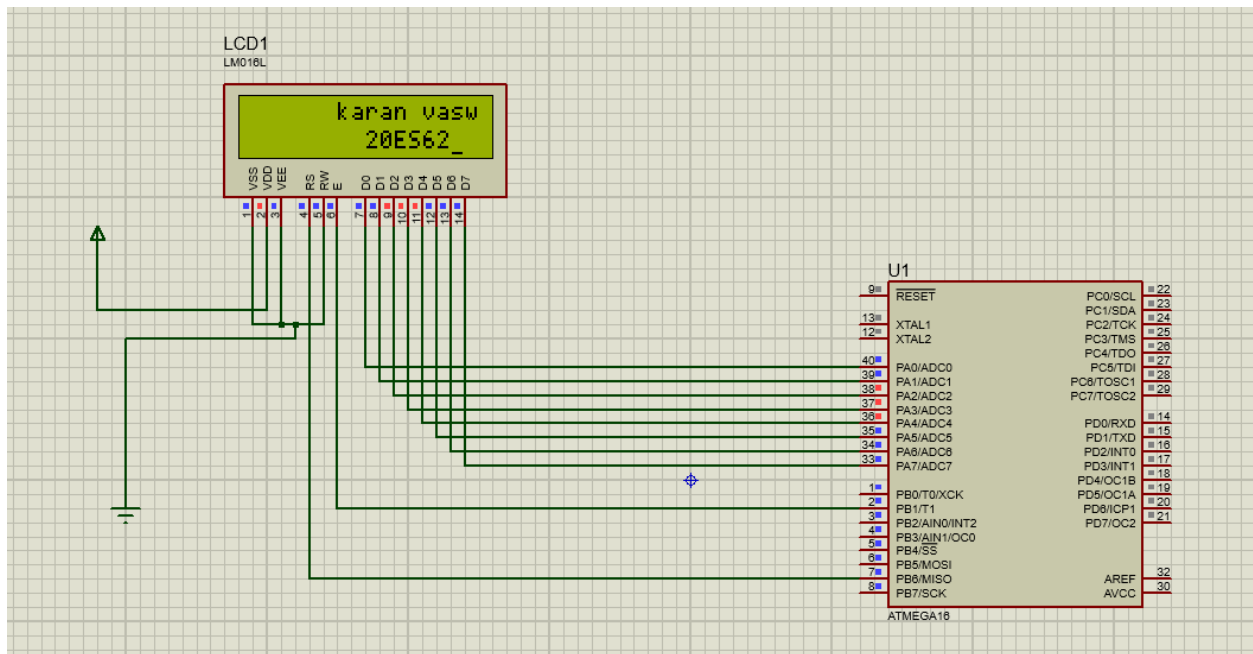


```

        _delay_ms(100);
    }
    for(int i=0; i<13; i++)
    {
        command(0x1C);
        _delay_ms(100);
    }
}

```





4. In task 1, make your Name visible all the time, whereas your roll number should blink continuously.

```
#include <avr/io.h>
#include <util/delay.h>
void lcd()
{
    command(0x38);
    _delay_ms(1);
    command(0x0E);
    _delay_ms(1);
}

void command(unsigned char comnd)
{
    PORTB &=~(1<<6);
    PORTA=comnd;
    PORTB |= (1<<1);
    _delay_ms(1);
    PORTB &=~(1<<1);
}

void DATA(unsigned char data)
{
    PORTB |= (1<<6);
    PORTA=data;
    PORTB |= (1<<1);
    _delay_ms(1);
    PORTB &=~(1<<1);
}

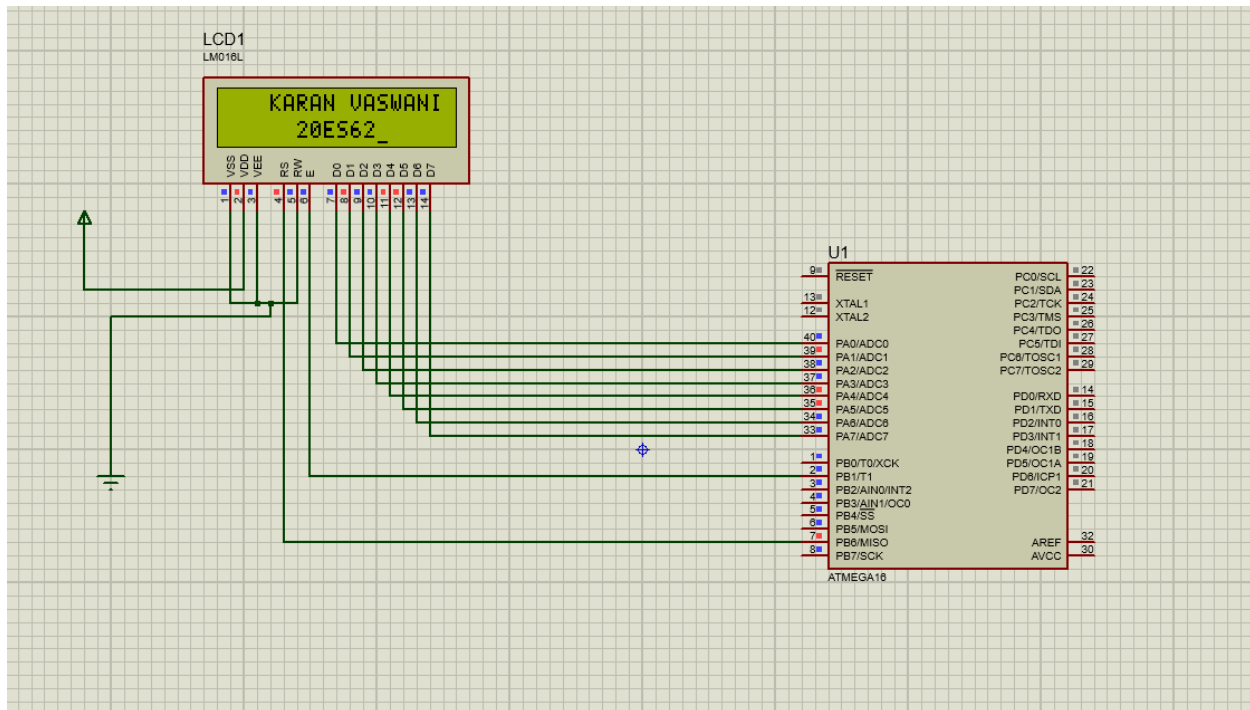
int main(void)
{
    DDRA=0XFF;
    DDRB=0XFF;
```

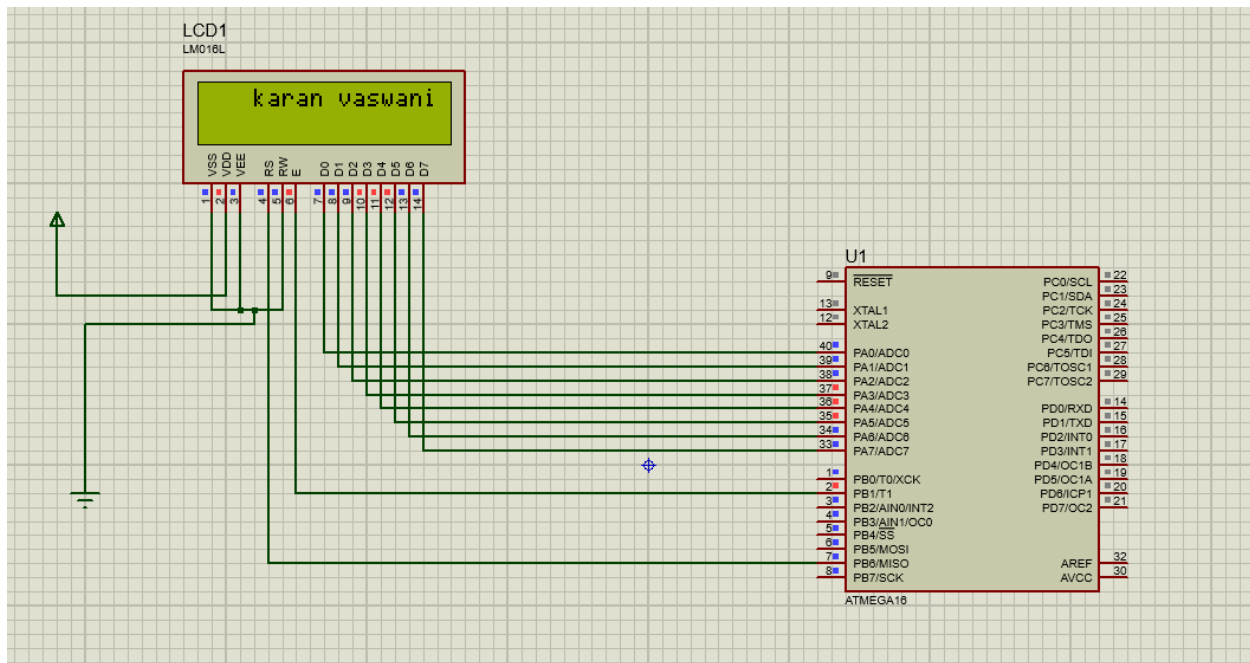
```

lcd();
command(0x83);
_delay_ms(1);
unsigned char array[]="KARAN VASWANI";
for (int z=0; z<10; z++)
{
    DATA(array[z]);
    _delay_ms(1);
}
command(0xC5);
_delay_ms(1);
unsigned char array1[]="20ES62";
for (int z=0; z<6; z++)
{
    DATA(array1[z]);
    _delay_ms(1);
}

while (1)
{
    command(0X30);
    _delay_ms(5);
    command(0x38);
    _delay_ms(1);
}
}

```





5. Write your name and roll number on LCD such that letters appear one by one.

```
#include <avr/io.h>
#include <util/delay.h>
void lcd()
{
    command(0x38);
    _delay_ms(1);
    command(0x0E);
    _delay_ms(1);
    command(0x06);
    _delay_ms(1);
    command(0x01);
    _delay_ms(1);
}

void command(unsigned char comnd)
{
    PORTB &=~(1<<6);
    PORTA=comnd;
    PORTB |=(1<<1);
    _delay_ms(1);
    PORTB &=~(1<<1);
}

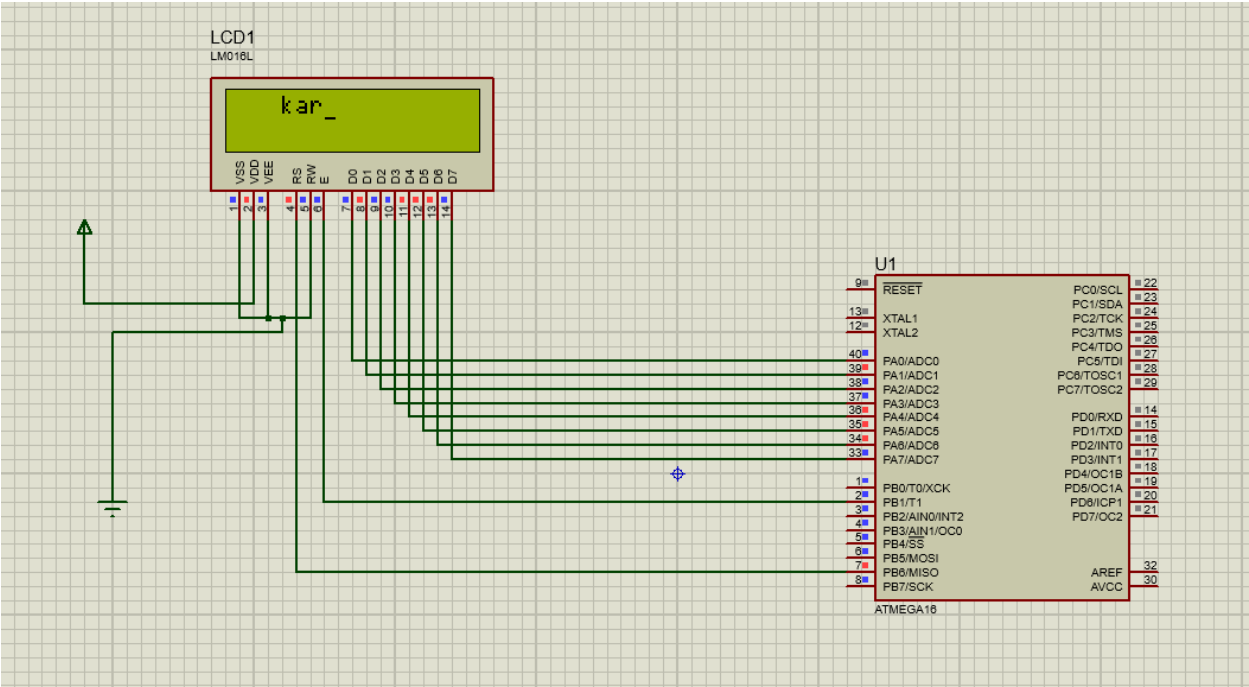
void DATA(unsigned char data)
{
    PORTB |=(1<<6);
```

```

PORTA=data;
PORTB |=(1<<1);
_delay_ms(1);
PORTB &=~(1<<1);
}
int main(void)
{
    DDRA=0XFF;
    DDRB=0XFF;

    while (1)
    {
        lcd();
        command(0x83);
        _delay_ms(1);
        unsigned char array[]="KARAN VASWANI";
        for (int z=0; z<13; z++)
        {
            DATA(array[z]);
            _delay_ms(1000);           //EVERY LETTER APPEAR AFTER 1 SEC
        }
        command(0xC5);
        _delay_ms(1);
        unsigned char array1[]="20ES62";
        for (int z=0; z<6; z++)
        {
            DATA(array1[z]);
            _delay_ms(1000);           //EVERY LETTER APPEAR AFTER 1 SEC
        }
    }
}

```





MEHRAN UNIVERSITY OF ENGINEERING AND TECHNOLOGY, JAMSHORO
DEPARTMENT OF ELECTRONIC ENGINEERING



INTRODUCTION TO EMBEDDED SYSTEMS (ES-314)

Batch: 20ES (5th Semester)

Lab Experiment #06

To Study the Interfacing of Matrix Keypad with AVR ATmega16/32

Name <u>karan</u>	Roll # <u>20ES062</u>
Signature of Lab Tutor _____	Date _____

RUBRICS:

Performance Metric	CONDUCTING EXPERIMENT	PARTICIPATION	TEAMWORK	USE OF MODERN ENGINEERING TOOLS	DATA ANALYSIS	CALCULATION AND CODING	OBSERVATION/ PROGRAM RESULTS	Total Score
	1	0.5	0.5	1		1	1	05
Obtained								

OBJECTIVE(S)

#	Topic	# Of Lectures	CLO	Taxonomy level
1	The purpose of this lab is to determine the different aspects of Keypad interfacing/programming with an ATMEGA 16.	3	4,5	P3, A2
2	To imitate and execute the programs to display data on LCD which is given by keypad by interfacing these two with ATmega16/32.			

OUTCOME(S)

a. An ability to use the techniques, skills, and modern engineering tools necessary for engineering practice.	PLO5: Modern Tool Usage
b. An ability to function on multi-disciplinary teams	PLO9: Individual and Teamwork

REQUIRED TOOLS AND SOFTWARES:

- Proteus Design Suite
- AVR Studio
- WinAVR

- 4x4 Matrix Keypad **INTRODUCTION:**

Keypads are widely used input devices in various Electronic and Embedded system projects. Keypad provides input such as numbers, alphabets, and symbols. Nowadays, various keypads are

being used but 4×4 matrix keypad is commonly used in embedded applications. Keypads can be seen in telephones, ATM machines, weight scales, fuel injectors and other daily applications.

4×4 Matrix Keypad:

The 4×4 keypad is usually used to send input in a project. It has 16 keys in total, which means it can have 16 different values. The 4×4 Keypad Module is a matrix type non- encoded consisting of 16 keys. The keys are arranged in 4 rows and 4 columns format. Each row and column are connected through the pins outside, such as pin R1 – R4 are specified as rows while C1 – C4 as columns. The schematic and physical layout of 4*4 matrix keypad are shown in figures 1 & 2, respectively. Physical keys label (name) can be changed according to the application; thus, program then also needed to be change based on the physical name of keys.

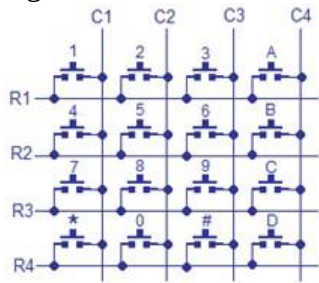


Figure 1: 4x4 matrix layout

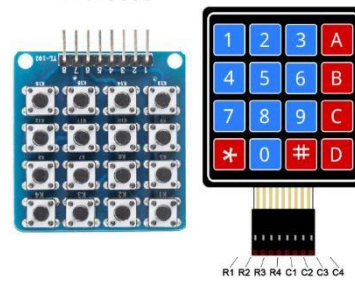


Figure 2: Physical layout of 4x4 matrix keypad

Note: Different Keypad manufactures use their own labels and pin locations for rows and columns.

Interfacing of Keypad:

Keyboards/Keypads are grouped in a matrix of rows and columns. The microcontroller accesses both rows and columns through ports (A-D in case of ATmega16/32). Therefore, with a single 8bit port, a 4*4 matrix keypad can be connected to a microcontroller. When a key is pressed, rows and columns make a connection and microcontroller detects which key has been pressed. All 4 columns are connected as an input to the lower nibble (PB0-PB3) of PORTB while the 4 rows are connected as an output to the upper nibble (PB4-PB7) of PORTB. The LCD can also be connected to show the value of the pressed key. Refer figure 3, LCD is connected to the PORTC and PORTD (PD6-PD7).

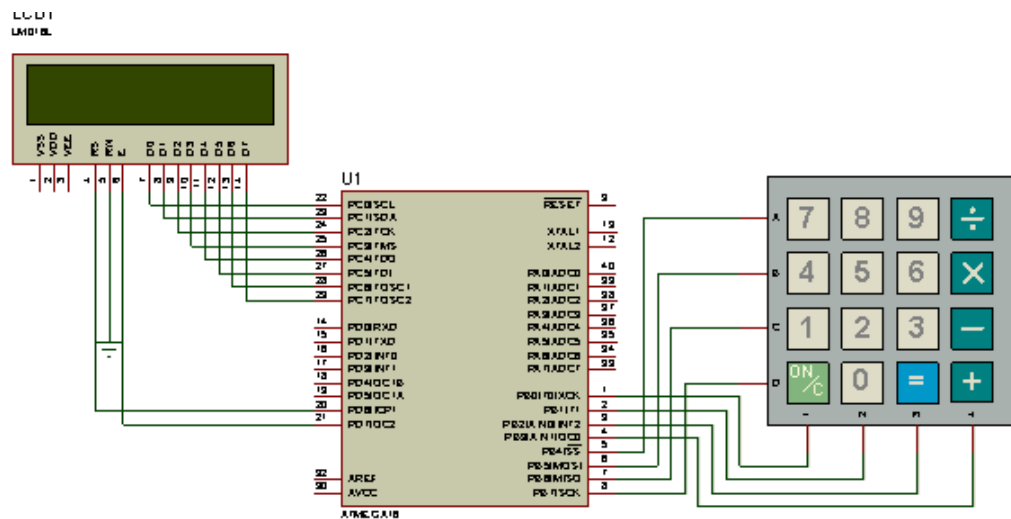


Figure 3: 4x4 Matrix Keypad and LCD connection to Ports

Initially, all columns are set to 1 (High). This will be accomplished by activating the internal pullup resistors of PORTB i.e., lower nibble (PB0-PB3) and all rows will be set to 0 (LOW) by sending 0s at the top nibble (PB4-PB7) of PORTB. When a key is pressed, the column pin that is connected to the row pin, makes the entire column state as low. To identify the exact pin at the column, **each row requires to scan by sending 1 (HIGH) to read the state of column pins**. The column which changes the state from 1 (HIGH) to 0 (Low) will be the location of the pressed key (passes the Low signal from Row to Column pin).

Detecting the Key Press:

The microcontroller's function is to scan the keypad continuously to detect and identify which key is pressed. This is explained by the following steps.

Stage 1: No Key Pressed

In Idle state the Columns C1-C4 = 1111 and Rows R1-R4 = 0000, the microcontroller waits until any key is to be pressed.

Stage 2: Key Pressed

Now, if the key 5 is pressed. The entire 2nd Column will change to low since the LOW (0) signal from Row pin flows through Column pin. Thus, the column values will become C1-C4 = 1011.

Stage 3: Scanning the Pressed Key

Now, the microcontroller knows the column of the key that has been pressed. But 4 keys exist in that column. To detect the exact key, scanning of the Rows is necessary. The scanning happens by sending 1 (HIGH) to the Row pins one by one and reading at column pins. This will return a HIGH (1) Signal only at the pressed key location.

Scanning Row 1:

The microcontroller sets R1 as 1 (HIGH) but also monitors C1-C4 for state change. If the key is not from Row 1, it will not make any change in C1-C4. So, no key has been pressed from R1.

Scanning Row 2:

Microcontroller then sets R2 as 1 (HIGH) and start monitoring C1-C4. Here, C2 changes from LOW (0) to HIGH (1). Thus, the key location is R2C2 which is 'Key 5' in keypad. Therefore, the microcontroller maps the character with sketch and prints it accordingly.

Scanning Row 3:

After that, the microcontroller sets R3 as 1 (HIGH) and again starts monitoring C1-C4 for state change. Since the pressed key is not from Row 3; thus, it will not make any change in C1-C4. So, no key has been pressed from R3.

Scanning Row 4:

The microcontroller now sets R4 as 1 (HIGH) and monitors C1- C4 for state change. Since the pressed key is also not from Row 4, then again it will not make any change in C1: C4. Thus, no key has been pressed from R4.

The key detection method is standard for all matrix keypads but the process of detecting the pressed key may vary. There are IC chips such as MM74C923 of National semiconductor that perform keyboard scanning and decoding all in one chip.

Key Debouncing:

When a key (switch) has been pressed/released, generally it changes one state to another; thus, the key will oscillate between two states for a certain time period until it is settled down to a new state,

called key bouncing. The key bouncing generates multiple signals when the microcontroller reads any pressed/released key from a keypad. The key debouncing technique is used to eliminate this false switching of keys. In key debouncing, the microcontroller waits until the key will have a stable state after it is pressed from the keypad. The key debouncing can be done either by hardware or by software.

Keypad Program:

```
#include <avr/io.h>
#include <util/delay.h>
#define KEY_PRT PORTB
#define KEY_DDR DDRB
#define KEY_PIN PINB

unsigned char keypad[4][4] = {
    {'7','8','9','/'},
    {'4','5','6','*'},
    {'1','2','3','-'},
    {'C','0','=','+'}};

int main (void)
{

    unsigned char colloc, rowloc;
    KEY_DDR = 0xFF;          /* set port direction as input-output */
    KEY_PRT = 0xFF;
    DDRA = 0xFF;

    while(1)
    {
        do
        {
            KEY_PRT &= 0x0F;    /* mask PORT for column read only */
            colloc = (KEY_PIN & 0x0F); /* read status of column */
        }while(colloc != 0x0F);

        do
        {
            do
            {
                _delay_ms(20);    /* 20ms key debounce time */
                colloc = (KEY_PIN & 0x0F); /* read status of column */
            }while(colloc == 0x0F); /* check for any key press */

            _delay_ms (20);    /* 20 ms key debounce time */
            colloc = (KEY_PIN & 0x0F);
        }while(colloc == 0x0F);

        while(1)
        {
            /* now check for rows */
            KEY_PRT = 0xEF;    /* check for pressed key in 1st row */
            colloc = (KEY_PIN & 0x0F);
            if(colloc != 0x0F)
            {
                rowloc = 0;
                break;
            }
        }
    }
}
```

```

    }

    KEY_PRT = 0xDF;          /* check for pressed key in 2nd row */
    colloc = (KEY_PIN & 0x0F); if(colloc != 0x0F)
    {
rowloc = 1;
        break;
    }

    KEY_PRT = 0xBF;          /* check for pressed key in 3rd row */

    colloc = (KEY_PIN & 0x0F);
    if(colloc != 0x0F)
    {
rowloc = 2;
        break;
    }

    KEY_PRT = 0x7F;          /* check for pressed key in 4th row */
    colloc = (KEY_PIN & 0x0F);
    if(colloc != 0x0F)
    {
rowloc = 3;
        break;
    }
    }

    if(colloc == 0x0E)        PORTA=
(keypad[rowloc][0]);        else
if(colloc == 0x0D)          PORTA=
(keypad[rowloc][1]);        else
if(colloc == 0x0B)          PORTA=
(keypad[rowloc][2]);
    else
        PORTA= (keypad[rowloc][3]);
} return;
}

```

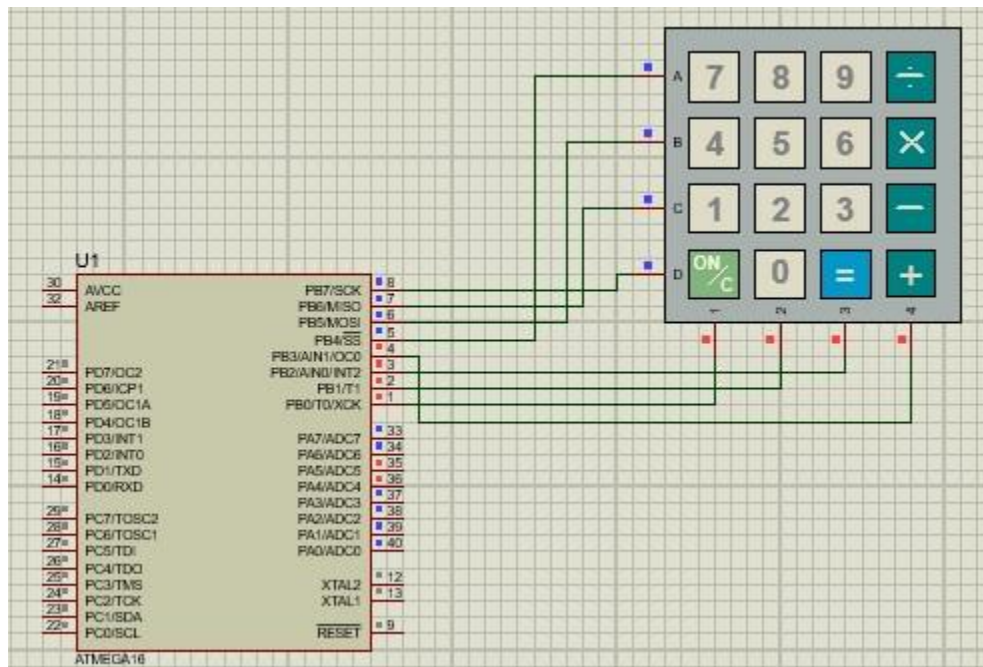


Figure 4: Proteus Output when '0' key is pressed

Lab Tasks:

- Imitate source code to display the key that has been pressed from a keypad after display "KEY PRESSED IS" on 2nd row of 16x2 character LCD connected at PORTC and PORTD. On 1st row of the LCD display your Roll Number.

Note: Attach the program files C Programing code and simulation result snapshots along with the handouts.

- 1) Imitate a source code to display the key that has been pressed from a keypad after display "KEY PRESSED IS" on 2nd row of 16x2 character LCD connected at PORTC and PORTD. On 1st row of the LCD display your Roll Number.

```
#include <avr/io.h>
#include <util/delay.h>

void LCD_CMD(unsigned char CMD)
{
    PORTD &=~(1<<6);
    PORTC=CMD;
    PORTD |= (1<<7);
    _delay_ms(1);
    PORTD &=~(1<<7);
}

void LCD_DATA(unsigned char DATA)
```

```

{
PORTD |=(1<<6);
PORTC=DATA;
PORTD |=(1<<7);
_delay_ms(1);
PORTD &=~(1<<7);
}
int main(void)
{

unsigned char COL, ROW, key,i;
DDRB=0XF0;
PORTB=0XFF;
DDRC=0XFF;
DDRD=0XFF;

unsigned char KEYPAD[4][4]={  {'7','8','9','/'},
                                {'4','5','6','*'},
                                {'1','2','3','-'},
                                {'C','0','=','+'}};

unsigned char roll[6]="20ES62";
unsigned char key_pressed[14]="KEY PRESSED IS";
unsigned char space[16]="          ";
LCD_CMD(0X38);
_delay_ms(1);
LCD_CMD(0X0E);
_delay_ms(1);
LCD_CMD(0X06);
_delay_ms(1);
LCD_CMD(0X01);
_delay_ms(1);
LCD_CMD(0x85);
_delay_ms(10);
for(i=0; i<6; i++)
{
    LCD_DATA(roll[i]);
    _delay_ms(10);
}
LCD_CMD(0xC1);
_delay_ms(10);
for(i=0; i<14; i++)
{
    LCD_DATA(key_pressed[i]);
    _delay_ms(10);
}
while (1)
{
    do
    {
        PORTB &=0X0F;
        COL=(PINB & 0X0F);
    }
    while(COL !=0X0F);
    do
    {

```



```

do
{
    _delay_ms(20);
    COL=(PINB & 0X0F);
}
while(COL==0X0F);
    _delay_ms(20);
    COL=(PINB & 0X0F);
}
while(COL==0X0F);
while(1)
{
    PORTB=0XEF;
    COL=(PINB & 0X0F);
    if(COL !=0X0F)
    {
        ROW=0;
        break;
    }
    PORTB=0XDF;
    COL=(PINB & 0X0F);
    if(COL !=0X0F)
    {
        ROW=1;
        break;
    }
    PORTB=0XBF;
    COL=(PINB & 0X0F);
    if(COL !=0X0F)
    {
        ROW=2;
        break;
    }
    PORTB=0X7F;
    COL=(PINB & 0X0F);
    if(COL !=0X0F)
    {
        ROW=3;
        break;
    }
}
if(COL==0X0E)
    key=(KEYPAD[ROW][0]);
else if(COL==0X0D)
    key=(KEYPAD[ROW][1]);
else if(COL==0X0B)
    key=(KEYPAD[ROW][2]);
else
    key=(KEYPAD[ROW][3]);

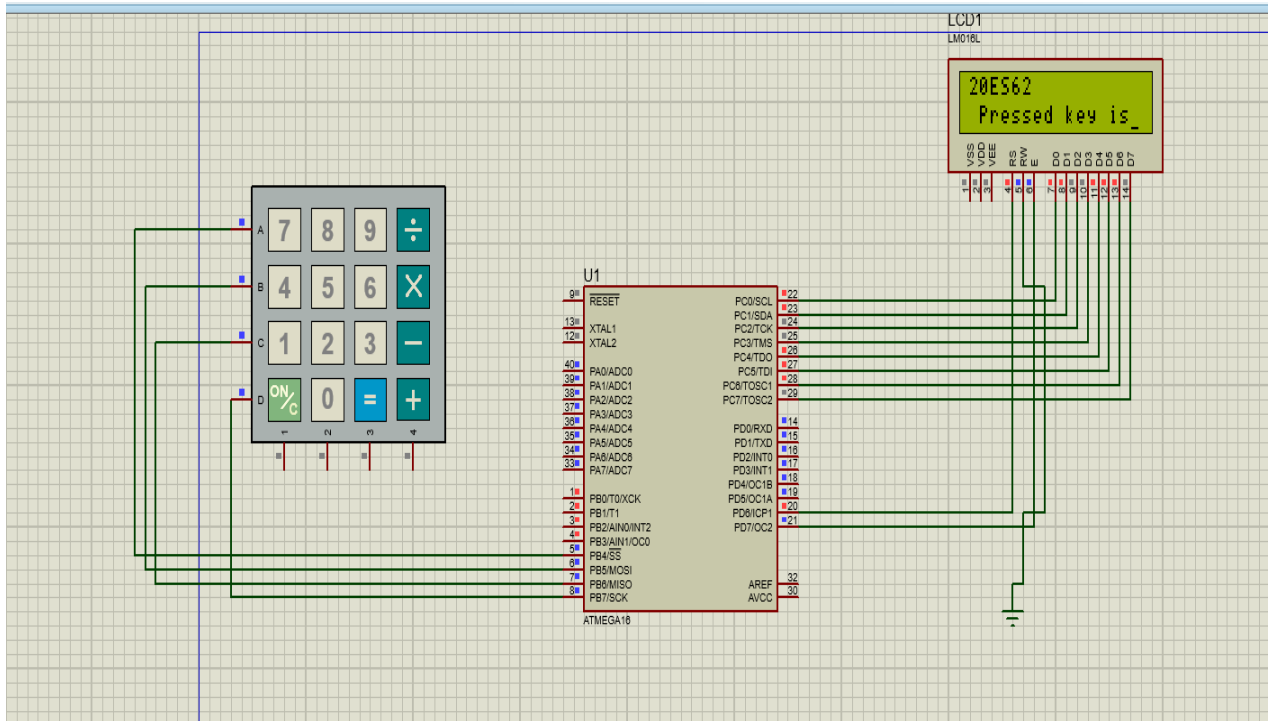
    LCD_CMD(0xC0);
    _delay_ms(10);
    for(i=0; i<16; i++)
        LCD_DATA(space[i]);
    _delay_ms(5);
    LCD_CMD(0xC7);

```

```

    LCD_DATA(key);
}
}

```





MEHRAN UNIVERSITY OF ENGINEERING AND TECHNOLOGY, JAMSHORO
DEPARTMENT OF ELECTRONIC ENGINEERING



INTRODUCTION TO EMBEDDED SYSTEMS (ES-314)

Batch: 20ES (5th Semester)

Lab Experiment #07

To Learn the Usage of External Interrupts in AVR Microcontroller

ATMega16/32

Name <u>karan</u>	Roll # <u>20ES062</u>
Signature of Lab Tutor _____	Date _____

RUBRICS:

Performance Metric	CONDUCTING EXPERIMENT	PARTICIPATION	TEAMWORK	USE OF MODERN ENGINEERING TOOLS	DATA ANALYSIS	CALCULATION AND CODING	OBSERVATION/ PROGRAM RESULTS	Total Score
	1	0.5	0.5	0.5	0.5	1	1	05
Obtained								

OBJECTIVE(S)

#	Topic	# Of Lectures	CLO	Taxonomy level
1	The purpose of this lab is to Imitate and execute the programs that can work with AVR Microcontroller's <i>External Interrupts</i> .	3	4,5	P3, A2

OUTCOME(S)

a. An ability to use the techniques, skills, and modern engineering tools necessary for engineering practice.	PLO5: Modern Tool Usage
b. An ability to function on multi-disciplinary teams	PLO9: Individual and Teamwork

REQUIRED TOOLS AND SOFTWARES:

- Proteus Design Suite
- AVR Studio
- WinAVR

INTRODUCTION:

A single Microcontroller can serve several devices. It sends or receives data from these devices which can be done by following methods.

- Interrupts

- Polling

INTERRUPTS:

Whenever any device needs microcontroller's services, the device notifies this to the microcontroller by sending an interrupt signal. Upon receiving an interrupt signal, the microcontroller stops whatever it is doing and serves the device. The program associated with an interrupt is called the Interrupt Service Routine (ISR) or Interrupt Handler.

POLLING:

In polling, Microcontroller continuously monitor the status of given device; when the status/condition meets, it performs the service. After that, it moves on to monitor the next device until each one of the devices are serviced. But it is not an efficient use of Microcontroller. On RESET, all interrupts will be disabled.

Interrupt Service Routine (ISR):

For every Interrupt there must be an Interrupt Service Routine (ISR) or Interrupt Handler. When an Interrupt is invoked, Microcontroller executes an ISR. There is an address for most of interrupts where an ISR can be placed. The group of Memory locations that hold these addresses of ISRs are called Interrupt Vector Table.

Steps During Execution of Interrupts:

When an Interrupt is activated, microcontroller completes its current instruction, store PC contents to Stack pointer then Jumps to ISR by executing it and at the end of ISR there is a Return Instruction to jump back to main program by getting the contents of PC from Stack pointer.

REGISTERS:

- SREG (Status Register) - Enabling Global Interrupts
- GICR (General Interrupt Control Register) - For masking External Interrupts
- MCUCR (MCU Control Register) - Used to set Activating Logic/Edge of INT1
- MCUCSR (MCU Control & Status Register) - Used to set Activating Logic/Edge of INT2
- GIFR (General Interrupt Flag Register) - Used to check the status of Interrupt.

Status Register (GICR): It is used to enable or disable all interrupts in AVR microcontroller.

Bit	7	6	5	4	3	2	1	0	
0x3F (0x5F)	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit 7 – I: Global Interrupt Enable (SREG)

The Global Interrupt Enable bit must be set for the interrupts to be enabled. The individual interrupt enable control is then performed in separate control registers.

If the Global Interrupt Enable Register is cleared, none of the interrupts are enabled independent of the individual interrupt enable settings. Finally, you must set the I bit in SREG enabling internal and external interrupts that have been set to operate. To stop all interrupts, just set SREG_I to 0.

General Interrupt Control Register (GICR): We can enable/disable external interrupts by the GICR register.

Bit	7	6	5	4	3	2	1	0	
	INT1	INT0	INT2	–	–	–	IVSEL	IVCE	GICR
Read/Write	R/W	R/W	R/W	R	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – INT1: External Interrupt Request 1 Enable**

0: Disable external interrupt

1: Enable external interrupt

- **Bit 6 – INT0: External Interrupt Request 0 Enable**

0: Disable external interrupt

1: Enable external interrupt

- **Bit 5 – INT2: External Interrupt Request 2 Enable**

0: Disable external interrupt

1: Enable external interrupt

MCU Control Register (MCUCR): To define a level trigger or edge trigger on external INT0 and INT1 pins MCUCR register is used.

Bit	7	6	5	4	3	2	1	0	
	SM2	SE	SM1	SM0	ISC11	ISC10	ISC01	ISC00	MCUCR
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 1-0: ISC01, ISC00 (Interrupt Sense Control bits).** These bits define the level or edge that triggers the INT0 pin.

ISC01	ISC00		Description
0	0		The low level on the INT0 pin generates an interrupt request.
0	1		Any logical change on the INT0 pin generates an interrupt request.
1	0		The falling edge on the INT0 pin generates an interrupt request.
1	1		The rising edge on the INT0 pin generates an interrupt request.

- **Bit 3-2: ISC11, ISC10 (Interrupt Sense Control bits)** These bits define the level or edge that triggers the INT1 pin.

ISC01	ISC00		Description
0	0		The low level on the INT1 pin generates an interrupt request.
0	1		Any logical change on the INT1 pin generates an interrupt request.
1	0		The falling edge on the INT1 pin generates an interrupt request.
1	1		The rising edge on the INT1 pin generates an interrupt request.

General Interrupt Flag Register (GIFR): The bits of GIFR register are set when an interrupt occurs and cleared automatically when it is processed.

Bit	7	6	5	4	3	2	1	0	
	INTF1	INTF0	INTF2	–	–	–	–	–	GIFR
Read/Write	R/W	R/W	R/W	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

Working Steps:

- Specify Vector Table of ISR
- Start main program
- Configure ports
- Configure Interrupts as level/ edge triggered. (MCUCR & MCUSR)
- Enable individual interrupts (GICR)
- Enable global Interrupt (SEI)
- Code ISR

Interrupts Programming in C:

Following conditions are necessary in programming of interrupts:

Header File:

```
#include "avr/interrupt.h"
```

To Enable/Disable Global Interrupt (SREG) cli()

Clear Interrupt

(Disable) sei()

Set Interrupt

(Enable)

Working Steps for C Interrupt Programming:

Include Interrupt file

```
#include <avr/interrupt.h> sei ( ) &
```

cli () instructions for I Flag.

Define ISR

ISR (interrupt vector name)

```
{
```

```
// our program
```

```
}
```

Hint: For interrupt vector name you must use ISR name from Table-1 For example:

```
ISR (INT0_vect)
{
    // service routine
}
```

Interrupt Vector Table in C:

Table 1: Interrupt vector Name for ATmega32/ATmega16 in WinAVR

Interrupt	Vector Name in WinAVR
External Interrupt request 0	INT0_vect
External Interrupt request 1	INT1_vect
External Interrupt request 2	INT2_vect
Time/Counter2 Compare Match	TIMER2_COMP_vect
Time/Counter2 Overflow	TIMER2_OVF_vect
Time/Counter1 Capture Event	TIMER1_CAPT_vect
Time/Counter1 Compare Match A	TIMER1_COMPA_vect
Time/Counter1 Compare Match B	TIMER1_COMPB_vect
Time/Counter1 Overflow	TIMER1_OVF_vect
Time/Counter0 Compare Match	TIMER0_COMP_vect
Time/Counter0 Overflow	TIMER0_OVF_vect
SPI Transfer complete	SPI_STC_vect
USART, Receive complete	USART0_RX_vect
USART, Data Register Empty	USART0_UDRE_vect
USART, Transmit Complete	USART0_TX_vect
ADC Conversion complete	ADC_vect
EEPROM ready	EE_RDY_vect
Analog Comparator	ANALOG_COMP_vect
Two-wire Serial Interface	TWI_vect
Store Program Memory Ready	SPM_RDY_vect

Program#01: Write a Program using Interrupts which toggles PC.3 & increment count at PORT A whenever SW on PD.2 goes LOW.

```
#include <avr/io.h> #include
<avr/interrupt.h> int
main(void)
{
    DDRC=1<<3;
    DDRD=1<<2; DDRA=
    0xFF;
    GICR=(1<<INT0);
    sei(); while
    (1);
}
ISR(INT0_vect)
{
    PORTC^=(1<<3);
    PORTA++;
}
```

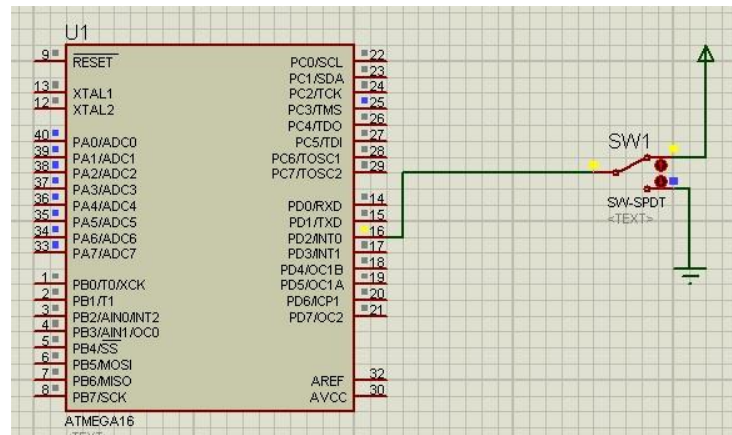



Figure 1: Proteus Output when the switch is high

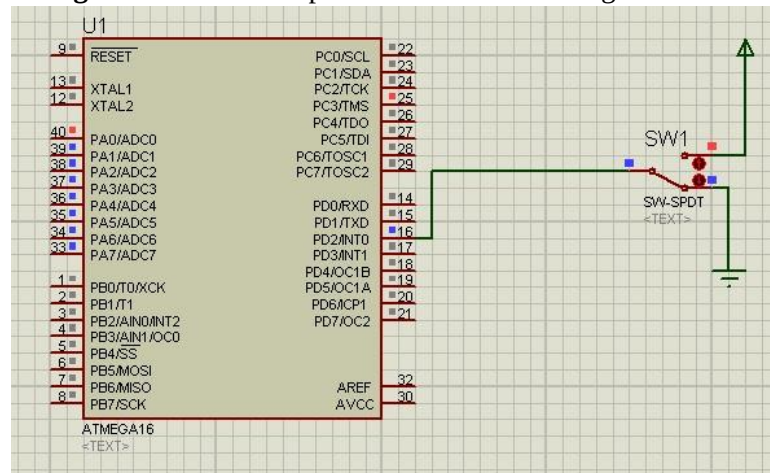


Figure 2: Proteus Output when the switch is low (i.e., PC.3 toggling, and Port A starts counting)

Program#02: Write a Program using Interrupts which toggles PC.3 whenever SW on PD.2 goes LOW, include 3 seconds delay in interrupt section. Also toggle PC.2 with a delay of 1sec in main program.

```
#include <avr/io.h> #include
<avr/interrupt.h> int
main(void)
{
DDRC|=(1<<3) | (1<<2);
DDRD=1<<2; GICR=(1<<INT0);
sei(); while
(1)
{
PORTC^=(1<<2);
_delay_ms(1000);
}
}
ISR(INT0_vect)
{
```

```

PORTC^=(1<<3);
_delay_ms(4000);

}

```

Note: Write above program and analyze the results.

Lab Tasks:

Imitate a source code by utilizing the external interrupt INT_1 with falling edge trigger of the ATmega16/32. which perform one (1) of two (2) operations. On start-up OPERATION 1 is executed and continues indefinitely. If the interrupt is executed by pressing push-button switch OPERATION 1 should pause and OPERATION 2 should be carried out after which OPERATION 1 should be resumed.

OPERATION 1 - A roll action should be performed using the LEDs at PORTA. The first LED should light and roll down to the last LED then back to the first LED. This operation should be continuous.

OPERATION 2 - ALL the LEDs must blink five X times. where X represents last numerical digit of your Roll number (e.g., 20ESXX).

```

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

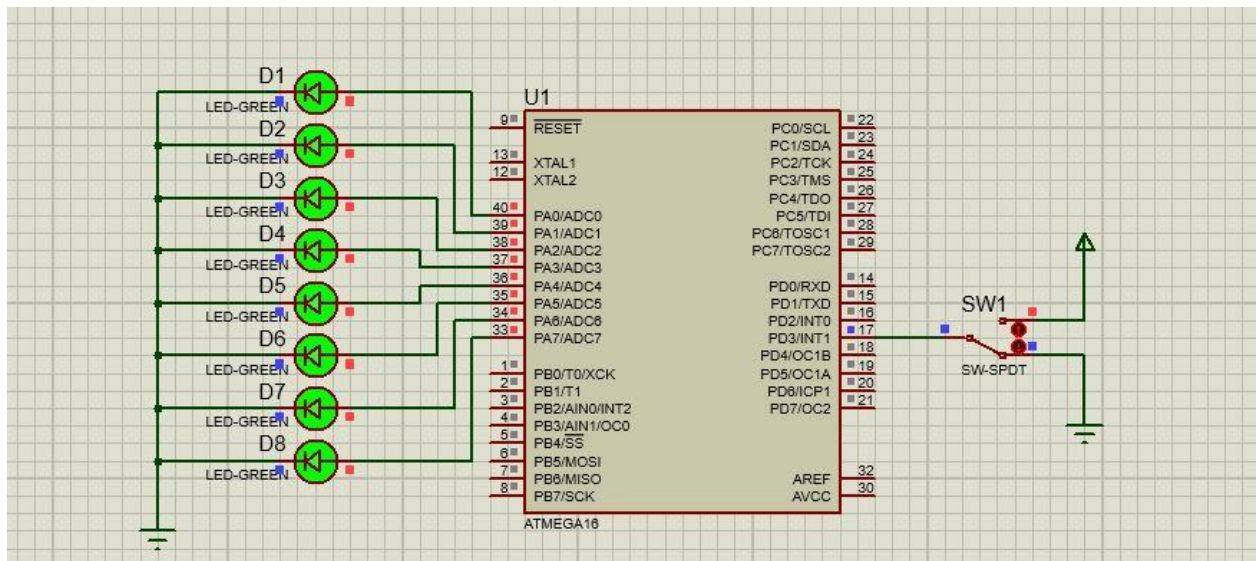
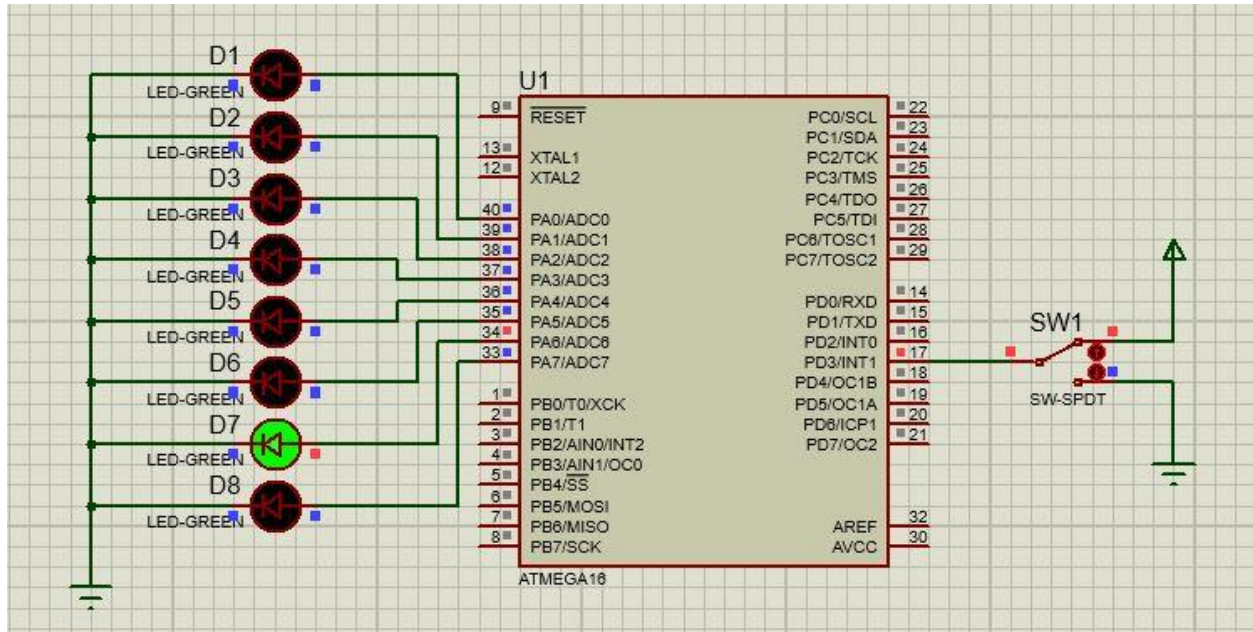
int main(void)
{
    DDRD=0x00;
    DDRA=0xFF;
    GICR=(1<<INT1);
    MCUCR=(1<<ISC11)|(0<<ISC10);
    sei();
    while (1)
    {
        for (int i=0; i<8; i++)
        {
            PORTA=(1<<i);
            _delay_ms(200);
        }
    }
    ISR(INT1_vect)
    {
        for (int j=0; j<30; j++)
        {
            PORTA=0xFF;

```

```

_delay_ms(200);
PORTA=0x00;
_delay_ms(200);
}
}

```





INTRODUCTION TO EMBEDDED SYSTEMS (ES-314)

Batch: 20ES (5th Semester)

Lab Experiment #08

To Perform Analog to Digital conversion using ADC with AVR ATmega16/32

Name	<u>KARAN</u>	Roll #	<u>20ES062</u>
Signature of Lab Tutor	_____	Date	_____

RUBRICS:

Performance Metric	CONDUCTING EXPERIMENT	PARTICIPATION	TEAMWORK	USE OF MODERN ENGINEERING TOOLS	DATA ANALYSIS	CALCULATION AND CODING	OBSERVATION/ PROGRAM RESULTS	Total Score
	0.5	0.5	0.5	0.5	1	1	1	05
Obtained								

OBJECTIVE(S)

#	Topic	# Of Lectures	CLO	Taxonomy level
1	The purpose of this lab is to approach different features of an ADC and Determine the working of different registers of a built-in ADC in ATMEGA 16.	3	4,5	P3, A2
2	Imitate and execute the programs to convert data from analog sensors by using built-in ADCs in ATmega16/32.			

OUTCOME(S)

a. An ability to use the techniques, skills, and modern engineering tools necessary for engineering practice.	PLO5: Modern Tool Usage
b. An ability to function on multi-disciplinary teams	PLO9: Individual and Teamwork

REQUIRED TOOLS AND SOFTWARES:

- Proteus Design Suite
- AVR Studio
- WinAVR

INTRODUCTION:

In the real world, most of the data has been characterized by analog signals. To manipulate the data, microprocessor is being used. But the microprocessor requires the conversion of analog signals into digital signals to be able to read, understand and manipulate the data.

Analog to Digital Converter:

An Analog to Digital Converter (ADC) is a very useful feature that converts an analog voltage at a pin to a digital number. To interface to the surrounding analog world, the analog data requires the conversion into digital data. Figure 1 shows the basic mechanism to use ADCs i.e., convert an analog signal to digital so that it can be processed by a processing unit.

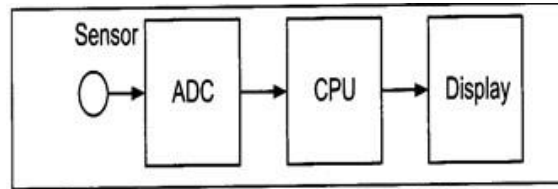


Figure 1: Microcontroller Connection to Sensor via ADC

An ADC consists of multiple pins, like Analog Input, Reference Voltage, Start Conversion, End of Conversion, and Data Out.

V_{ref} (Reference Voltage) is the maximum possible input voltage that can be given to ADC, or the input voltage at which all output lines become HIGH.

For example: If the Reference voltage (V_{ref}) is set to 3 volts by user, then at input voltage = 3 volts, all output pins (V_{out}) become HIGH, whether it is 4-bit, 8-bit, 16-bit or any-bit ADC, all output bits will be HIGH when analog input voltage is equal to V_{ref} .

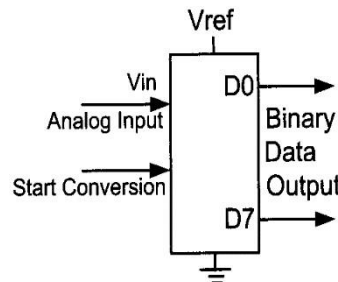


Figure 2: Reference voltage representation

Resolution of an ADC:

The resolution means smallest change in the input voltage that causes one-bit change in output. Resolution depends on V_{ref} and n (no. of bits of ADC).

$$Res = \frac{V_{ref}}{2^n}$$

Lower the value of resolution, better it is. Hence, either by decreasing value of V_{ref} or by increasing no. of bits, resolution of ADC can be improved.

- $V_{ref} = 5$ volts, and $n = 2$

The voltage between 0 and 5 volts is divided into 2^2 parts, as shown in the figure 3. In the figure, it is clearly shown that 5 volts (V_{ref}) is divided into $2^2 = 4$ parts, each part is of 1.25 volts (Resolution), when input voltage is between 0 and 1.25 volts, output will be 00, when input is

between 1.25 and 2.50 volts, output will be 01 digital number, when analog input is between 2.50 v and 2.75 volts, digital output will be 10 and 11 for greater than 3.75 and below V_{ref} .

$V_{ref}=5$	
3.75 v	11
2.50 v	10
1.25 v	01
0 Volts	00

Figure 3: Digital values when $V_{ref}=5$ volts, and $n=2$

In this no change can be found, if analog sensor value is changed from 0 to 1 volt, output will be same 00 all the time between 0 volts and 1.24 volts. To add more accuracy, dividing V_{ref} into more parts, like dividing V_{ref} (5v) into 5 parts will give resolution of 1v means little more accuracy.

- **$V_{ref}=2$ volts and $n=2$**

The voltage between 0 volts and 2 volts is divided into 2^2 parts, as shown in figure 4. By decreasing V_{ref} a better resolution in 2-bit ADC can be achieved. In the figure, it is clearly shown that 2 volts (V_{ref}) has been divided in $2^2=4$ parts. Each part is of 0.5 volts (Resolution), when input voltage is between 0 and 0.50 volts, output will be 00, when input is between 0.50 and 1.00 volts, output will be 01 digital number, when analog input is between 1.00 v and 1.50 volts, digital output will be 10 and 11 for greater than 1.50 and below V_{ref} .

$V_{ref}=2$	
1.50 v	11
1.00 v	10
0.50 v	01
0 Volts	00

Figure 4: Digital values when $V_{ref}=5$ volts, and $n=2$

In this, change can be found after input is increased by 0.5 volts. This is better from above example, in which output was changing only when input is increased by 1.25 volts, here if input is changed by 0.5 volts, change in the output can easily be found. Thus, either increasing number of bits or decreasing V_{ref} , ADC can be improved.

Numbers of bits are set during manufacturing. For instance, an 8-bit ADC's bits can never change, only V_{ref} can be changed to get desired resolution.

Desired Resolution:

Resolution of ADC should be set according to the defined resolution of sensor interfaced; at least their resolution should match. If the Resolution of ADC is even better than sensor, it is good.

To interface LM35 (Temperature Sensor) having resolution of 10 mV/°C.

- For an 8-bit ADC with $V_{ref} = 5$ volts, the resolution of ADC will be 19.53 mV.

When temperature is 1°C the output of sensor will be 10 mV, and ADC will produce 0x00 between 0 mV to 19.53 mV. At 2°C, temperature sensor will produce 20mV and at 20 mV and ADC will produce 0x01. Hence, nearly every 2°C rise in temperature will cause ADC to increase one bit, which is not required as detection of accurate temperature is necessary, but change will appear after every 2°C change.

- If V_{ref} is changed to 2.56V then the Resolution of ADC will be 10 mV. Hence both will have same resolution.

At 1°C, sensor will produce 10mV, at 10mV input the ADC will produce 0x01 because of ADC's new resolution. But again, in case if temperature changes in fractions, the exact value cannot be measured. At 10.2°C LM35 will produce 10.2mV, and ADC will produce 0x0A at 100mV and 0x0B at 110mV.

To get even more accuracy, even better resolution of ADC is required.

- If resolution of ADC is set to 1mV, then

Every 1°C rise in temperature, sensor will produce 10 mV and at 10 mV, ADC will produce 0x0A (10), dividing output by 10 through programming, exact values can be achieved. At temperature 10.2°C, LM35 will produce 102 mV. Therefore at 102 mV, ADC will produce 102 in decimal, which will be divided by 10 through programming hence, an exact value of 10.2 has been achieved.

AVR Microcontrollers:

ATMEGA Series microcontrollers of AVR have multiple built-in ADCs of 10-bit with one V_r pin to set resolution. AVR microcontrollers also support two builtin Reference voltages of VCC and 2.56V, which can be set by setting value in an internal Registers of ADC. Look at the pin diagram of AVR ATMEGA16 given in figure 5, all ports are dual function. Port A can work as digital I/O port as well as Analog Input Port (ADC). It has 8 ADCs each of 10-bit. Pin 32 is used to set Analog Reference voltage.

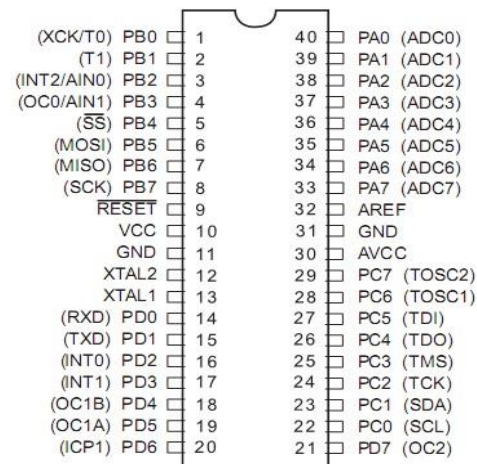


Figure 5: Pin diagram of ATMEGA 16

Internal Registers of AVR:

Registers of AVR microcontroller that are involved in configuration and control of ADC are:

- **ADCH** (High Data): ADC Data (MSB)
- **ADCL** (Low Data): ADC Data (LSB)
- **ADCSRA** (ADC Control and Status Register): To start and check status of ADC conversion.
- **ADMUX** (ADC Multiplexer Selection Register): To select ADC and Reference Voltage

ADMUX Register:

ADMUX Register is used to choose Reference voltage and to select an ADC from several ADCs available in AVR Microcontroller.

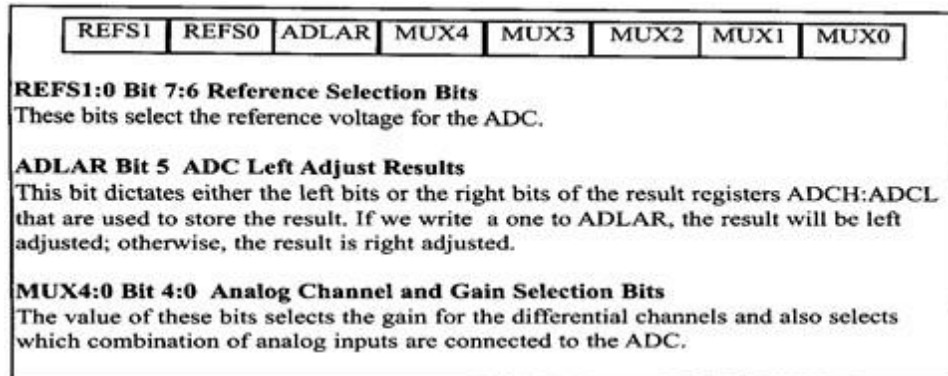


Figure 6: ADMUX Register bit description

Bit 7 & 6 (REFS1 & REFS0):

Table 1: V_{ref} source selection table for AVR

REFS1	REFS0	V_{ref}	
0	0	AREF pin	Set externally
0	1	AVCC pin	Same as VCC
1	0	Reserved	----
1	1	Internal 2.56 V	Fixed regardless of VCC value

Bit 5 (ADLAR):

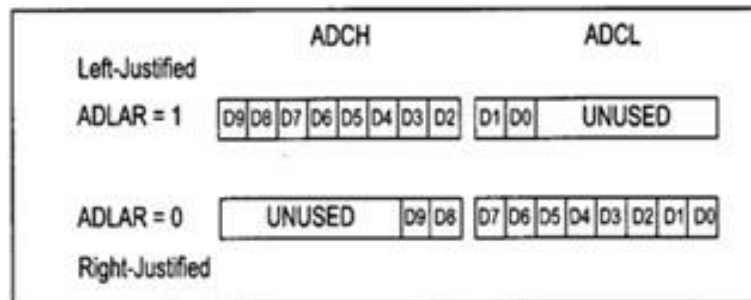


Figure 7: ADLAR Bit and ADCx Registers

Bit 4-0 (MUX4- MUX0)

Table 2: Single-ended Channels

MUX4...0	Single-ended Input
00000	ADC0
00001	ADC1
00010	ADC2
00011	ADC3
00100	ADC4
00101	ADC5
00110	ADC6
00111	ADC7

ADCSRA Register:

It is a Status and Control Register of ADC. Through this the status of ADC can be checked and controlled. Also, conversion can be started or monitored i.e., whether the conversion has been completed or not.

ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0
ADEN Bit 7 ADC Enable This bit enables or disables the ADC. Setting this bit to one will enable the ADC, and clearing this bit to zero will disable it even while a conversion is in progress.							
ADSC Bit 6 ADC Start Conversion To start each conversion you have to set this bit to one.							
ADATE Bit 5 ADC Auto Trigger Enable Auto triggering of the ADC is enabled when you set this bit to one.							
ADIF Bit 4 ADC Interrupt Flag This bit is set when an ADC conversion completes and the data registers are updated.							
ADIE Bit 3 ADC Interrupt Enable Setting this bit to one enables the ADC conversion complete interrupt.							
ADPS2:0 Bit 2:0 ADC Prescaler Select Bits These bits determine the division factor between the XTAL frequency and the input clock to the ADC.							

Figure 8: ADSCRA (A/D control and status register A)

ADPS 2-0:

Table 3: ADC Pre-scaler Select Bits

ADPS2	ADPS1	ADPS0	ADC Clock
0	0	0	Reserved
0	0	1	CK/2
0	1	0	CK/4
0	1	1	CK/8
1	0	0	CK/16
1	0	1	CK/32
1	1	0	CK/64
1	1	1	CK/128

ADC Operation:

Steps to work with ADCs:

1. Select ADC and set its Reference Voltage (ADMUX)
2. Enable ADC and select Pre-Scaler (ADCSRA)
3. Start ADC by placing one at ADSC (ADCSRA)
4. Wait until End of conversion and ADIF is set to one
5. Get Data from Registers

Programming:

The Basic programming flow will be like:

```

void main()
{
    Set ADMUX value;
    Set ADCSRA value;

    while(1)
    {

```

```

        Start ADC;
        wait here until ADIF is set to 1 (in ADCSRA)
        copy data from ADCL and ADCH
    }

    return 0;
}

```

Example Program: Write a program to perform Analog to digital conversion from ADC0 and keep Internal Reference voltage $V_r = 2.56$ volts.

Here, information to select ADC0 (ADMUX) is given along with Internal Reference value of 2.56 volts, other than that no information has been given. So, it is up to the user to select ADLAR, Prescaler, etc. Thus:

ADMUX:

REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0
1	1	0	0	0	0	0	0

`ADMUX = 0xC0;`

ADCSRA:

ADC conversion will start later in while loop. Hence for now, ADSC=0 and pre-Scalar has been selected to the last one.

ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0
1	0	0	0	0	1	1	1

`ADCSRA=0x87;`

Program:

```

#include "avr/io.h"

int main(void)
{
    DDRA = 0x00;    // Port A as Input
    DDRB = 0xFF;
    DDRD = 0xFF;
    ADMUX = 0xC0;
    ADCSRA = 0x87;

    while(1)
    {
        ADCSRA |= (1<<ADSC);
        while(ADCSRA & (1<<ADIF) == 0);
        PORTB=ADCL;
        PORTD=ADCH;
    }
}

```

```

return 0;
}

```

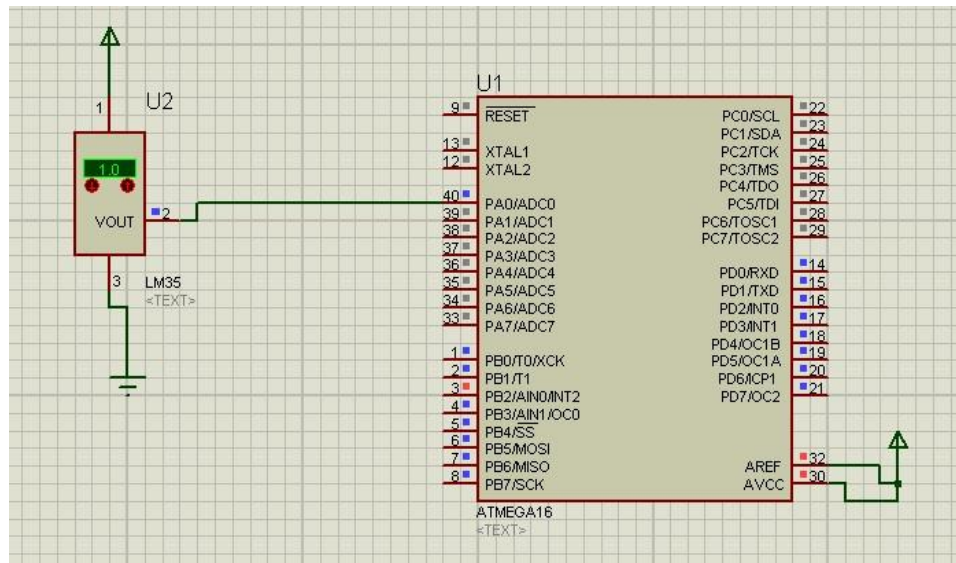
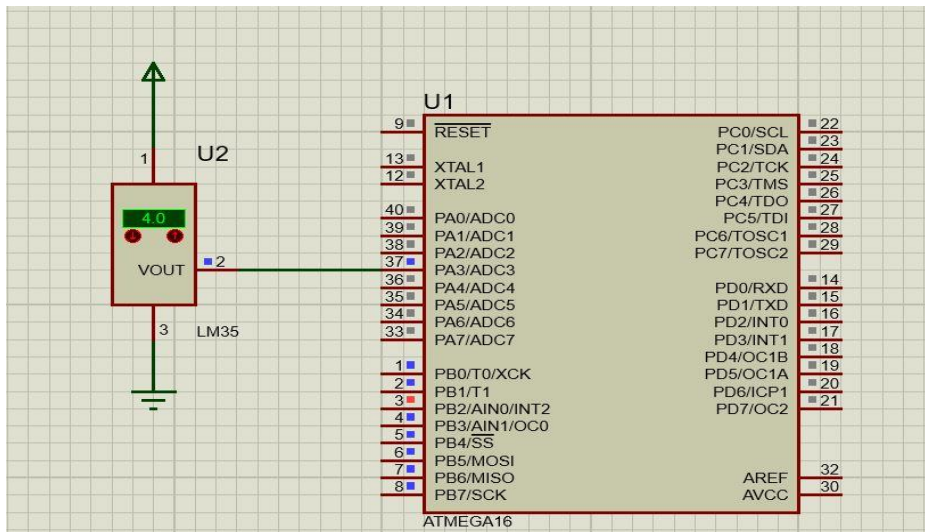


Figure 9: Proteus Output

Lab Tasks:

1. Imitate a source code to convert Analog value from ADC3 and show the higher 8-bits of converted data to Port B.
2. `#include <avr/io.h>`
- 3.
- 4.
5. `int main(void)`
6. {
7. `DDRA=0x00;`
8. `DDRB=0xFF;`
9. `ADMUX=0xE3;` ; 4 steps to reach 10mv
10. `ADCSRA=0x87;`
11. `while (1)`
12. {
13. `ADCSRA |=(1<<ADSC);`
14. `while(ADCSRA&(1<<ADIF)==0);`
15. `PORTB=ADCH;`
16. }
17. }

As reference is 2.5mv it takes 4 steps to change data. Hence at 4C data becomes 16, 10 in hex 0000010000

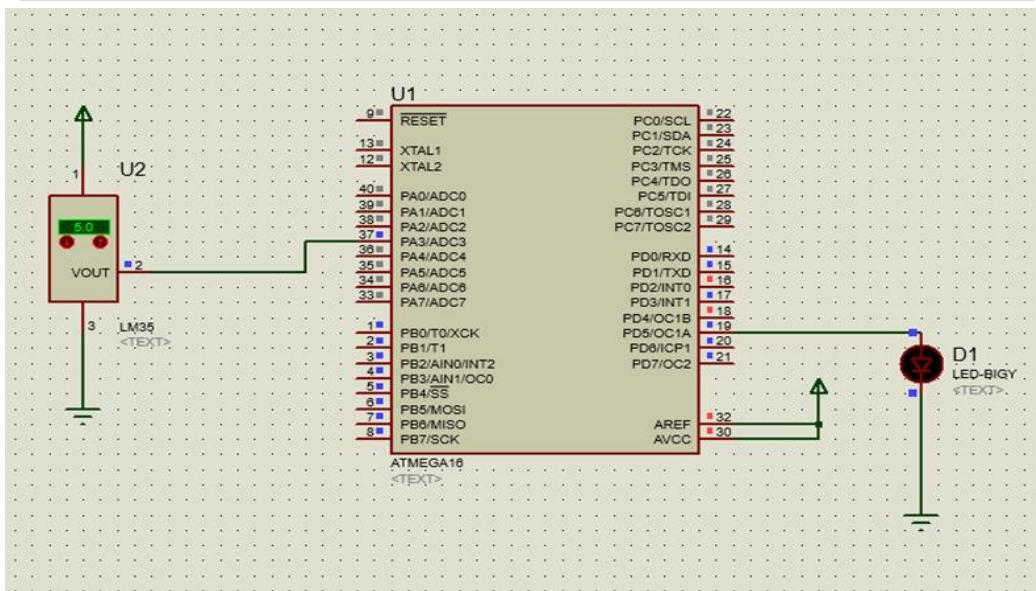
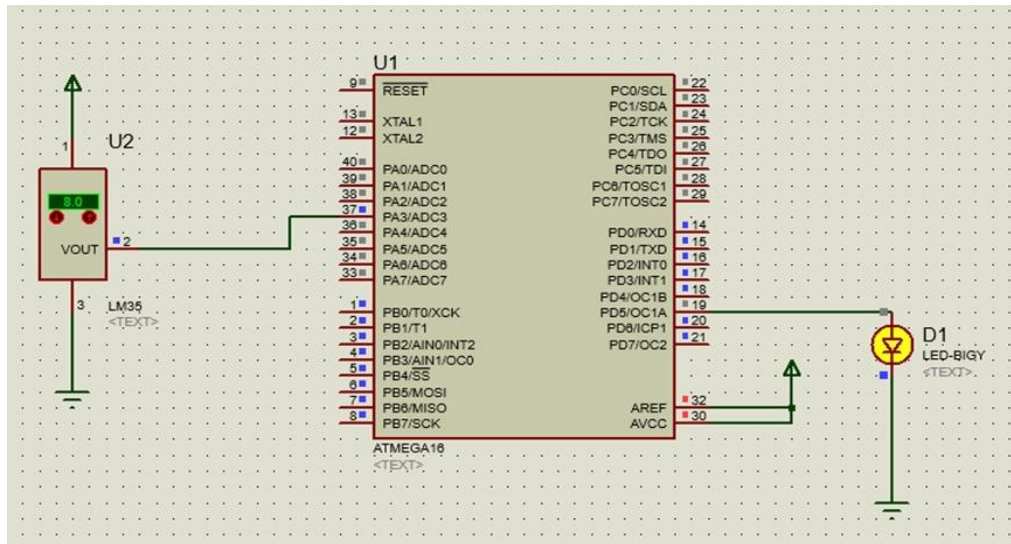


2: Imitate a source code to check if the converted data is greater than XX, then it should write Logic HIGH to Port D pin 6, else LOW.

```
#include "avr/io.h"
#define ADIF 4
#define ADSC 6
int main(void)
{
    DDRA = 0x00; // Port A as Input
    DDRB = 0xFF;
    DDRD = 0xFF;
    unsigned char Roll_No=0b01100010; // 62 in binary

    ADCSRA = 0x87;
    ADMUX = 0xC0;
    while(1)
    {
        ADCSRA |= (1<<ADSC);
        while(ADCSRA & (1<<ADIF) == 0);
        PORTB=ADCL;
        if (PORTB>Roll_No)
        {
            PORTD |= (1<<6);
        }
        else
        {
            PORTD &= ~(1<<6);
        }
    }

    return 0;
}
```



18. By considering figure 10, imitate a source code for a system, that continuously check temperature, if temperature is greater than XX, it should turn the fan ON & Heater OFF and if it is less than XX, it should turn ON Heater and turn OFF Fan. where XX represents your Roll number (e.g., 19ESXX).

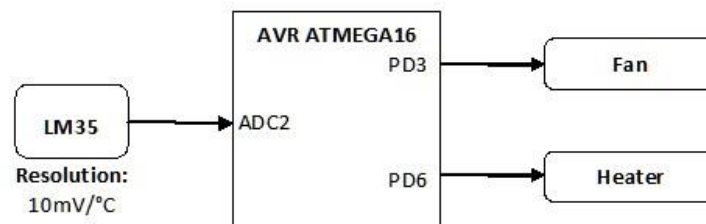


Figure 10: Controlling Heat of the environment based on the temperature of the environment

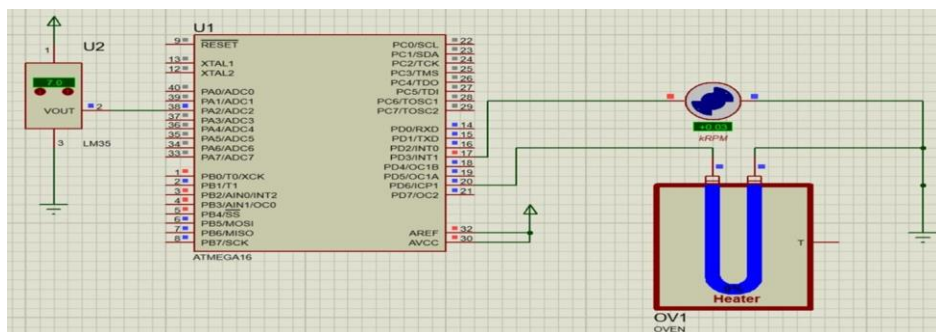
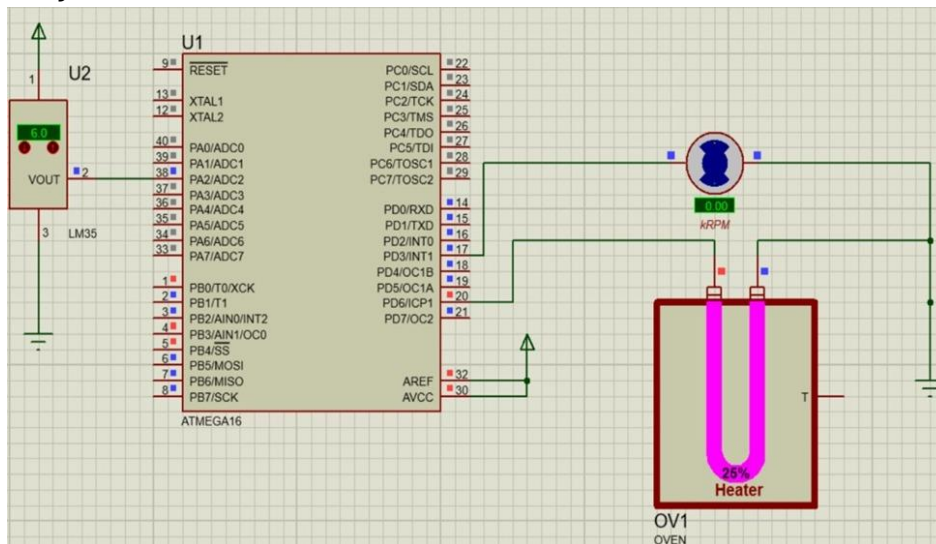
`#include <avr/io.h>`

```

#define fan 3
#define heater 6
int main(void)
{
    DDRA=0x00;
    DDRB=0xFF;
    DDRD=(1<<fan)|(1<<heater);
    ADMUX=0xC2;
    ADCSRA=0x87;
    while (1)
    {
        ADCSRA|=(1<<ADSC);
        while(!(ADCSRA & (1<<ADIF))==0);
        PORTB=ADC;

        if (PORTB>0x24) //my roll no 36 in hex
            PORTD=(1<<fan);
        else
            PORTD=(1<<heater);
    }
}

```

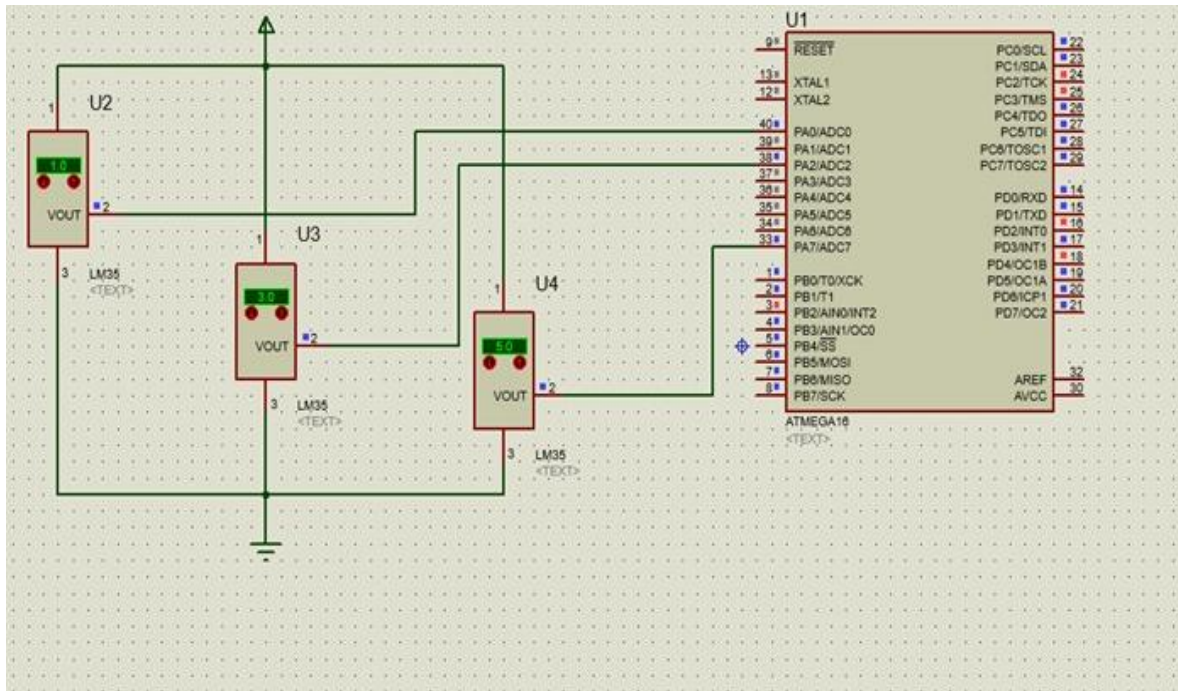


4: Imitate a source code that read data from 3 ADCs (ADC0, ADC2 & ADC7), and display the lower 8-bit data to Port B, Port C and Port D, respectively.

Note: Attach the program files C Programing code and simulation result snapshots along with the handouts.

```
#include"avr/io.h"
#include <util/delay.h>
#define ADIF 4
#define ADSC 6
int main(void)
{
    DDRB=0xFF;
    DDRC=0xFF;
    DDRD=0xFF;
    DDRA=0x00;
    ADCSRA=0x87;
    unsigned char A,B,C;
    while(1)
    {
        // ADC 5
        ADMUX=0xC0;
        ADCSRA|=(1<<ADSC);
        while(ADCSRA&(1<<ADIF)==0);
        A=ADCL;
        PORTB=A;
        // ADC 6
        ADMUX=0xC2;
        ADCSRA|=(1<<ADSC);
        while(ADCSRA&(1<<ADIF)==0);

        B=ADCL;
        PORTC=B;
        // ADC 7
        ADMUX=0xC7;
        ADCSRA=(1<<ADSC);
        while(ADCSRA&(1<<ADIF)==0);
        _delay_ms(500);
        C=ADCL;
        PORTD=C;
    }
    return 0;
}
```





MEHRAN UNIVERSITY OF ENGINEERING AND TECHNOLOGY, JAMSHORO
DEPARTMENT OF ELECTRONIC ENGINEERING



INTRODUCTION TO EMBEDDED SYSTEMS (ES-314)

Batch: 20ES (5th Semester)

Lab Experiment #10

To Learn Interfacing of MAX7221 (8-Digit, LED Display Drivers) and its AVR
Programming Using SPI Serial Protocol

Name <u>KARAN</u>	Roll # <u>20ES062</u>
Signature of Lab Tutor _____	Date _____

RUBRICS:

Performance Metric	CONDUCTING EXPERIMENT	PARTICIPATION	TEAMWORK	USE OF MODERN ENGINEERING TOOLS	DATA ANALYSIS	CALCULATION AND CODING	OBSERVATION/ PROGRAM RESULTS	Total Score
	0.5	0.5	0.5	0.5	1	1	1	05
Obtained								

OBJECTIVE(S)

#	Topic	# Of Lectures	CLO	Taxonomy level
1	The purpose of this lab is to approach different features of Serial Peripheral Interface (SPI) Protocol and MAX7221 IC.	3	4,5	P3, A2
2	Imitate and execute the programs that can be interfaced with MAX7221 IC using AVR Microcontrollers via SPI protocol.			

OUTCOME(S)

a. An ability to use the techniques, skills, and modern engineering tools necessary for engineering practice.	PLO5: Modern Tool Usage
b. An ability to function on multi-disciplinary teams	PLO9: Individual and Teamwork

REQUIRED TOOLS AND SOFTWARES:

- Proteus Design Suite
- AVR Studio
- WinAVR

INTRODUCTION:

There are different protocols for serial communication between two devices like, USART, SPI, I2C etc. Before selecting any communication protocol, data transfer rate is an important parameter. SPI, I2C, and UART are quite a bit slower than protocols like USB, ethernet, Bluetooth,

and Wi-Fi, but they are a lot simpler and use less hardware and system resources. SPI, I2C, and UART are ideal for communication between microcontrollers and between microcontrollers and sensors where large amounts of high-speed data do not need to be transferred.

One unique benefit of SPI is the fact that data can be transferred without interruption. Any number of bits can be sent or received in a continuous stream. With I2C and UART, data is sent in packets, limited to a specific number of bits. Start and stop conditions define the beginning and end of each packet, so the data is interrupted during transmission.

SPI is a common communication protocol used by many different devices. For example, SD card modules, RFID card reader modules, and 2.4 GHz wireless transmitter/receivers all use SPI to communicate with microcontrollers.

Serial Peripheral Interface (SPI):

Serial Peripheral Interface is a synchronous, full-duplex protocol. SPI is also known as “3-wire interface” protocol because it needs 3 communication lines named MISO, MOSI and SCK. SPI protocol needs two devices for communication. One of them is considered as a MASTER and another one as a SLAVE. AVR microcontrollers contain both MASTER and SLAVE interface on single chip. Thus, a microcontroller can work as both master and slave device.

Devices communicating via SPI are in a master-slave relationship. The master is the controlling device (usually a microcontroller), while the slave (usually a sensor, display, or memory chip) takes instruction from the master. The simplest configuration of SPI is a single master, single slave system, but one master can control more than one slave.

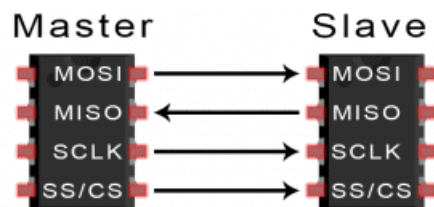


Figure 1: Master Slave Connection

MOSI (Master Output/Slave Input) – Line for the master to send data to the slave.

MISO (Master Input/Slave Output) – Line for the slave to send data to the master.

SCLK (Clock) – Line for the clock signal.

SS/CS (Slave Select/Chip Select) – Line for the master to select which slave to send data to.

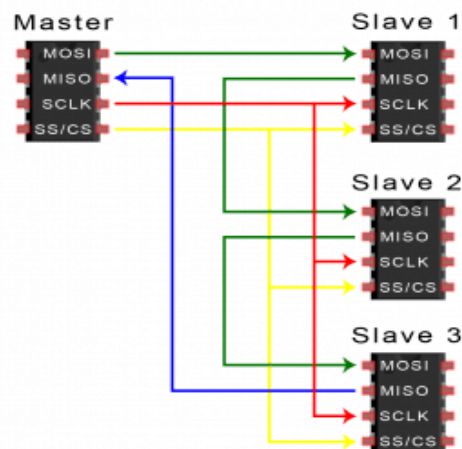


Figure 2: Multiple Slaves

MAX7221 IC:

In many applications two or more 7-segment LEDs are needed to connect to a microcontroller. For example, if it is required to connect four 7-segment LEDs directly to a microcontroller i.e., $4 \times 8 = 32$ pins which is not feasible. The MAX 7221 IC is an ideal chip for such applications as it supports up to 8 7-segment LEDs. MAX 7221 can be connected to the AVR chip by using SPI protocol and control up to eight 7-segment LEDs. The MAX7221 contains an internal decoder that can be used to convert binary numbers to 7-segment codes. The device includes analog and digital brightness control, an 8x8 static RAM that stores each digit, and a test mode that forces all LEDs On.

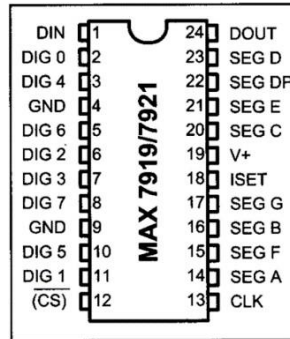


Figure 3: MAX7221 IC Pin Configuration

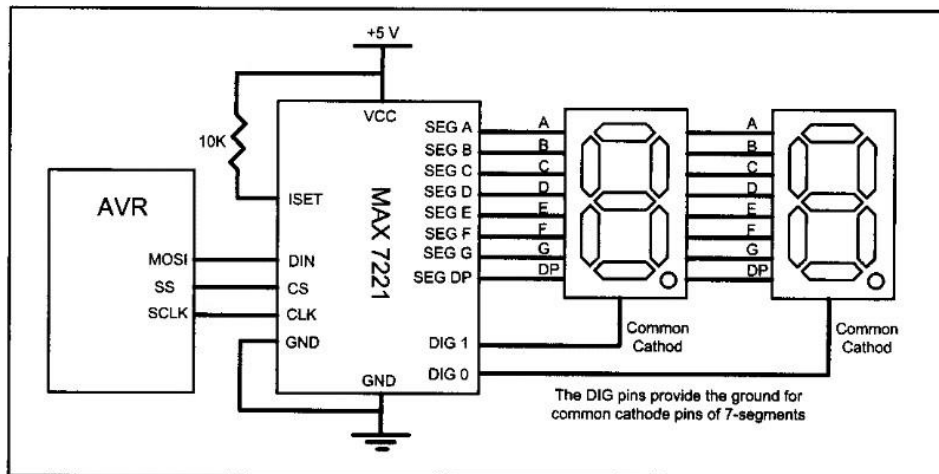


Figure 4: MAX7221 Connections to the AVR

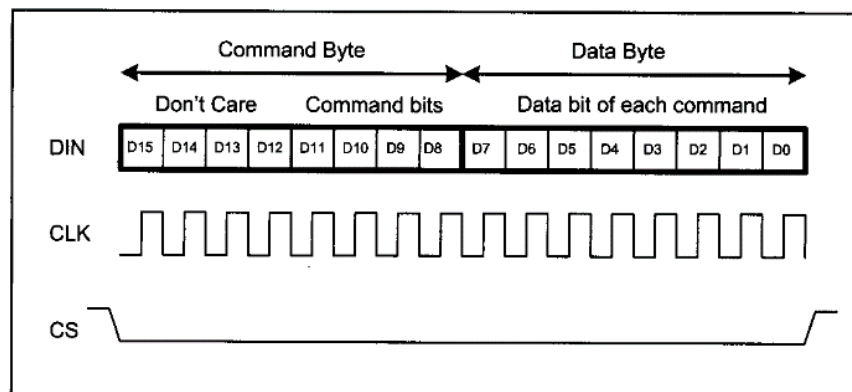


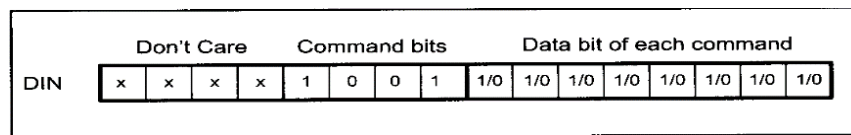
Figure 5: MAX7221 Packet Format:

Table 1: List of Commands in MAX7221

Command	D15-12	D11	D10	D9	D8	Hex Code
No operation	X	0	0	0	0	X0
Set value of digit 0	X	0	0	0	1	X1
Set value of digit 1	X	0	0	1	0	X2
Set value of digit 2	X	0	0	1	1	X3
Set value of digit 3	X	0	1	0	0	X4
Set value of digit 4	X	0	1	0	1	X5
Set value of digit 5	X	0	1	1	0	X6
Set value of digit 6	X	0	1	1	1	X7
Set value of digit 7	X	1	0	0	0	X8
Set decoding mode	X	1	0	0	1	X9
Set intensity of light	X	1	0	1	0	XA
Set scan limit	X	1	0	1	1	XB
Turn on/ off	X	1	1	0	0	XC
Display test	X	1	1	1	1	XF

Notes: 1) X means do not care.

2) Digits are designated as 0–7 to drive total of eight 7-segment LEDs.

**Figure 6:** Set Decoding Mode (Command X9)**Internal registers of AVR:**

AVR ATmega16 uses three registers to configure SPI communication that are SPI Control Register, SPI Status Register and SPI Data Register.

SPCR: SPI Control Register:

7	6	5	4	3	2	1	0	
SPIE	SPE	DORD	MSTR	CPOL	CPHA	SPR1	SPR0	SPCR

Bit 7 – SPIE: SPI Interrupt Enable bit

1 = Enable SPI interrupt.

0 = Disable SPI interrupt.

Bit 6 – SPE: SPI Enable bit

1 = Enable SPI.

0 = Disable SPI.

Bit 5 – DORD: Data Order bit

1 = LSB transmitted first.

0 = MSB transmitted first.

Bit 4 – MSTR: Master/Slave Select bit

1 = Master mode.

0 = Slave Mode.

Bit 3 – CPOL: Clock Polarity Select bit

1 = Clock start from logical one.

0 = Clock start from logical zero.

Bit 2 – CPHA: Clock Phase Select bit

1 = Data sample on trailing clock edge.

0 = Data sample on the leading clock edge.

Bit 1:0 – SPR1: SPR0 SPI Clock Rate Select bits

The below table shows the SCK clock frequency select bit setting.

Table 2: bit setting for SCK clock frequency

SPI2X	SPR1	SPR0	SCK Frequency
0	0	0	Fosc/4
0	0	1	Fosc/16
0	1	0	Fosc/64
0	1	1	Fosc/128
1	0	0	Fosc/2
1	0	1	Fosc/8
1	1	0	Fosc/32
1	1	1	Fosc/64

SPSR: SPI Status Register

7	6	5	4	3	2	1	0	
SPIF	WCOL						SPI2X	SPSR

Bit 7 – SPIF: SPI interrupt flag bit

- This flag gets set when the serial transfer is complete.
- Also gets set when the SS pin is driven low in master mode.
- It can generate an interrupt when SPIE bit in SPCR and a global interrupt is enabled.

Bit 6 – WCOL: Write Collision Flag bit

- This bit gets set when SPI data register writes occur during previous data transfer.

Bit 5:1 – Reserved Bits

Bit 0 – SPI2X: Double SPI Speed bit

- When set, SPI speed (SCK Frequency) gets doubled.

SPDR: SPI Data Register

7	6	5	4	3	2	1	0	
								SPDR

- SPI Data register used to transfer data between the Register file and SPI Shift Register.
- Writing to the SPDR initiates data transmission.

Program: Write an AVR program to display 57 on 7-segment.

```
#include <avr/io.h>
# define MOSI 5
# define SCK 7
# define SS 4
void execute( unsigned char cmd, unsigned char data)
{
```



```

PORTB &= ~(1<<SS);
SPDR=cmd;
while(!(SPSR & (1<<SPIF)));
SPDR= data;
while(!(SPSR & (1<<SPIF)));
PORTB |= (1<<SS);
}
int main(void)
{
DDRB= (1<<MOSI)|(1<<SCK)|(1<<SS);
SPCR=(1<<SPE)|(1<<MSTR)|(1<<SPR0);
execute(0x09,0b00000011); // enable decoding for digits 1,2
execute(0x0B,0x02);       // scan two 7 segments
execute(0x0C,0x01);       // turn on the chip
execute(0x01,0x07);       // display 7 on digit 1
execute(0x02,0x05);       // display 5 on digit 2
while (1);
return 0;
}

```

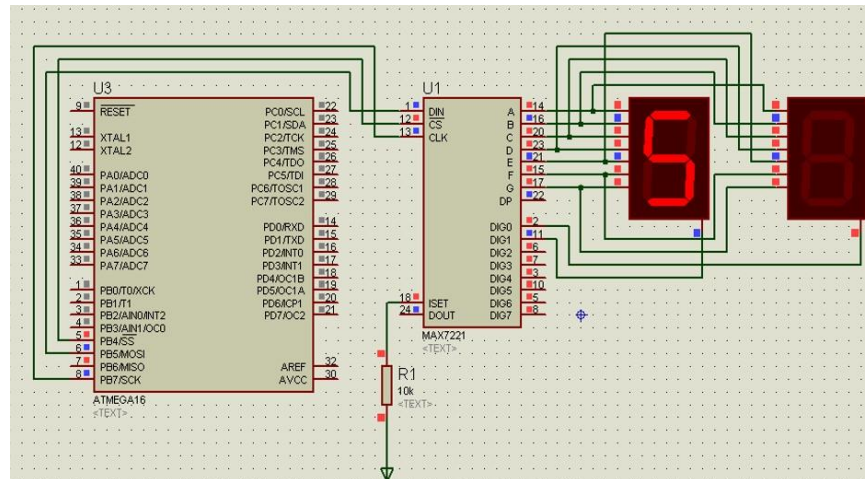


Figure 7: Proteus Output of digit 2

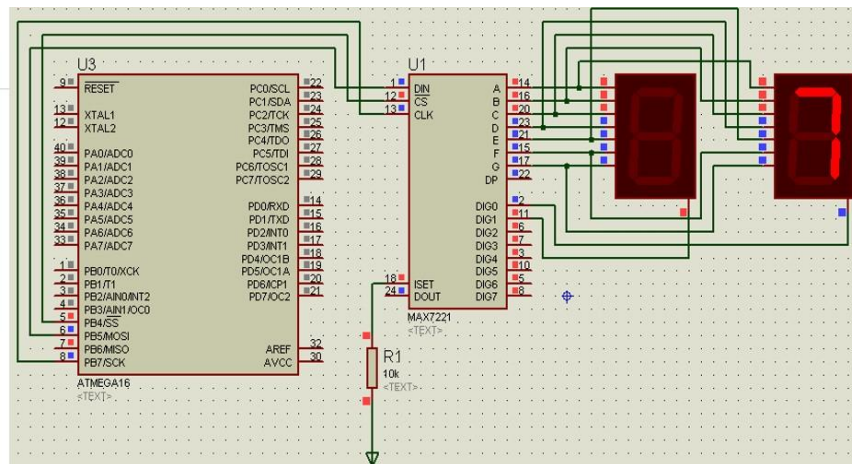


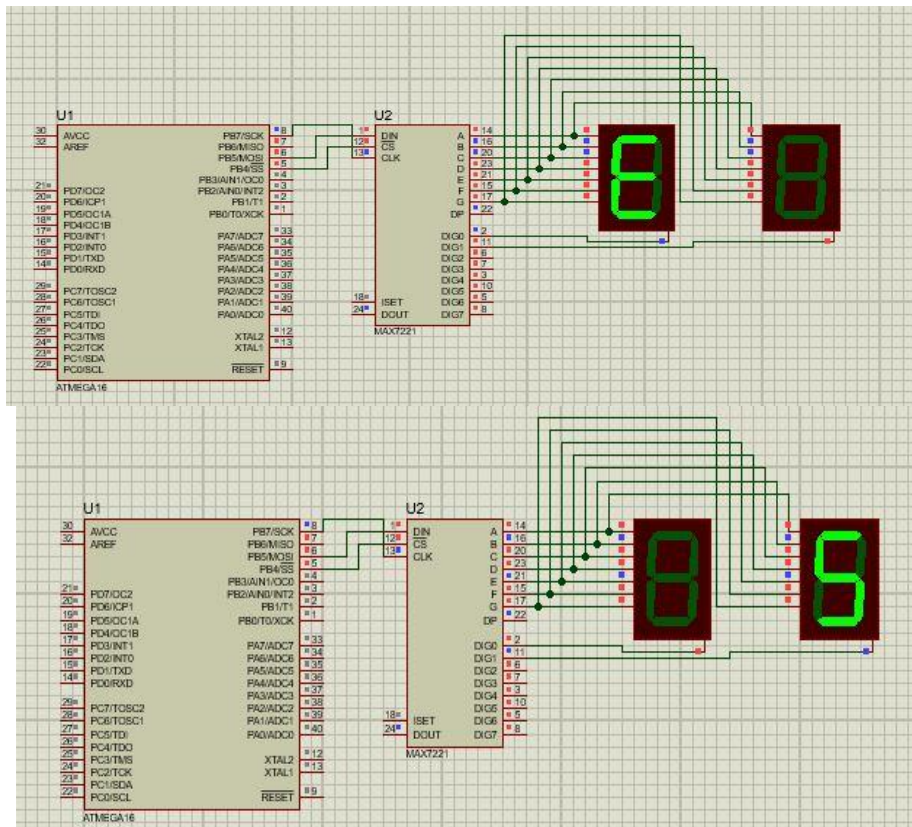
Figure 8: Proteus Output of digit 1

Lab Tasks:

1. Imitate a source code to display ES on the two digits of the 7-segment display.

```
#include <avr/io.h>
#include <util/delay.h>
#define MOSI 5
#define SCK 7
#define SS 4
void execute(unsigned char cmd,unsigned char data)
{
    PORTB &=~(1<<SS);
    SPDR=cmd;
    while(!(SPSR &(1<<SPIF)));
    SPDR=data;
    while(!(SPSR &(1<<SPIF)));
    PORTB |= (1<<SS);
}
int main(void)
{
    DDRB=(1<<MOSI)|(1<<SCK)|(1<<SS);
    SPCR=(1<<SPE)|(1<<MSTR)|(1<<SPR0);
    execute(0x09,0b00000000);//enable decoding for digit 1,2
    execute(0x0B,0x02);//Scan two 7 segment
    execute(0x0C,0x01);//turn on the chip
    execute(0x01,0x4F);//Display E on digit 1
    execute(0x02,0x5B);//Display S on digit 2
    while(1);
    return 0;
}
```

Simulation:

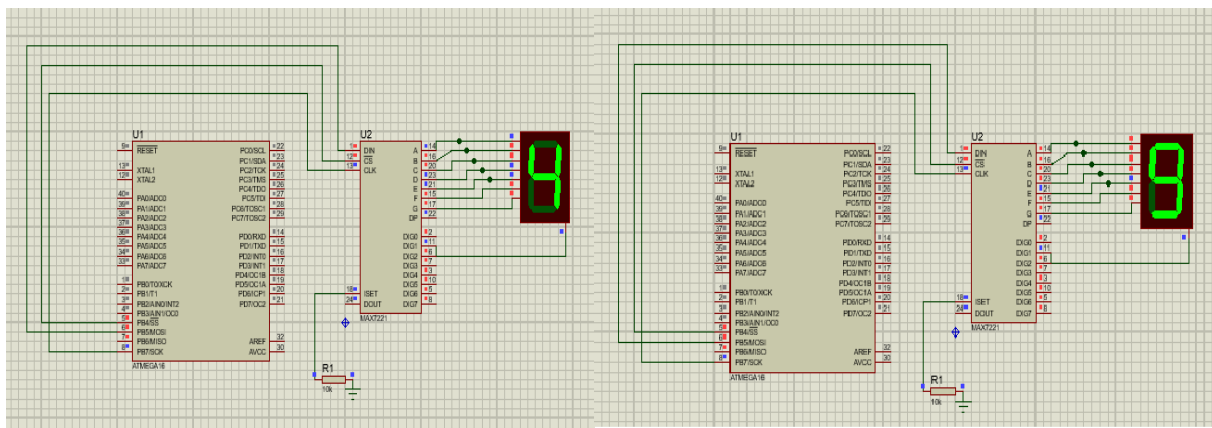


2. Imitate a source code which Counts BCD numbers.

Code:

```
#include <avr/io.h>
#include<util/delay.h>
#define MOSI 5
#define SCK 7
#define SS 4
void execute( unsigned char cmd, unsigned char data)
{
    PORTB &= ~(1<<SS);
    SPDR=cmd;
    while(!(SPSR & (1<<SPIF)));
    SPDR= data;
    while(!(SPSR & (1<<SPIF)));
    PORTB |= (1<<SS);
}
int main(void)
{
    DDRB= (1<<MOSI)|(1<<SCK)|(1<<SS);
    SPCR=(1<<SPE)|(1<<MSTR)|(1<<SPR0);
    execute(0x09,0b0000001); // enable decoding for digits 1,2
    execute(0x0B,0x02); // scan the 7 segments
    execute(0x0C,0x01); // turn on the chip
    while(1)
    {
        int i=0;
        for(i=0;i<=9;i++)
        {
            _delay_ms(500);
            execute(0x01,i);
            _delay_ms(500);
        }
    }
    while (1);
    return 0;
}
```

Simulation:

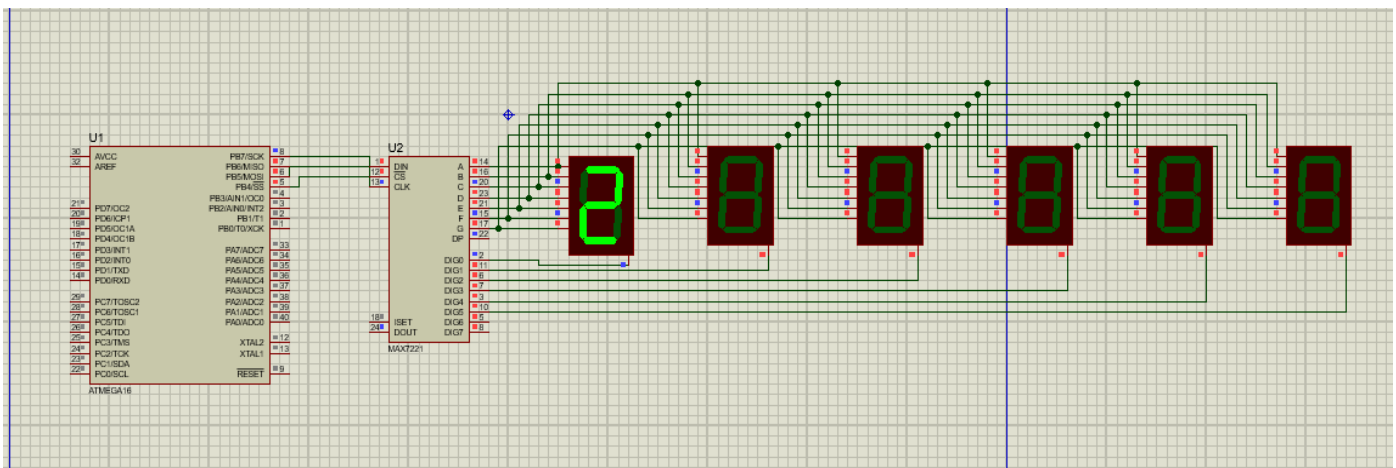


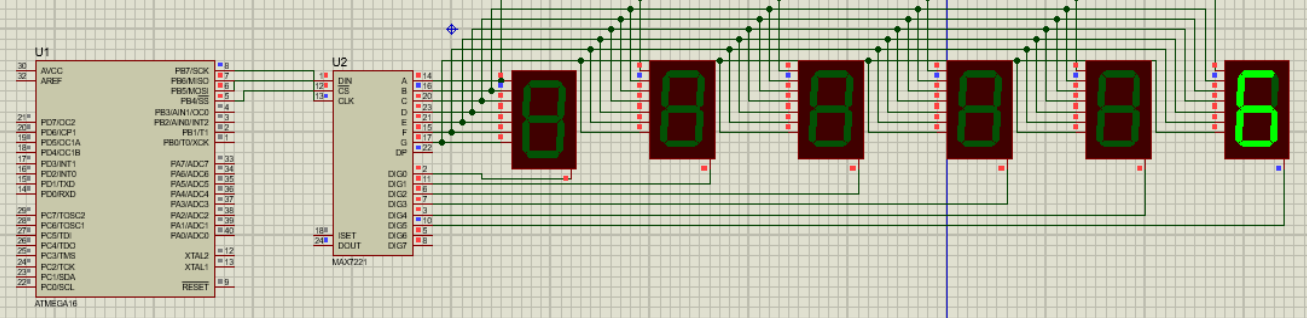
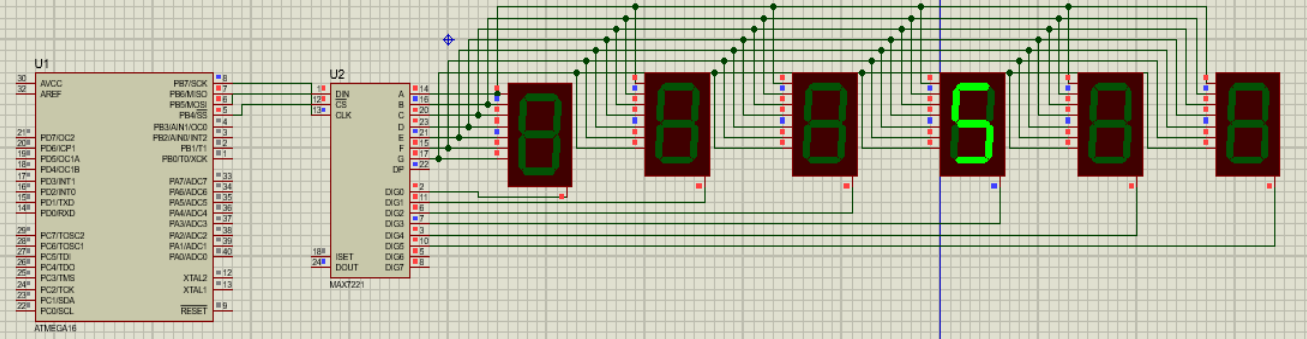
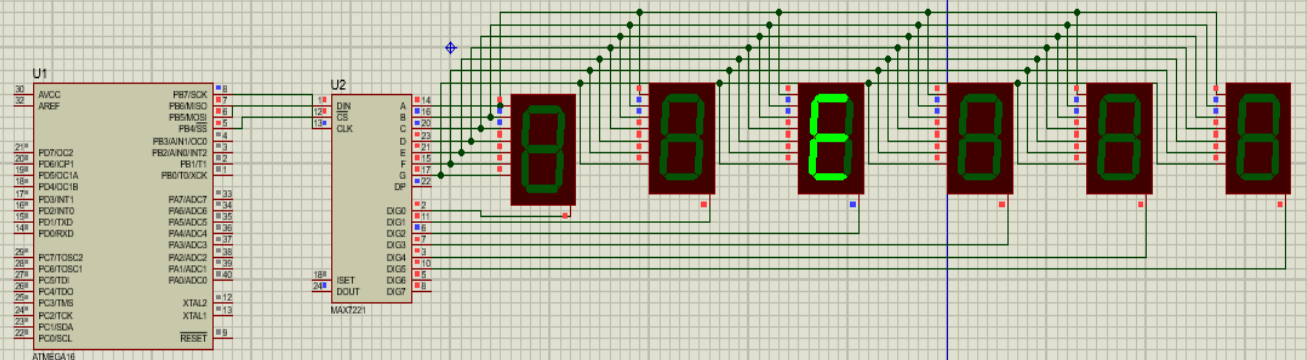
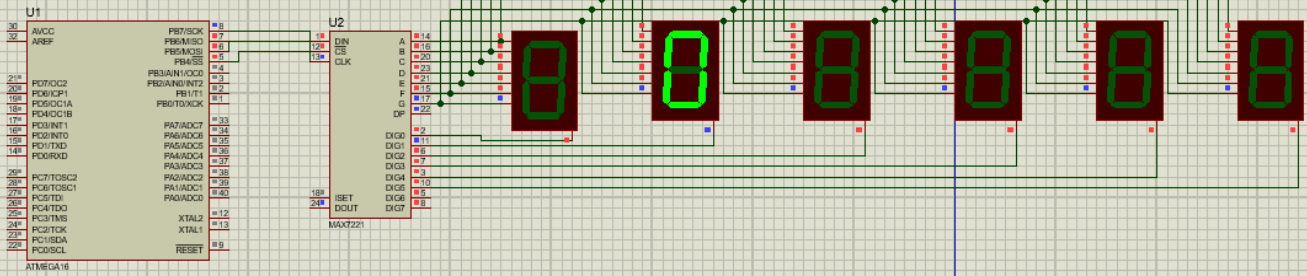
3. Imitate a source code to display your complete Roll No. on the 7-segment display.

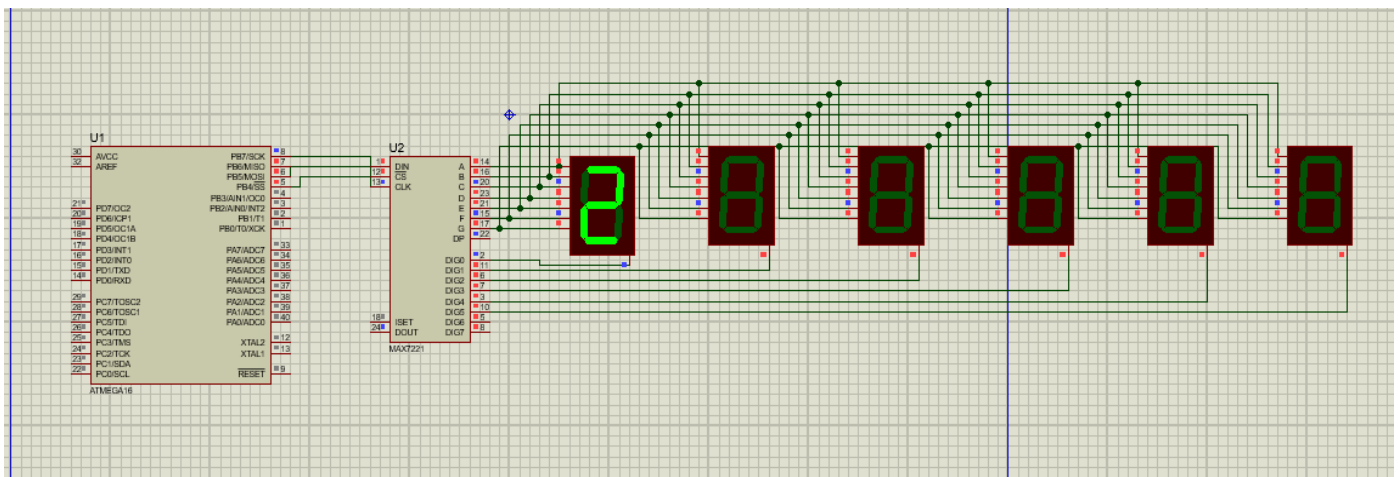
Code:

```
#include <avr/io.h>
#include <util/delay.h>
#define F_CPU 8000000
#define MOSI 5
#define SCK 7
#define SS 4
void execute(unsigned char cmd,unsigned char data)
{
    PORTB &=~(1<<SS);
    SPDR=cmd;
    while(!(SPSR &(1<<SPIF)));
    SPDR=data;
    while(!(SPSR &(1<<SPIF)));
    PORTB |= (1<<SS);
}
int main(void)
{
    DDRB=(1<<MOSI)|(1<<SCK)|(1<<SS);
    SPCR=(1<<SPE)|(1<<MSTR)|(1<<SPR0);
    execute(0x0B,0x07);//Scan seven 7 segment
    _delay_ms(200);
    execute(0x0C,0x01);//turn on the chip
    _delay_ms(200);
    execute(0x01,0x6D);//Display 2 on digit 1
    _delay_ms(200);
    execute(0x02,0x7E);//Display 0 on digit 2
    _delay_ms(200);
    execute(0x03,0x4F); //Display E on digit 1
    _delay_ms(200);
    execute(0x04,0x5B);//Display S on digit 1
    _delay_ms(200);
    execute(0x05,0x5F);//Display 6 on digit 2
    _delay_ms(200);
    execute(0x06,0x6D);//Display 2 on digit 2
    while(1);
    return 0;
}
```

Simulation:







Note: Attach the program files C Programming code and simulation result snapshots along with the handouts.



MEHRAN UNIVERSITY OF ENGINEERING AND TECHNOLOGY, JAMSHORO
DEPARTMENT OF ELECTRONIC ENGINEERING



INTRODUCTION TO EMBEDDED SYSTEMS (ES-314)

Batch: 20ES (5th Semester)

Lab Experiment #11

To Analyze Interfacing of PCF 8574 to ATmega16 using I2C communication

Name <u>KARAN</u>	Roll # <u>20ES062</u>
Signature of Lab Tutor _____	Date _____

RUBRICS:

Performance Metric	CONDUCTING EXPERIMENT	PARTICIPATION	TEAMWORK	USE OF MODERN ENGINEERING TOOLS	DATA ANALYSIS	CALCULATION AND CODING	OBSERVATION/ PROGRAM RESULTS	Total Score
	0.5	0.5	0.5	0.5	1	1	1	05
Obtained								

OBJECTIVE(S)

#	Topic	# Of Lectures	CLO	Taxonomy level
1	The purpose of this lab is to determine the different aspects of I2C protocol communication and determine the features of PCF 8574 IO Expander IC.	3	4,5	P3, A2
2	Imitate and execute read and write programs by interfacing PCF8574 to ATmega16 using I2C communication.			

OUTCOME(S)

a. An ability to use the techniques, skills, and modern engineering tools necessary for engineering practice.	PLO5: Modern Tool Usage
b. An ability to function on multi-disciplinary teams	PLO9: Individual and Teamwork

REQUIRED TOOLS AND SOFTWARES:

- Proteus Design Suite
- AVR Studio
- WinAVR

INTRODUCTION:

A typical embedded system consists of one or more microcontrollers and peripheral devices like memories, converters, I/O expanders, LCD drivers, sensors, matrix switches, etc. The complexity and the cost of connecting all those devices together must be kept to a minimum level. The system must be designed in such a way that slower devices can communicate with the system without slowing down faster ones. To satisfy these requirements a serial bus is needed. A bus means specification for the connections, protocol, formats, addresses and procedures that define the rules on the bus. This is exactly what I2C bus specifications define.

I2C - Inter Integrated Circuit:

The I2C bus uses two wires: serial data (SDA) and serial clock (SCL). All I2C master and slave devices relate to only those two wires. Each device can be a transmitter, a receiver or both. Some devices are masters – they generate bus clock and initiate communication on the bus, other devices are slaves and respond to the commands on the bus. To communicate with specific device, each slave device must have an address which is unique on the bus. I2C master devices (usually microcontrollers) do not need an address since no other (slave) device sends commands to the master.

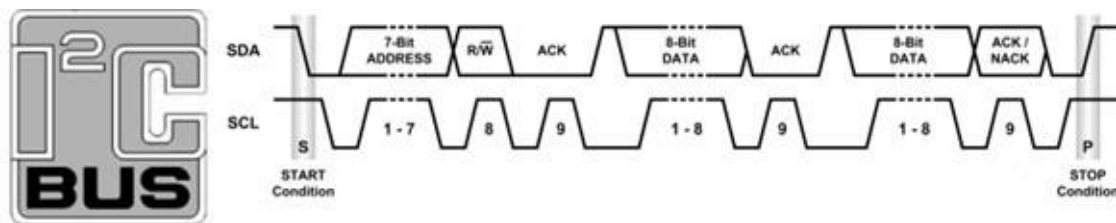


Figure 1: Writing multiple bytes to same slave

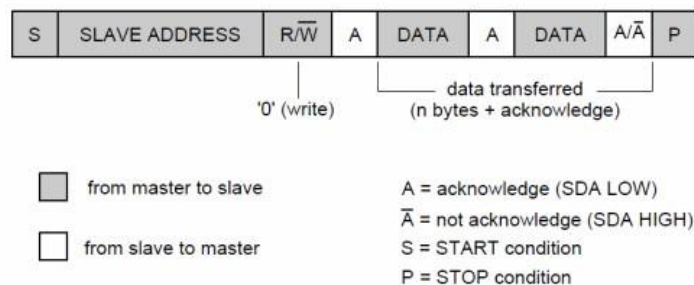


Figure 2: Data Packet Description

To work with I2C there are following Condition:

1. Start
2. Stop
3. Write
4. Read

Following are Step to initiate I2C communication.

1. Initial Condition:

- a. Set Baud Rate for Data Transfer in TWBR
- b. Set the TWEN bit in TWCR reg, for enabling TWI communication.
- c. Set Pre-scaler in TWSR.

• TWBR: Two Wire Baud Rate

Bit	7	6	5	4	3	2	1	0
-----	---	---	---	---	---	---	---	---

Name	TWBR 7	TWBR 6	TWBR 5	TWBR 4	TWBR 3	TWBR 2	TWBR 1	TWBR 0
Value								

TWBR Value is calculated by using this equation:

$$\frac{SCL}{CLOCK} = \frac{CPU\ CLOCK\ FREQUENCY}{16 + 2 * (TWBR \cdot TWPS)} * 4$$

In equation TWPSR value is set by the two bits in that reg.

TWPS 1	TWPS 0	Pre-scaler
0	0	1
0	1	4
1	0	16
1	1	64

In this case, assume SCL as 10 kHz, F_CPU as 8MHz and setting Pre-scaler as 1. By Calculating we get TWBR Value as 98 in decimal we will enter in Hex as 62

• **TWCR: Two Wire Control Register**

Bit	7	6	5	4	3	2	1	0
Name	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE
Value						1		

• **TWSR: Two Wire Status Register**

Bit	7	6	5	4	3	2	1	0
Name	TWS7	TWS6	TWS5	TWS4	TWS3	-	TWPS1	TWPS0
Value								

In this case it is TWPS1 = 0 and TWPS0 = 0.

You should set TWPS1 & TWPS0 for pre-scaler.

Rest of bit as the status bit which indicates the status of I2C communication.

2. Start Condition: To Initiate start condition.

- Clear TWINT by setting it 1.
- Set TWSTA Bit in TWCR reg as 1.
- Set TWEN Bit.
- Check for TWINT bit set in TWCR reg.

TWCR

Bit 7	7	6	5	4	3	2	1	0
Name	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE
Value	1		1			1		

Syntax: while (!(TWCR & (1<<TWINT)));

3. STOP CONDITION: To Stop the I2c

- Clear TWINT bit by setting it 1.
- Set TWEN bit as 1.
- Set TWSTO bit as 1.

TWCR

Bit 7	7	6	5	4	3	2	1	0
-------	---	---	---	---	---	---	---	---

Name	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE
Value	1			1		1		

4. Write Condition: To write to slave.

- Move the data to be transmitted to slave in TWDR (I2C Data Register).
- Clear TWINT by setting bit as 1 in TWCR and TWEN is set.
- Check for TWINT bit set in TWCR reg.

TWCR

Bit 7	7	6	5	4	3	2	1	0
Name	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE
Value	1					1		

Syntax: while (! (TWCR & (1<<TWINT)));

Control word is form by:

D7	D6	D5	D4	D3	D2	D1	D0
0	1	0	0	A2	A1	A0	0

A2-A0 is used to different device we can connect around 7 devices like this.

D0 = 0 means **write** to IO Pin Register.
D0 = 1 means **read** from IO Pin Register.

In our case its
We are using as output. So writing to IO Expander

D7	D6	D5	D4	D3	D2	D1	D0
0	1	0	0	0	0	0	0

Control Word to write is 0x40

We will now program it's as input pin for Atmega16 and move data in IO pin into PORTA

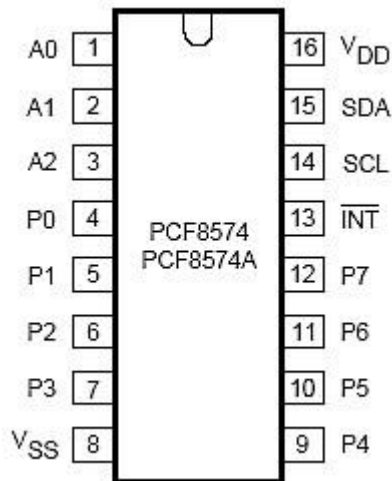
We must give start condition to get each 8 bit data so we should get start in while loop

D7	D6	D5	D4	D3	D2	D1	D0(R/W)
0	1	0	0	0	0	0	0

Control words to read is 0x41

5. Read Condition: To read data from slave

- Clear TWINT bit as 1 in TWCR and TWEN is set.
- Check for TWINT bit set in TWCR reg.
- Return TWDR using return function.



TWCR

Bit 7	7	6	5	4	3	2	1	0
Name	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE
Value	1					1		

Syntax: while (! (TWCR & (1<<TWINT))); Return

Value in TWDR.

Figure 3: (a) PCF8574IO
Expander IC Pin description

(b) Control word

Write an AVR program to write data to IO Expander IC PCF 8574.

```
#include <avr/io.h>          //--- Reg file of Atmega16
#include <util/delay.h>       //--- Delay Header

void i2c_init() {

    TWBR = 0x62;              //--- Baud rate
    TWCR = (1<<TWEN);         //--- Enable I2C
    TWSR = 0x00;              //--- Prescaler set to 1
}
//--- I2C Start Condition ---// void
i2c_start()
{
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWSTA);
    //--- Start Conditon
    while (!(TWCR & (1<<TWINT)));
}
//--- I2C Stop Condition ---// void
i2c_stop()
{
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);
}
void i2c_write(char x)
{
    TWDR = x; //--- Move value to I2C reg TWCR
    TWCR = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR & (1<<TWINT)));
}
//--- I2C Read Condition ---// char
i2c_read()
{
    TWCR = (1<<TWEN) | (1<<TWINT);
    while (!(TWCR & (1<<TWINT)));
    return TWDR;
}

int main(void)
{
    DDRA = 0xFF; //--- PORTA as OUTPUT

    i2c_init();          //--- Initiate I2C communication
    i2c_start();          //--- Start Condition to Slave device
```

The screenshot displays the I2C Debug - \$I2C DEBUGGER#0005 window. The main area shows a list of I2C transactions, including addresses, data, and status. The transactions are as follows:

Time	Address	Data	Status
17.625us	3.883	S 40 A AA A 55 A AA A 55 A A	Start
3.883	7.891	S 55 A AA A 55 A AA A 55 A A	Start
3.883	Bit 6	SDA=1 (WHI)	Write
3.883	Bit 5	SDA=0 (SLO)	Write
3.883	Bit 4	SDA=1 (WHI)	Write
3.883	Bit 3	SDA=0 (SLO)	Write
3.883	Bit 2	SDA=1 (WHI)	Write
3.883	Bit 1	SDA=0 (SLO)	Write
3.883	Bit 0	SDA=1 (WHI)	Write
3.883	Ack	SDA=0 (SLO)	Write
7.891	12.023	S A AA A 55 A AA A 55 A A	Start
7.891	Ack	SDA=0 (SLO)	Write
12.023	13.026	S AA A 55 A AA A 55 A A	Start
12.023	Bit 6	SDA=0 (SLO)	Write
12.023	Bit 5	SDA=1 (WHI)	Write

The circuit diagram shows the ATMEGA16 microcontroller (U1) connected to the PCF8574 I2C interface (U2). The ATMEGA16 pins are connected as follows:

- RESET (9) to PCF8574 SDA (22)
- XTAL1 (13) to PCF8574 SCL (23)
- XTAL2 (12) to PCF8574 TRIG (24)
- PA0/ADC0 (40) to PCF8574 PC0/SDA (23)
- PA1/ADC1 (39) to PCF8574 PC1/SCL (24)
- PA2/ADC2 (38) to PCF8574 PC2/TMS (25)
- PA3/ADC3 (37) to PCF8574 PC3/TDO (26)
- PA4/ADC4 (36) to PCF8574 PC4/TDI (27)
- PA5/ADC5 (35) to PCF8574 PC5/TDI (28)
- PA6/ADC6 (34) to PCF8574 PC6/TOSC1 (29)
- PA7/ADC7 (33) to PCF8574 PC7/TOSC2 (30)
- PB0/T0/XCK (1) to PCF8574 PD0/RXD (14)
- PB1/T1 (2) to PCF8574 PD1/TXD (15)
- PB2/AIN0/MNT2 (3) to PCF8574 PD2/MNT0 (16)
- PB3/AIN1/OC0 (4) to PCF8574 PD3/MNT1 (17)
- PB4/SS (5) to PCF8574 PD4/OC1B (18)
- PB5/MOSI (6) to PCF8574 PD5/OC1A (19)
- PB6/MISO (7) to PCF8574 PD6/PCP1 (20)
- PB7/SCK (8) to PCF8574 PD7/OC2 (21)
- AREF (32) to PCF8574 AREF (32)
- AVCC (30) to PCF8574 AVCC (30)

The PCF8574 I2C interface (U2) is connected to the I2C bus. The pins are connected as follows:

- SCL (14) to I2C SCL (23)
- SDA (15) to I2C SDA (22)
- INT (13) to I2C TRIG (24)
- A0 (1) to I2C SCL (23)
- A1 (2) to I2C SCL (23)
- A2 (3) to I2C SCL (23)

Figure 4: Proteus Results

Lab Tasks:

Imitate an AVR source code that reads data from the switches connected to IO Expander IC PCF 8574 and assign it to the Port A. Also, analyze the I2C protocol data transfer using I2C debugger.

Note: Attach the program files C Programing code and simulation result snapshots along with the handouts.

```
#include <avr/io.h>
#define F_CPU 8000000UL
#include <util/delay.h>
//--- I2C Initialize ---//
void i2c_init()
{
    TWBR = 0x62;           //Baud rate
    TWCR = (1<<TWEN);      // Enable I2C
    TWSR = 0x00;
}
void i2c_start()
```

```

{
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWSTA);
    //--- Start Conditon
    while (!(TWCR & (1<<TWINT)));
}
void i2c_write(char x)
{
    TWDR = x; //--- Move value to I2C reg
    TWCR = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR & (1<<TWINT)));
}
char i2c_read()
{
    TWCR = (1<<TWEN) | (1<<TWINT);
    while (!(TWCR & (1<<TWINT)));
    return TWDR;
}
int main(void)
{
    DDRA = 0xFF;    //Defining PORT A as output
    i2c_init();    // I2C Initialization
    while(1)
    {
        i2c_start();    //Start condition to slave
        i2c_write(0x41); // Write Slave address to enable it
        PORTA=i2c_read(); // Move data in IO Pin to Variable
        _delay_ms(1000); //delay of 1 sec
    }
}

```

```

17.625us  1.632 s S 41 A 35 N Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 35
1.632 s   3.264 s Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 3
1.632 s   Sr
3.264 s   4.896 s Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 3
3.264 s   Sr
4.896 s   6.528 s Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 3
4.896 s   Sr
6.528 s   8.160 s Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 3
6.528 s   Sr
8.160 s   9.792 s Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 3
8.160 s   Sr
9.792 s   11.424 s Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 3
9.792 s   Sr
11.424 s  12.052 s Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 35 N Sr 41 A 3
11.424 s  Sr

```

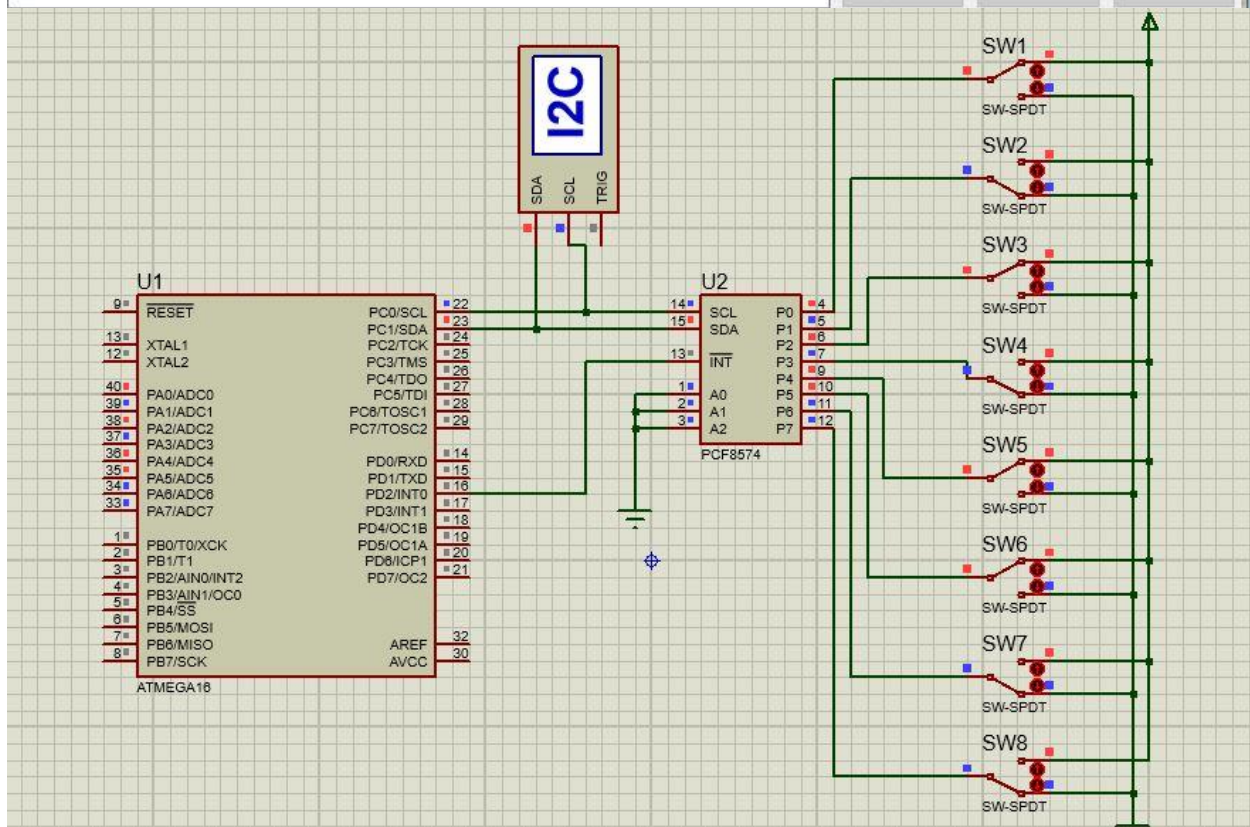
Queued Sequences

Predefined Sequences

Queue

Add

Delete





MEHRAN UNIVERSITY OF ENGINEERING AND TECHNOLOGY, JAMSHORO
DEPARTMENT OF ELECTRONIC ENGINEERING



INTRODUCTION TO EMBEDDED SYSTEMS (ES-314)

Batch: 20ES (5th Semester)

Lab Experiment #12

To Understand the Basics of Arduino Microcontroller Boards

Name	<u>karan</u>	Roll #	<u>20ES062</u>
Signature of Lab Tutor		Date	

RUBRICS:

Performance Metric	CONDUCTING EXPERIMENT	PARTICIPATION	TEAMWORK	USE OF MODERN ENGINEERING TOOLS	DATA ANALYSIS	CALCULATION AND CODING	OBSERVATION/ PROGRAM RESULTS	Total Score
	1	0.5	0.5	1		1	1	05
Obtained								

OBJECTIVE(S)

#	Topic	# Of Lectures	CLO	Taxonomy level
1	The purpose of this lab is to Approach different features of Arduino Microcontroller Board.	3	4,5	P3, A2
2	To write and execute basic programs using Arduino IDE.			

OUTCOME(S)

a. An ability to use the techniques, skills, and modern engineering tools necessary for engineering practice.	PLO5: Modern Tool Usage
b. An ability to function on multi-disciplinary teams	PLO9: Individual and Teamwork

REQUIRED TOOLS AND SOFTWARES:

- Arduino IDE Software
- Arduino UNO board

INTRODUCTION:

Arduino is a tool through which computers can sense and control more of the physical world comparative to a desktop computer. It is an open-source physical computing platform based on a simple microcontroller board, and a development environment for writing software for the board.

Arduino can be used to develop interactive objects, taking inputs from a variety of switches or sensors, and controlling a variety of lights, motors, and other physical outputs. Arduino projects can be stand-alone, or they can communicate with software running on your computer (e.g., Flash, Processing, MaxMSP.) The boards can be assembled by hand or purchased preassembled; the open-source IDE can be downloaded for free.

Advantages of Arduino Technology:

- Inexpensive
- Cross-platform
- Simple, clear programming environment
- Open source and extensible software
- Open source and extensible hardware

Arduino Boards:

Arduino is an open-source platform, many of Arduino users have designed their own customized boards which are compatible with Arduino boards and can be easily fixed on the board.

Arduino Uno:

The Uno differs from all preceding boards in that it does not use the FTDI USB-to-serial driver chip. Instead, it features the Atmega16U2 (Atmega8U2 up to version R2) programmed as a USBto-serial converter.

"Uno" means one in Italian and has been named to mark the upcoming release of Arduino 1.0. The Uno and version 1.0 will be the reference versions of Arduino, moving forward. The Uno is the latest in a series of USB Arduino boards, and the reference model for the Arduino platform.

• Microcontroller	ATmega328
• Operating Voltage	5V
• Input Voltage (recommended)	7-12V
• Input Voltage (limits)	6-20V
• Digital I/O Pins	14 (of which 6 provide PWM output)
• Analog Input Pins	6
• DC Current per I/O Pin	40 mA
• DC Current for 3.3V Pin	50 mA
• Flash Memory	32 KB (ATmega328) with 0.5KB used for bootloader
• SRAM	2 KB (ATmega328)
• EEPROM	1 KB (ATmega328)
• Clock Speed	16 MHz

Other than that, it has a USB connection, a power jack, an ICSP header, and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with an AC-to-DC adapter or battery to get started. The power pins are as follows:

- VIN
- 5V
- 3V3
- GND
- IOREF

The board can be supplied with power either from the DC power jack (7 - 12V), the USB connector (5V), or the VIN pin of the board (7-12V). Supplying voltage via the 5V or 3.3V pins bypasses the regulator and can damage the board therefore it is not advised to use.

The input/output pins are as follows

- Each of the 14 digital pins on the Uno can be used as an input or output. They operate at 5 volts. Each pin can provide or receive a maximum of 40 mA and has an internal pull-up resistor (disconnected by default) of 20-50 k Ω .
- The Uno has 6 analog inputs, labeled A0 through A5, each of which provide 10 bits of resolution (i.e., 1024 different values). By default, they measure from ground to 5 volts, though it is possible to change the upper end of their range using the AREF pin.

In addition, some pins have specialized functions:

- **Serial (0 (RX) and 1 (TX)):** Used to receive (RX) and transmit (TX) TTL serial data. These pins are connected to the corresponding pins of the ATmega8U2 USB-to-TTL Serial chip.
- **External Interrupts (2 and 3):** These pins can be configured to trigger an interrupt on a low value, a rising or falling edge, or a change in value.
- **PWM (3, 5, 6, 9, 10, and 11):** Provide 8-bit PWM output with the analogWrite() function.
- **SPI (10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK)):** These pins support SPI communication using the SPI library.
- **LED (13):** There is a built-in LED. When the pin has HIGH value, the LED glows, when the pin is LOW, it will be off.
- **TWI:** A4 or SDA pin and A5 or SCL **pin**. Support TWI communication using the Wire library.
- **AREF.** Reference voltage for the analog inputs, used with analogReference().
- **Reset.** Bring this line LOW to reset the microcontroller. Typically used to add a reset button to shields which block the one on the board.

Arduino microcontroller boards also use AVR microcontrollers ATMEGA Series. AVR Microcontrollers or all other microcontrollers normally have Ports for connecting devices externally. However, Arduino board has pins not ports.

Arduino Development Environment:

The Arduino development environment contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions, and a series of menus. It connects to the Arduino hardware to upload programs and communicate with them.

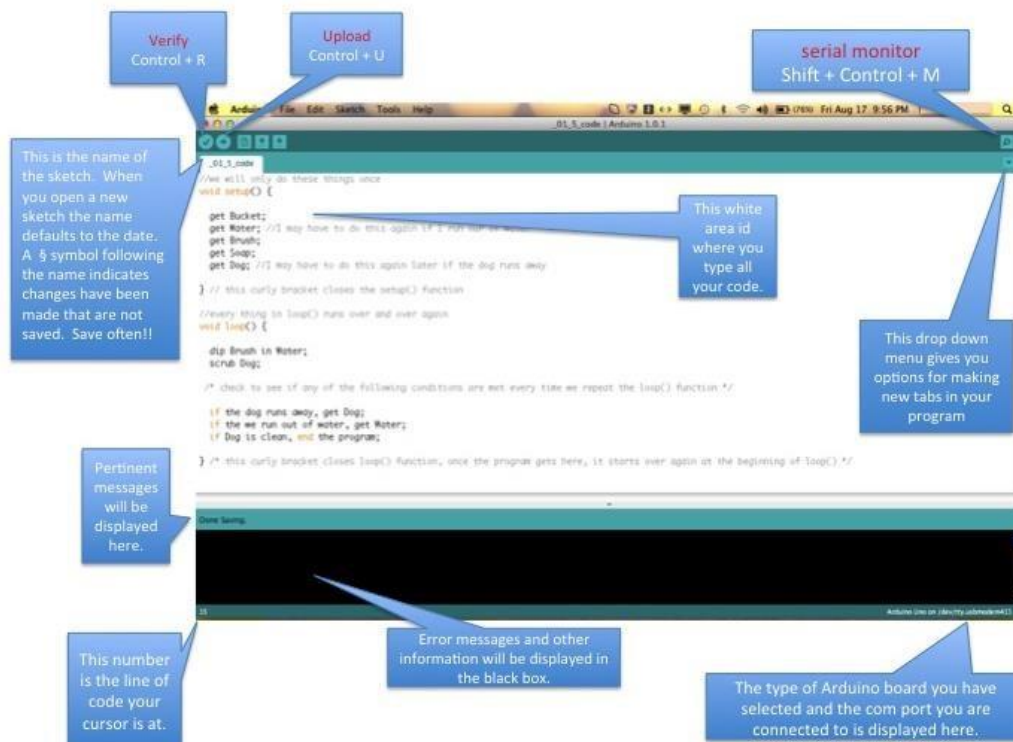


Figure 1: Arduino IDE Features

Table 1: Special Icons in Arduino IDE

	Verify: Checks your code for errors.
	Upload: Compiles your code and uploads it to the Arduino I/O board. Note: If you are using an external programmer, you can hold down the "shift" key on your computer when using this icon. The text will change to "Upload using Programmer"
	New: Creates a new sketch.
	Open: Presents a menu of all the sketches in your sketchbook. Clicking one will open it within the current window. Note: due to a bug in Java, this menu does not scroll; if needed to open a sketch late in the list, File Sketchbook menu can be used instead.
	Save: Saves your sketch.
	Serial Monitor: Opens the serial monitor.

Additional commands are found within the five menus: **File**, **Edit**, **Sketch**, **Tools**, **Help**. The menus are context sensitive which means only those items relevant to the work currently being carried out are available.

Table 2: Menus in Arduino IDE

Edit	Copy for Forum: Copies the code of your sketch to the clipboard in a form suitable for posting to the forum, complete with syntax coloring.
	Copy as HTML: Copies the code of your sketch to the clipboard as HTML, suitable for embedding in web pages.
Sketch	Verify/Compile: Checks the sketch for errors.
	Show Sketch Folder: Opens the current sketch folder.
	Add File...: Adds a source file to the sketch (it will be copied from its current location).
	Import Library: Adds a library to the sketch by inserting #include statements at the start of the code.
Tools	Auto Format: Sets the format of code.
	Archive Sketch: Archives a copy of the current sketch in .zip format. The archive is placed in the same directory as the sketch.
	Board: Select the board that is being utilized.
	Serial Port: This menu contains all the serial devices (real or virtual) on the machine. It should automatically refresh every time when top-level tools menu is opened.
	Programmer: For selecting a hardware programmer when programming a board or chip and not using the onboard USB-serial connection. Normally it will not be required, but if burning a bootloader to a new microcontroller is utilized then this will be required.
	Burn Bootloader: The items in this menu allow to burn a bootloader onto the microcontroller on an Arduino board. This is not required for normal use of an Arduino board but is useful if a new ATmega microcontroller has been purchased (which normally come without a bootloader). Ensure that the correct board from the Boards menu is selected before burning the bootloader.

Working with Arduino IDE:

Sketchbook:

The Arduino environment uses the concept of a sketchbook: a standard place to store the programs (or sketches). The sketches in the sketchbook can be opened from the **File > Sketchbook menu** or from the Open button on the toolbar.

Beginning with version 1.0, files are saved with a .ino file extension. Previous versions use the .pde extension. Still .pde named files in version 1.0 can be opened and later, the software will automatically rename the extension to .ino.

Tabs, Multiple Files, and Compilation:

Allows you to manage sketches with more than one file (each of which appears in its own tab). These can be normal Arduino code files (no extension), C files (.c extension), C++ files (.cpp), or header files (.h).

Uploading:

Before uploading the sketch, the correct items from the **Tools > Board and Tools > Serial Port** menus should be selected. The boards are described below. On

Windows, it is probably COM1 or COM2 (for a serial board) or COM4, COM5, COM7, or higher (for a USB board) - to find out, look for USB serial device in the port section of the Windows Device Manager. Once the correct serial port and board is selected, press the upload button in the toolbar or select the Upload item from the File menu. The Arduino environment will display a message when the upload completes or show an error.

Libraries:

Libraries provide extra functionality for use in sketches, e.g., working with hardware or manipulating data. To use a library in a sketch, select it from the **Sketch > Import Library** menu. This will insert one or more `#include` statements at the top of the sketch and compile the library with the sketch. Since, libraries are uploaded to the board with the sketch, they increase the amount of space it takes up. If a sketch no longer needs a library, simply delete its `#include` statements from the top of the code.

Programming:

Arduino uses C-language to write sketches. The basic instructions are described below:

Digital I/O:

To work with Digital pins, three main functions that are used are:

- pinMode()
- digitalWrite()
- digitalRead()

pinMode()	
Description:	Configures the specified pin to behave either as an input or an output.
Syntax:	pinMode(pin, mode);
Parameters:	Pin: the number of the pin whose mode is required to set
	Mode: <u>INPUT</u> , <u>OUTPUT</u> , or <u>INPUT_PULLUP</u> .
Returns:	None

digitalWrite()	
Description:	Write a HIGH or a LOW value to a digital pin. If the pin is configured as an OUTPUT with pinMode(), its voltage will be set to the corresponding value: 5V (or 3.3V on 3.3V boards) for HIGH, 0V (ground) for LOW.
Syntax:	digitalWrite(pin, value)
Parameters:	Pin: the pin number
	Value: HIGH or LOW
Returns:	None

Task 01: Write a program that continuously makes pin 13 High and Low.

```
int ledPin = 13;           // LED connected to digital pin 13
void setup()
{
  pinMode(ledPin, OUTPUT); // sets the digital pin as output
}

void loop()
{
  digitalWrite(ledPin, HIGH); // sets the LED on
  delay(1000);                // waits for a second
  digitalWrite(ledPin, LOW);  // sets the LED off
  delay(1000);                // waits for a second
}
```

digitalRead()	
Description:	Reads the value from a specified digital pin, either HIGH or LOW.

Syntax:	<code>digitalRead(pin);</code>
Parameters:	Pin: the number of the digital pin that needs to be read (<i>int</i>)
Returns:	HIGH or LOW

Task 02: Write a program that sets pin 13 to the same value as pin 7.

```

int ledPin = 13;           // LED connected to digital pin 13 int
inPin = 7;                 // pushbutton connected to digital pin 7
int val = 0;               // variable to store the read value

void setup()
{
  pinMode(ledPin, OUTPUT);      // sets the digital pin 13 as output
  pinMode(inPin, INPUT);        // sets the digital pin 7 as input }

void loop()
{
  val = digitalRead(inPin);      // read the input pin
  digitalWrite(ledPin, val);     // sets the LED to the button's value }

```

Lab Tasks:

Imitate a source code that make 8 LEDs continuously High, and low connected to the digital pins.

Note: Attach either the proteus simulation or hardware results of labs tasks. Also, attach programing code and simulation result snapshots along with the handouts.

PROGRAM

```

int ledpin1=13; //LED connected to digital pin 13
int ledpin2=12; //LED connected to digital pin 12
int ledpin3=11; //LED connected to digital pin 11
int ledpin4=10; //LED connected to digital pin 10
int ledpin5=9;  //LED connected to digital pin 9
int ledpin6=8;  //LED connected to digital pin 8
int ledpin7=7;  //LED connected to digital pin 7
int ledpin8=6;  //LED connected to digital pin 6
void setup()
{
  pinMode(ledpin1,OUTPUT);
  pinMode(ledpin2,OUTPUT);
  pinMode(ledpin3,OUTPUT);
  pinMode(ledpin4,OUTPUT);
  pinMode(ledpin5,OUTPUT);
  pinMode(ledpin6,OUTPUT);
  pinMode(ledpin7,OUTPUT);
  pinMode(ledpin8,OUTPUT);
}

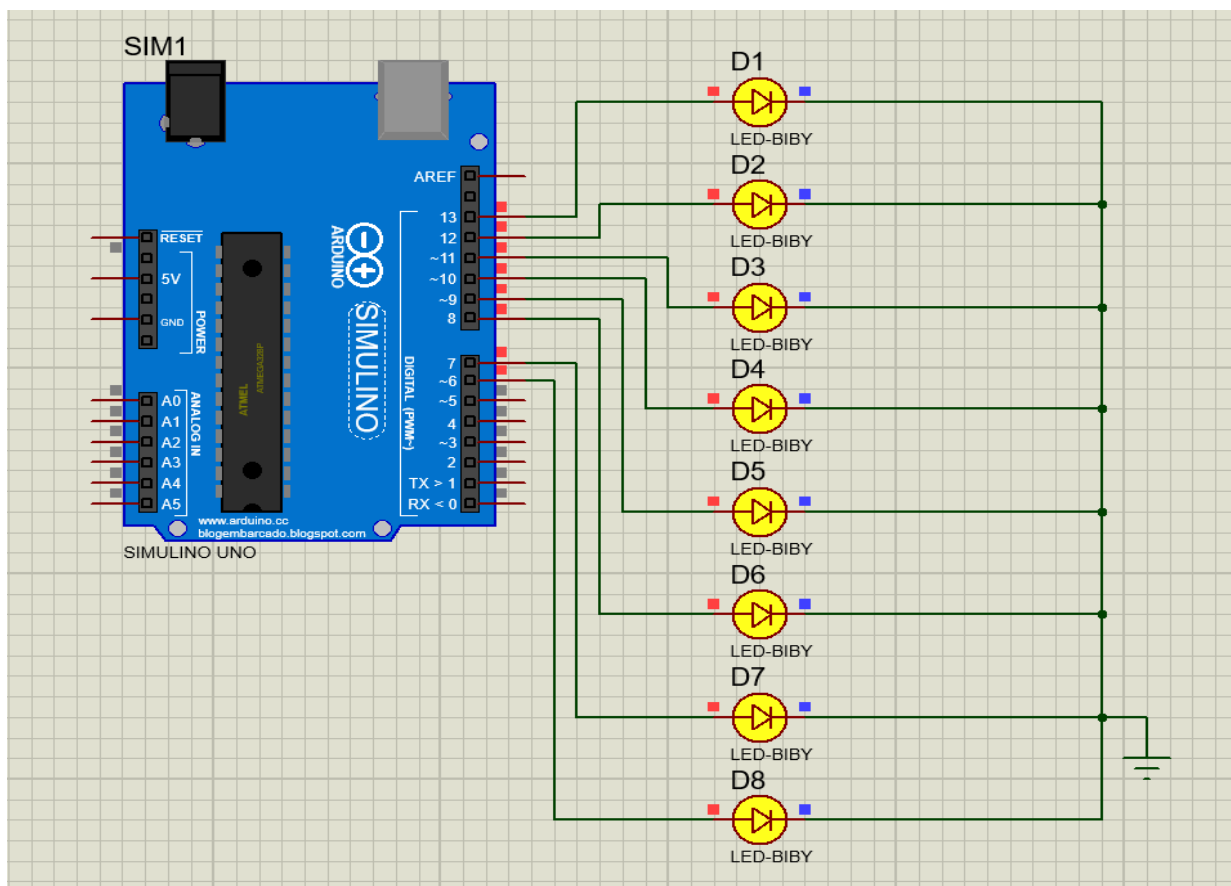
```

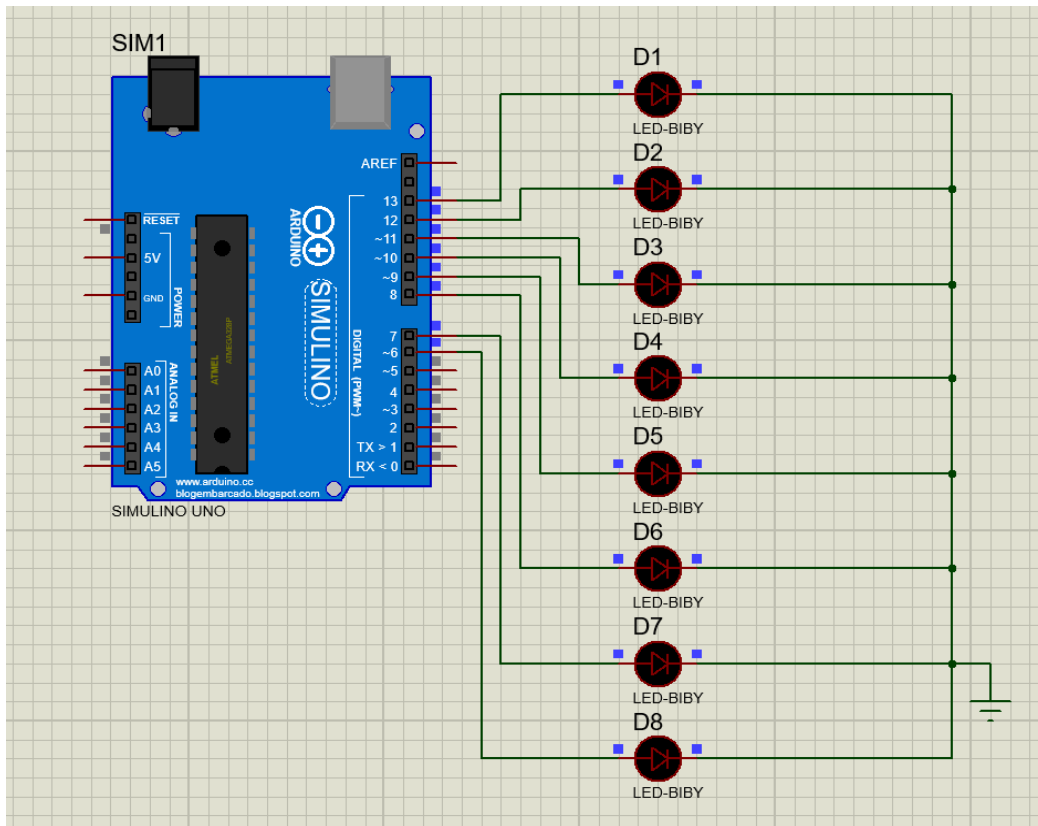


```

void loop()
{
digitalWrite(ledpin1,HIGH);
digitalWrite(ledpin2,HIGH);
digitalWrite(ledpin3,HIGH);
digitalWrite(ledpin4,HIGH);
digitalWrite(ledpin5,HIGH);
digitalWrite(ledpin6,HIGH);
digitalWrite(ledpin7,HIGH);
digitalWrite(ledpin8,HIGH);
delay(1000);
digitalWrite(ledpin1,LOW);
digitalWrite(ledpin2,LOW);
digitalWrite(ledpin3,LOW);
digitalWrite(ledpin4,LOW);
digitalWrite(ledpin5,LOW);
digitalWrite(ledpin6,LOW);
digitalWrite(ledpin7,LOW);
digitalWrite(ledpin8,LOW);
delay(1000);
}

```







MEHRAN UNIVERSITY OF ENGINEERING AND TECHNOLOGY, JAMSHORO
DEPARTMENT OF ELECTRONIC ENGINEERING



INTRODUCTION TO EMBEDDED SYSTEMS (ES-314)

Batch: 20ES (5th Semester)

Lab Experiment #13 & 14

Serial Communication and Analog to Digital Conversion Using Arduino IDE

Name	<u>KARAN</u>	Roll #	<u>20Es062</u>
Signature of Lab Tutor	<u></u>	Date	<u></u>

RUBRICS:

Performance Metric	CONDUCTING EXPERIMENT	PARTICIPATION	TEAMWORK	USE OF MODERN ENGINEERING TOOLS	DATA ANALYSIS	CALCULATION AND CODING	OBSERVATION/ PROGRAM RESULTS	Total Score
	1	0.5	0.5	1		1	1	05
Obtained								

OBJECTIVE(S)

#	Topic	# Of Lectures	CLO	Taxonomy level
1	The purpose of this lab is to approach different features of serial communication in Arduino Microcontroller.	3	4,5	P3, A2
2	To imitate and execute basic programs based on serial communication of Arduino IDE.			
3	To imitate and execute basic programs based on ADC of Arduino IDE.			

OUTCOME(S)

a. An ability to use the techniques, skills, and modern engineering tools necessary for engineering practice.	PLO5: Modern Tool Usage
b. An ability to function on multi-disciplinary teams	PLO9: Individual and Teamwork

REQUIRED TOOLS AND SOFTWARES:

- Arduino IDE Software
- Arduino UNO board

INTRODUCTION TO SERIAL PORT:

It is used for communication between the Arduino board and a computer/other device. All Arduino boards have at least one serial port (also known as a UART or USART). It communicates on digital pins 0 (RX) and 1 (TX) as well as with the computer via USB. Thus, if these functions are utilized then pins 0 and 1 for digital input or output cannot be used.

Arduino environment's built-in serial monitor can be used to communicate with an Arduino board. Click the serial monitor button in the toolbar and select the same baud rate used in the call to `begin()`.

Serial Communication with Arduino Mega:

The Arduino Mega has three additional serial ports: Serial1 on pins 19 (RX) and 18 (TX), Serial2 on pins 17 (RX) and 16 (TX), Serial3 on pins 15 (RX) and 14 (TX). To use these pins to communicate with the computer an additional USB-to-serial adaptor is required, as they are not connected to the Mega's USB-to-serial adaptor. To use them to communicate with an external TTL serial device, connect the TX pin to the device's RX pin, the RX to the device's TX pin, and the ground of the Mega to the device's ground.

Note: Don't connect these pins directly to an RS232 serial port; they operate at +/- 12V and can damage the Arduino board.

begin()	
Description:	Sets the data rate in bits per second (baud) for serial data transmission. For communicating with the computer, use one of these rates: 300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, or 115200. However, other rates can also be specified. For example, to communicate over pins 0 and 1 with a component that requires a particular baud rate, an optional second argument configures the data, parity, and stop bits. The default is 8 data bits, no parity, one stop bit.
Syntax:	<code>Serial.begin(speed)</code>
Parameters:	Speed: in bits per second (baud) - long
Returns:	Nothing

write()	
Description:	Writes binary data to the serial port. This data is sent as a byte or series of bytes. To send the characters representing the digits of a number use the <code>print()</code> function instead.
Syntax:	<code>Serial.write(val)</code> <code>Serial.write(str)</code> <code>Serial.write(buf, len)</code> Arduino Mega also supports: <code>Serial1</code> , <code>Serial2</code> , <code>Serial3</code> (in place of <code>Serial</code>)
Parameters:	val : a value to send as a single byte
	str : a string to send as a series of bytes

	buf: an array to send as a series of bytes
	len: the length of the buffer
Returns:	byte write() will return the number of bytes written, though reading that number is optional. Note: one can also use print() or println() to print data to serial port.

Task 01: Write a program that writes data serially at the baud rate of 9600.

```
void      setup()      {
  Serial.begin(9600);
}
void loop() {
  Serial.write(45); // send a byte with the value 45  int bytesSent = Serial.write("hello");
  //send the string "hello" and return the length of the string. }
```

available()	
Description:	Get the number of bytes (characters) available for reading from the serial port. This is data that is already arrived and stored in the serial receive buffer (which holds 64 bytes).
Syntax:	Serial.available()
Parameters:	None
Returns:	the number of bytes available to read

read()	
Description:	Reads incoming serial data.
Syntax:	Serial.read()
Parameters:	None
Returns:	the first byte of incoming serial data available (or -1 if no data is available) - int

Task 02: Write a program that reads serial data with the baud rate of 9600 and prints the received data.

```
int incomingByte = 0; // for incoming serial data
void setup() {
  Serial.begin(9600); // opens serial port, sets data rate to 9600 bps }
```

```
void loop() {
  // send data only when you receive data:
  if (Serial.available() > 0) {
    // read the incoming byte:
    incomingByte = Serial.read();

    // say what you got:
    Serial.print("I received: ");
    Serial.println(incomingByte, DEC);
  }
}
```

}

Analog I/O

To work with Analog to Digital Converter and PWM:

- analogReference()
- analogRead()
- analogWrite() - PWM

analogReference(type)	
Description:	Configures the reference voltage used for analog input (i.e., the value used as the top of the input range). The options are: Default: the default analog reference of 5 volts (on 5V Arduino boards) or 3.3 volts (on 3.3V Arduino boards) Internal: a built-in reference, equal to 1.1 volts on the ATmega168 or ATmega328 and 2.56 volts on the ATmega8 (not available on the Arduino Mega) Internal 1v1: a built-in 1.1V reference (Arduino Mega only) Internal 2v56: a built-in 2.56V reference (Arduino Mega only) External: the voltage applied to the AREF pin (0 to 5V only) is used as the reference.
Parameters:	Type: which type of reference to use DEFAULT, INTERNAL, INTERNAL1V1, INTERNAL2V56, or EXTERNAL.
Returns:	None

analogRead()	
Description:	Reads the value from the specified analog pin. The Arduino board contains a 6 channel (8 channels on the Mini and Nano, 16 on the Mega), 10-bit analog to digital converter. This means that it will map input voltages between 0 and 5 volts into integer values between 0 and 1023. This yields a resolution between readings of 5volts /1024 units or, 0.0049 volts (4.9mV) per unit. The input range and resolution can be changed using analogReference(). It takes about 100 microseconds (0.0001 s) to read an analog input, so the maximum reading rate is about 10,000 times a second.
Syntax:	analogRead(pin);
Parameters:	Pin: the number of the analog input pin to read from (0 to 5 on most boards, 0 to 7 on the Mini and Nano, 0 to 15 on the Mega)
Returns:	int (0 to 1023)

Task 03: Write a program that reads the value of a potentiometer connected to analog pin 3 with the baud rate of 9600.

```
int analogPin=3;           // potentiometer wiper (middle terminal) connected to analog pin 3
                           // outside leads to ground and +5V
```

```
int val=0;                 // variable to store the value read
```

```

void setup()
{
  Serial.begin(9600);    // setup serial
}
void loop()
{
  val=analogRead(analogPin); // read the input pin
  Serial.println(val);      // debug value }

```

Lab Tasks:

1. Imitate a source code for which consider a potentiometer is connected to an analog input pin while the code reads the voltage and turns ON a specific LED according to the given input:
 If voltage is in between 0 to 2, the LED on pin:13 is turned ON.
 If voltage is in between 2 to 3.5, the LED on pin:14 is turned ON. If
 voltage is in between 3.5 to 5, the LED on pin:15 is turned ON.

```

int analogpin=A1;
int led1=13;
int led2=12;
int led3=11;
int value=0;
void setup() {
  // put your setup code here, to run once:
  pinMode(led1,OUTPUT);
  pinMode(led2,OUTPUT);
  pinMode(led3,OUTPUT);
  Serial.begin(9600);
}
void loop() {
  // put your main code here, to run repeatedly:
  value=analogRead(analogpin);
  if(value<=200)
  {
    digitalWrite(led1,HIGH);
    digitalWrite(led2,LOW);
    digitalWrite(led3,LOW);
  }
  else if(value<=512)
  {
    digitalWrite(led1,LOW);
    digitalWrite(led2,HIGH);
    digitalWrite(led3,LOW);
  }
  else if(value<=1023)
  {

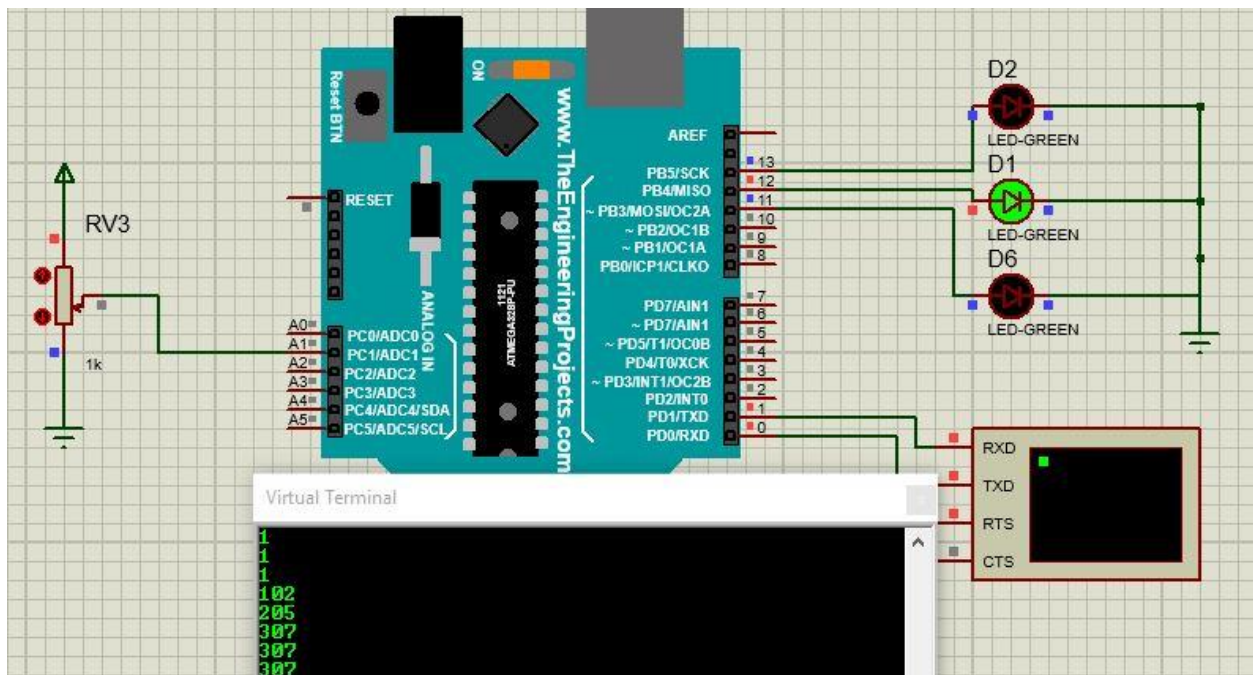
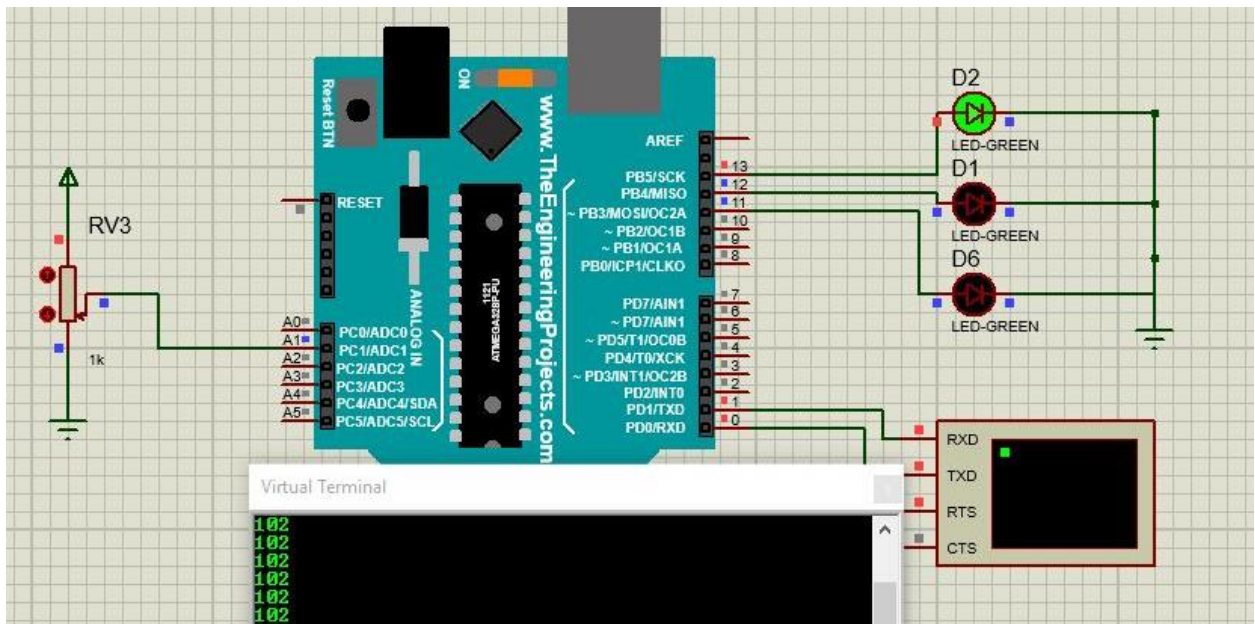
```

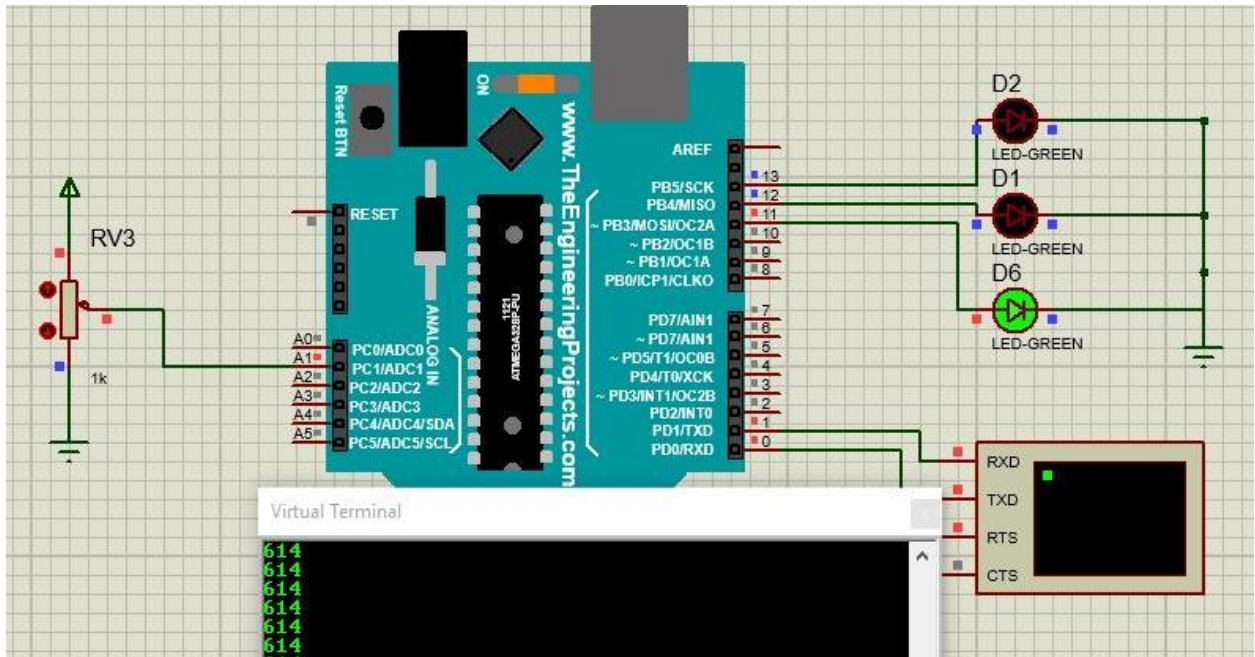


```

digitalWrite(led1,LOW);
digitalWrite(led2,LOW);
digitalWrite(led3,HIGH);
}
Serial.println(value);
delay(100);
}

```





2. Imitate a source code to design a system, that continuously checks Serial data, if data is “FAN”, it should turn on FAN and turn off HEATER, but if received data is “HTR”, the heater should be turn ON and Fan should be OFF.

CODE:

```
int fan=13; //Fan connected to the digital pin 5
int htr=12; //Heater connected to the digital pin 4
unsigned char var[3]; //Variable assigned to save the recieved data
int i=0;
void setup()
{
  pinMode(fan,OUTPUT);
  pinMode(htr,OUTPUT);
  Serial.begin(9600); //opens serial port, sets data rate to 9600 bps
}
void loop()
{
  if(Serial.available()>0)
  {
    //read the incoming data and store it in var[i]:
    var[i]=Serial.read();
    i++;
    if(i==3)
```

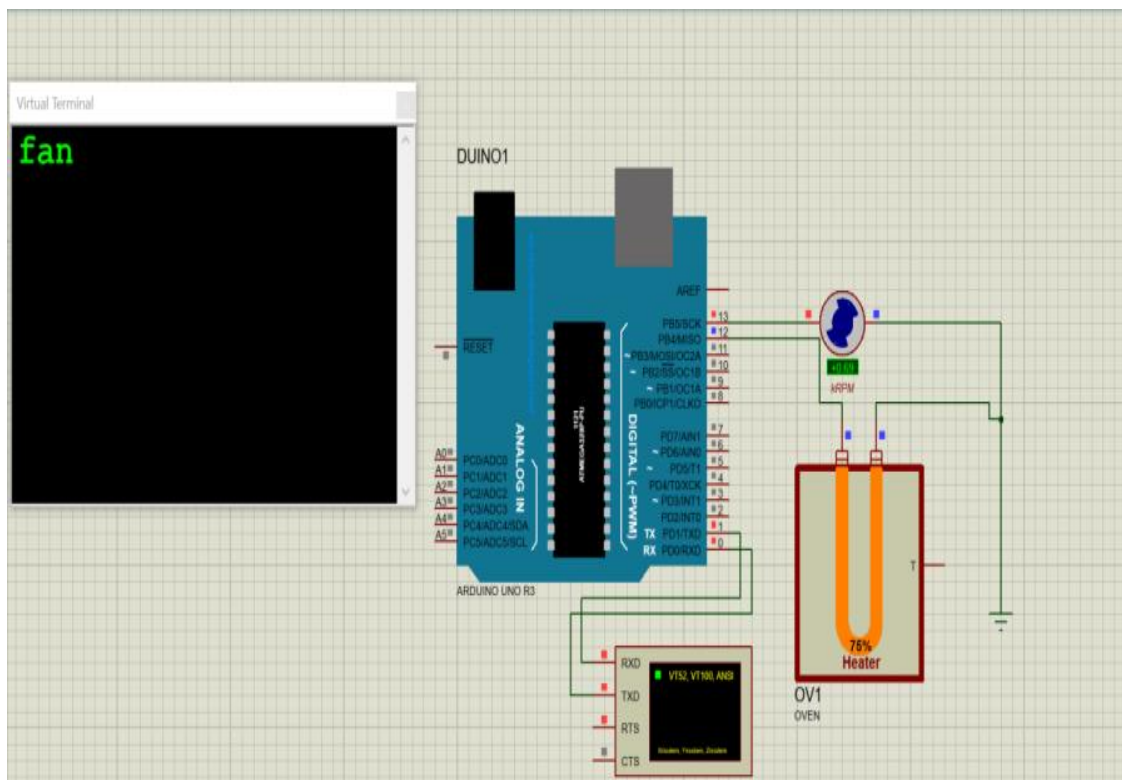
```

{
i=0;
//Condition set according to the different inputs (lowercase data,
uppercase data or first letter upper case and the rest lower case
letters)
if(var[0]=='f' & var[1]=='a' & var[2]=='n'|var[0]=='F' & var[1]=='A' &
var[2]=='N'|var[0]=='F' &
var[1]=='a' & var[2]=='n')

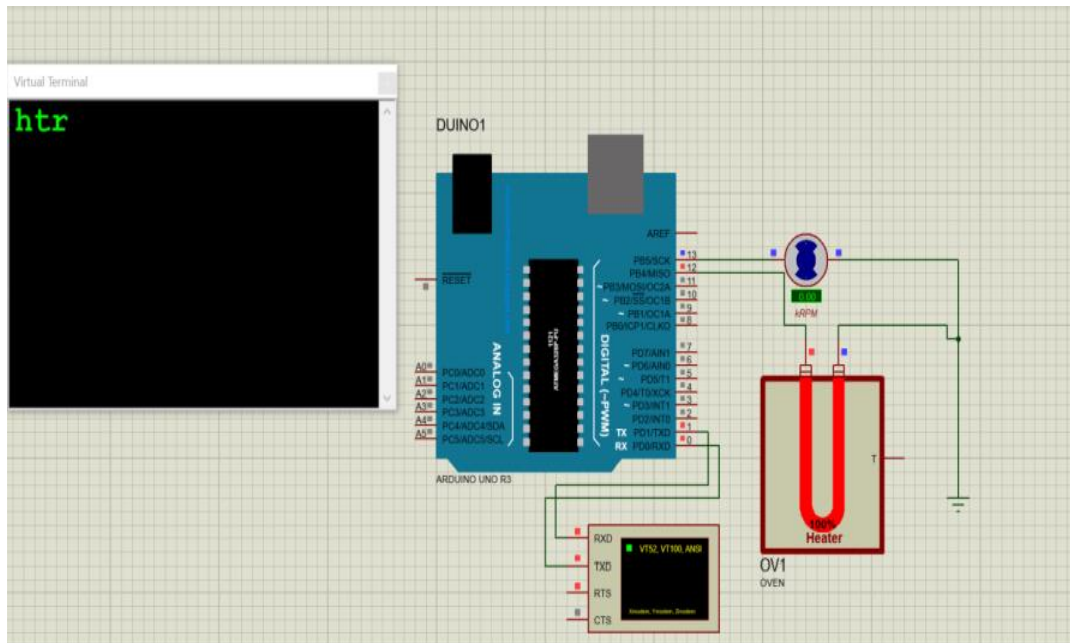
{
digitalWrite(fan,HIGH); //Fan On
digitalWrite(htr,LOW); //Heater Off
}
//Condition set according to the different inputs (lowercase
data,uppercase data or first letter upper case and the rest lower case
letters)
if(var[0]=='h' & var[1]=='t' & var[2]=='r'|var[0]=='H' & var[1]=='T' &
var[2]=='R'|var[0]=='H' &
var[1]=='t' & var[2]=='r')
{
digitalWrite(fan,LOW); //Fan Off
digitalWrite(htr,HIGH); //Heater On
}
}
}
}
}
}

```

- When the received data is 'fan' then it turns the fan ON & heater OFF



- Then the received data is 'htr' then it turns the heater ON & fan OFF.

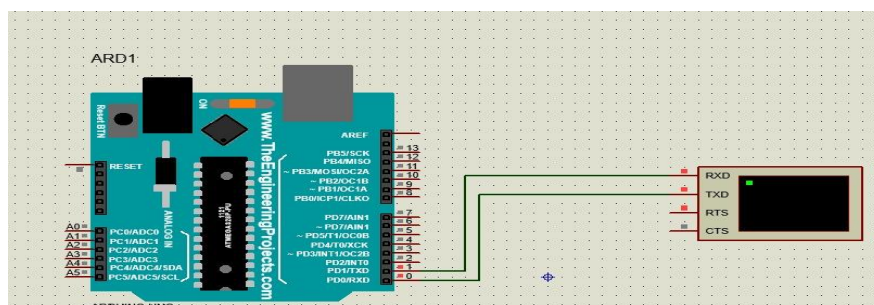


- Imitate a source code that reads data from Serial port and if the data on Serial port is x, your name should be sent over serial port.

Note: Attach either the proteus simulation or hardware results of labs tasks. Also, attach programing code and simulation result snapshots along with the handouts.

CODE

```
unsigned char z;
void setup()
{
  Serial.begin(9600); // Enable the Serial port and set the data rate to 9600 bps
}
void loop()
{
  if(Serial.available() > 0)
  {
    // Read the incoming data and store it in variable z:
    z = Serial.read();
    if(z == '4')
    {
      Serial.println("karan");
    }
  }
}
```



}